THE UNIVERSITY OF SYDNEY

SOFT2201

Software Construction and Design

中文完整复习教材

Week 1-10

设计思想　结构与协作　测试证据

依据 10 份讲义、Ed 补充正文与 Assignment 1-3

含知识地图、双语术语表、43 道每周情境题
及 20 道综合 MCQ、跨章节分析与参考解析

编制日期：2026 年 10 月 22 日

独立学习用教材；非课程官方出版物，练习非实际试题

目录

编制说明与阅读路线 **4**

课程整体知识地图 **5**

第 1 章 Week 1：软件构造的目标与 Python 设计基础 **7**

1.1 核心思想（Core Idea） . . . 7
1.2 历史渊源与问题背景（Origin & Motivation） . . . 7
1.3 重要定义（Definitions） . . . 7
1.4 实际意义（Practical Significance） . . . 8
1.5 完整课件内容（Lecture Reconstruction） . . . 8
1.6 知识关联（Connections） . . . 12
1.7 考试应用（Exam Application） . . . 12

第 2 章 Week 2：OOP 契约、设计原则、UML 与代码审查 **14**

2.1 核心思想（Core Idea） . . . 14
2.2 历史渊源与问题背景（Origin & Motivation） . . . 14
2.3 重要定义（Definitions） . . . 14
2.4 实际意义（Practical Significance） . . . 15
2.5 完整课件内容（Lecture Reconstruction） . . . 15
2.6 知识关联（Connections） . . . 22
2.7 考试应用（Exam Application） . . . 22

第 3 章 Week 3：质量属性、最少原则与 Factory Method **24**

3.1 核心思想（Core Idea） . . . 24
3.2 历史渊源与问题背景（Origin & Motivation） . . . 24
3.3 重要定义（Definitions） . . . 24
3.4 实际意义（Practical Significance） . . . 25
3.5 完整课件内容（Lecture Reconstruction） . . . 25
3.6 知识关联（Connections） . . . 30
3.7 考试应用（Exam Application） . . . 30

第 4 章 Week 4：Composite 的递归统一与 Command 的操作对象 **31**

4.1 核心思想（Core Idea） . . . 31
4.2 历史渊源与问题背景（Origin & Motivation） . . . 31
4.3 重要定义（Definitions） . . . 31
4.4 实际意义（Practical Significance） . . . 32
4.5 完整课件内容（Lecture Reconstruction） . . . 32
4.6 知识关联（Connections） . . . 37
4.7 考试应用（Exam Application） . . . 38

第 5 章 Week 5：Decorator、Proxy 与 Singleton **39**
5.1 核心思想（Core Idea） . . . 39
5.2 历史渊源与问题背景（Origin & Motivation） . . . 39
5.3 重要定义（Definitions） . . . 39
5.4 实际意义（Practical Significance） . . . 40
5.5 完整课件内容（Lecture Reconstruction） . . . 40
5.6 知识关联（Connections） . . . 46
5.7 考试应用（Exam Application） . . . 46

第 6 章 Week 6：幂等、Adapter 与 Facade **47**
6.1 核心思想（Core Idea） . . . 47
6.2 历史渊源与问题背景（Origin & Motivation） . . . 47
6.3 重要定义（Definitions） . . . 47
6.4 实际意义（Practical Significance） . . . 48
6.5 完整课件内容（Lecture Reconstruction） . . . 48
6.6 知识关联（Connections） . . . 53
6.7 考试应用（Exam Application） . . . 53

第 7 章 Week 7：测试基础、RIPR 与白盒覆盖 **54**
7.1 核心思想（Core Idea） . . . 54
7.2 历史渊源与问题背景（Origin & Motivation） . . . 54
7.3 重要定义（Definitions） . . . 54
7.4 实际意义（Practical Significance） . . . 55
7.5 完整课件内容（Lecture Reconstruction） . . . 55
7.6 知识关联（Connections） . . . 60
7.7 考试应用（Exam Application） . . . 60

第 8 章 Week 8：黑盒测试、时间边界与 Iterator **62**
8.1 核心思想（Core Idea） . . . 62
8.2 历史渊源与问题背景（Origin & Motivation） . . . 62
8.3 重要定义（Definitions） . . . 62
8.4 实际意义（Practical Significance） . . . 63
8.5 完整课件内容（Lecture Reconstruction） . . . 63
8.6 知识关联（Connections） . . . 70
8.7 考试应用（Exam Application） . . . 70

第 9 章 Week 9：责任链、Observer 与 Strategy **71**
9.1 核心思想（Core Idea） . . . 71
9.2 历史渊源与问题背景（Origin & Motivation） . . . 71
9.3 重要定义（Definitions） . . . 71
9.4 实际意义（Practical Significance） . . . 72
9.5 完整课件内容（Lecture Reconstruction） . . . 72
9.6 知识关联（Connections） . . . 78
9.7 考试应用（Exam Application） . . . 78

第 10 章 Week 10：Template Method、控制反转与模式全景 **80**
10.1 核心思想（Core Idea） . . . 80
10.2 历史渊源与问题背景（Origin & Motivation） . . . 80
10.3 重要定义（Definitions） . . . 80
10.4 实际意义（Practical Significance） . . . 81
10.5 完整课件内容（Lecture Reconstruction） . . . 81
10.6 知识关联（Connections） . . . 86
10.7 考试应用（Exam Application） . . . 86

第 11 章 核心概念中英文对照 **88**
11.1 语言、对象与设计原则 . . . 88
11.2 UML 与设计模式 . . . 89
11.3 测试与验证 . . . 91

第 12 章 考前综合复习与跨章节训练 **93**
12.1 从十周内容建立答题方法 . . . 93
12.2 Section A：20 道情境 MCQ . . . 93
12.3 Section B：跨章节设计审查 . . . 96
12.4 Section C：模式选择、结构与比较 . . . 97
12.5 Section D：软件测试综合分析 . . . 98
12.6 综合复习的最后检查 . . . 101

第 13 章 资料索引、覆盖核对与扩展依据 **102**
13.1 本地资料及归属依据 . . . 102
13.2 补读的 Ed 课程正文 . . . 103
13.3 扩展资料的精确用途 . . . 103
13.4 教材自身的验证边界 . . . 104
13.5 阅读完后应能够完成的任务 . . . 104

编制说明与阅读路线

本书面向已掌握 Python、Java OOP、数据结构和基础软件工程、但未参加课堂教学的读者。它依据当前资料夹中的十份讲义，重构知识的动机、定义、结构、执行过程和权衡，并补读讲义明确跳转的 Ed 教学正文。它不是逐页翻译，也不以预测考题代替完整学习。

资料边界。本地十份讲义均可读取，合计 658 页；没有发现 PPT/PPTX 原文件。另有 Assignment 1-3 的说明、编码标准、代码和测试。课件里的 Ed Python、OOP、Factory Method、Composite 与 Iterator 正文已用于补足 PDF 中的跳转内容。Assignment 4 原卷不在本地资料夹；已核对 Ed 的 Introduction 和 FAQs，确认 Section A 为 20 道 MCQ，B、C 为设计原则与模式简答，D 为软件测试。本书不提供原卷答案，所有练习均为自编。

周次依据。以 PDF 封面的 Lecture 编号、实际内容和 Ed 周次目录交叉核对。课程目录有些 Tutorial 名称相对于讲课晚一周，Ed 中另有标为 Week 11 的 Template Method 页面；本书按本地 Lecture 10 的实际内容把 Template Method 归入 Week 10，不把这些标签差异当作额外的一周课。

内容标识。未另行标注的概念与案例主线来自课程；“教学重写”表示保留原教学目标而修正或简化代码；“扩展”表示为理解增加的边界说明、背景或工程建议。原课件中的示意代码、笔误、简化说法不会不加判断地复制为正式定义。新闻案例按其课堂用途转述，不断言其中截图描述仍是当前事实。

使用方法。首次阅读按周次完成正文，再用知识关联回看前面的原则。每个模式都应能回答：变化在哪里、谁承担职责、调用怎样经过各角色、代价是什么。每个测试都应能回答：依据什么规格、执行什么路径、断言什么行为、仍遗漏什么。最后独立完成综合练习，再阅读解析。

期末完整范围可能包含 Week 11-13 的后续讲授；Lecture 10 的预告列出接口、类型、泛型、调试和进一步代码审查。本地没有对应讲义，本书不凭预告扩写为“已经覆盖”。本书完整覆盖的是所提供的 Week 1-10，不代表未知后续资料已被纳入。

课程整体知识地图

课程的技术主线是：理解语言与对象语义，建立质量判断准则，把变化封装到适当职责中，再通过测试获得可审查的行为证据。

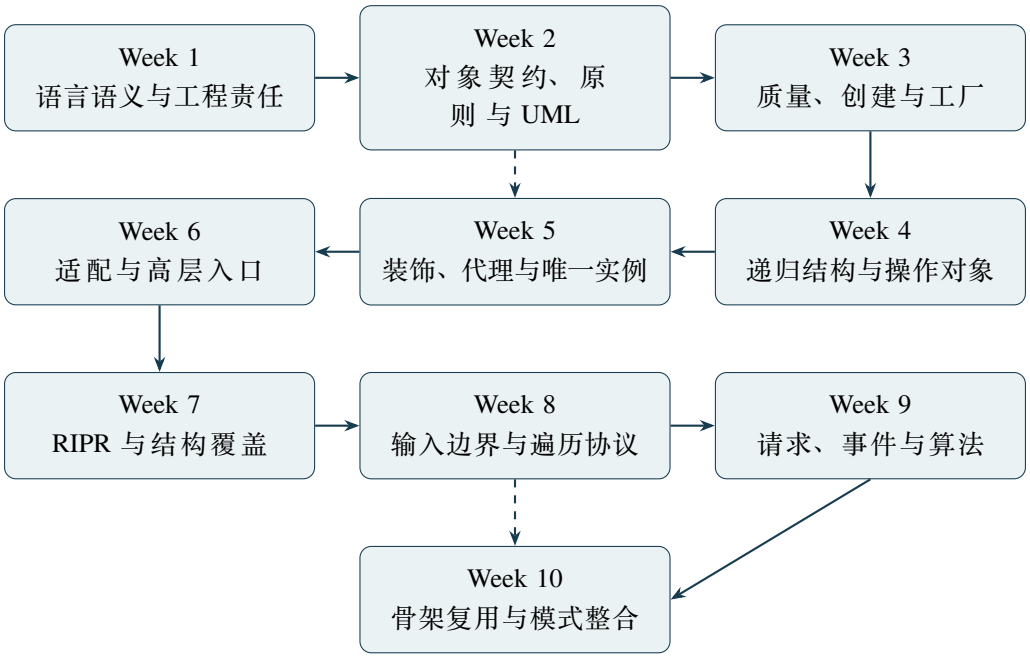


图 1: 课程知识地图：实线为阅读主线，虚线为跨周联系。

周次	本地讲义与页数	实际教学主线
Week 1	Lecture1.pdf，44 页	工程责任、设计动机、Python 语义与惯用法
Week 2	UML_and_code_review.pdf，69 页	OOP、设计原则、类图、顺序图、代码审查
Week 3	designpatterns.pdf，37 页	质量属性、最少原则、对象创建、Factory Method
Week 4	composite_and_command.pdf，69 页	Composite、Command、递归、撤销
Week 5	decorators_proxies_singletons.pdf，77 页	Decorator、Proxy、Singleton、延迟初始化
Week 6	adapter_and_facade.pdf，60 页	幂等、Adapter、Facade、边界契约
Week 7	whitebox_testing.pdf，68 页	测试基础、RIPR、白盒覆盖、充分性
Week 8	blackbox_testing.pdf，73 页	黑盒设计、日期时间、Iterator、生成器
Week 9	chain_observer_and_strategy.pdf，82 页	Chain of Responsibility、Observer、Strategy
Week 10	template_and_recap.pdf，79 页	Template Method、Hook、控制反转、模式比较

第一条因果链：封装使修改局部化。 Week 2 说明暴露内部结构会把调用者绑在实现上。Week 3 隔离创建，Week 6 隔离边界差异，Week 8 隔离遍历。它们都减少“一个内部变化迫使多少客户修改”。

第二条因果链：职责分配决定可测试性。直接依赖具体数据库的类难以隔离；依赖抽象并接受注入后，测试可替换协作者。Command 把操作保存为对象，Observer 把反应从状态主体中移出，Strategy 把算法从上下文中移出。每次分离同时增加了需要检验的接口契约。

第三条因果链：结构证据与行为证据互补。UML 描述设计意图，审查检查实现与规则，白盒测试检查执行到了哪里，黑盒测试检查输入空间与需求边界。没有一项能独立证明系统整体正确。

第四条因果链：模式之间的相似结构不等于相同目的。 Decorator、Proxy、Adapter 和责任链都可能持有另一个对象并转发调用。辨认模式时必须先看所解决的问题，再看结构；Week 10 会从变化位置和调用控制权重新组织这 13 种模式。

第 1 章　Week 1：软件构造的目标与 Python 设计基础

来源：L1 第 1-44 页；Ed「Week 1: Brushing up on Python」全部 13 页。Python 细节是后续设计与测试的先修知识；Ed 明确说明该复习页本身不直接作为考试内容。本章仍完整讲解其中与课程相关的语义，避免读者在后续追踪代码时失去依据。

1.1　核心思想（Core Idea）

软件构造的目标不是尽快得到一个能执行的程序，而是构造一个人能够理解、修改和验证的系统。软件工程师的大量工作发生在已有代码上：读懂设计、找出问题、加入变化，并证明原有行为得到保留。程序能通过几个测试，只说明它在这些条件下表现符合预期，不能替代设计质量。

本周要形成两种能力：第一，解释为什么可读性、可维护性和责任归属属于工程要求；第二，准确追踪 Python 的控制流、对象共享和函数行为。已有 Java OOP 基础的读者应把注意力放在语言差异及其设计后果上。

1.2　历史渊源与问题背景（Origin & Motivation）

课件以航天、医疗、基础设施和互联网服务的事故说明：软件的局部错误可能通过系统依赖扩大为严重后果。Ariane 5 的数值转换问题、Therac-25 的并发问题、北美停电中的报警软件问题，分别提醒我们检查数值范围、执行次序与系统级反馈。L1 还展示自动驾驶、航空与通信故障的新闻案例。它们的教学作用是建立工程责任意识，不是要求背诵新闻数字或把复杂事故归因于单一技术。

早期完成一次任务的程序，可能由作者自己长期维护；当软件持续演进、多人协作时，“作者自己知道是什么意思”便不再足够。L1 的 code golf 案例把代码压到极短，却使定位错误非常困难。因此，简短不是清晰的充分条件，设计成本也应包括未来理解和修改的成本。

AI 生成代码改变了输入代码的方式，却没有转移提交与部署的责任。课件批评三种反模式：Prompt-and-pray（不思考问题而寄希望于提示词）、Cognitive offloading（把关键思考完全外包）和 Loss of ownership（不再认真审查生成内容）。校准信任（Calibrated trust）意味着依据理解、审查与测试接受结果；盲目信任和一概拒绝都不能代替判断。有关招聘趋势和模型能力的截图是课堂讨论材料，不作为本教材的现状结论。

1.3　重要定义（Definitions）

Software engineering ／软件工程：以系统化的工程方法开发、运行和维护软件。这里强调软件生命周期中的协作、质量和变化，而非仅指编码。

Software design ／软件设计：在不必完成全部实现的前提下，决定系统的结构、职责、接口及协作方式。代码从来不会“没有设计”；缺少主动设计时，仍会形成由局部决定积累的默认设计。

Maintainability ／可维护性：软件在理解、修正、适应和扩展时所具有的便利程度。运行正确但职责混乱的代码仍可能难以维护。

Refactoring ／重构：在保留约定的外部可观察行为的条件下，改进代码内部结构。新增功能本身不是重构；实际任务可以同时包含重构与功能扩展，但应区分验证目标。

Python idiom ／ Python 惯用法：符合语言机制与常用表达习惯的写法，例如 `enumerate`、推导式和 `with`。惯用法应改善表达，不应追求难以读懂的一行代码。

Dynamic typing ／动态类型：对象具有类型，名字在运行时绑定到对象，不必像 Java 局部变量声明那样固定一个静态类型。类型注解（Type hint）提供说明和静态分析依据，通常不会自动阻止运行时传入其他类型。

Value equality ／值相等：`==` 比较两个对象按照相等性协议定义的值；**Object identity ／对象身份**：`is` 判断是否是同一个对象。两个独立创建、内容相同的列表可以值相等而身份不同。

Mutability ／可变性：对象创建后，其状态能否原地改变。给名字重新赋值与修改原对象是两件事。`tuple` 不可改变元素引用的排列，但其中引用的列表仍可能被修改。

First-class function ／一等函数：函数能像普通值一样被赋值、传参和返回。**Late binding ／晚绑定**：闭包中的自由变量一般在调用时查找，而不是自动保存每次循环时的值。

Duck typing ／鸭子类型：调用方关注对象是否支持所需操作及语义，而不强制其继承某个指定类。“碰巧有同名方法”不能保证行为契约相容。

1.4 实际意义（Practical Significance）

课程要求的读代码能力，首先是把事实与暗示分开。注释可能过时，变量名可能误导，测试可能只覆盖少数输入。阅读一个导入函数时，应追踪它调用了什么、改变了什么、返回了什么，以及异常由谁处理，而不是只看名字。

Python 中的对象共享是维护风险的重要来源。配置、默认参数、返回的内部列表和模块级数据库连接都可能把本应独立的调用耦合在一起。修复这些问题不一定需要一个模式；有时把默认值改成 `None`、显式检查缺失值或复制集合就已经足够。

课件建议记录日常遇到的软件错误：应用与操作条件、预期结果、实际结果。这个记录方法同时训练需求理解、测试判定与缺陷报告。扩展使用时，可以补充最小复现步骤和环境；它们属于本教材的实践建议，而非原课件新增要求。

1.5 完整课件内容（Lecture Reconstruction）

1.5.1 从“能运行”到“能长期修改”

L1 以“写代码不是软件工程师的主导活动”为起点，连接三个课程任务：理解陌生代码、讨论其潜在问题、实施可验证的重构。设计的作用是让变化局部化，使开发者不必理解整个系统才能修改一个职责。

例如，多个界面分别硬编码所有支付方式，看起来今天能运行，但增加一种支付方式就会触及多处代码。后续 Factory Method、Strategy 等模式提供不同的职责分配方法。此时不应急于选择模式；先明确变化的是对象创建、算法还是外部接口。

L1 的“越早发现问题，修复成本往往越低”应理解为一般工程动机。它不提供所有项目通用的固定倍数。需求误解若在发布后才被发现，通常还会引入部署、数据修复、兼容性和用户沟通成本。

1.5.2 Python 控制流：看执行条件，不看视觉直觉

`if/elif/else` 从上到下选择第一个满足的分支；独立的两个 `if` 则可能都执行。`for` 遍历可迭代对象；`range(0, 5)` 产生 0 到 4。`enumerate` 同时提供索引与元素，通常比手动维护索引更直接。

循环的 `else` 属于循环本身：仅在循环正常耗尽、没有执行 `break` 时运行。它不是“最后一次条件为假时的 if-else”。空序列也会正常耗尽，因此可能立即执行循环的 `else`。

PYTHON

```python
def contains_software(names):
    for name in names:
        if "Software" in name:
            return True
    else:
        return False

assert contains_software(["Software Engineering"])
assert not contains_software([])
```

这个例子使用 `return` 提前返回，也可用 `break` 表示找到目标。读者必须区分退出循环与退出整个函数。`continue` 跳过本次循环的剩余部分；在计数程序中，它放在错误的条件之后，可能使程序实际统计“失败者”而非“通过者”。

`and` 和 `or` 采用短路求值：`x and y` 在 `x` 为假时不求值 `y`；`x or y` 在 `x` 为真时不求值 `y`。因此，右侧函数的副作用不一定发生。扩展语义澄清：Python 的这两个运算符返回某个操作数，不保证返回 `bool`。例如 `[] or [1]` 的结果是 `[1]`。

1.5.3 异常是控制流的一部分

`raise` 发出异常；`except` 按声明顺序寻找匹配的异常类型；没有匹配处理器时，异常沿调用栈传播。`finally` 用于正常执行及通常的异常退出路径上的清理。“总会执行”是课堂简写，不应理解为进程被强制终止后仍能运行清理代码。

`try` 的 `else` 在 try 主体没有抛出异常、没有提前退出时执行，可避免把成功后的其他操作也错误纳入同一捕获范围。不要把所有错误转换成一个看似正常的空列表；否则调用者无法区分“成功读取空数据”与“根本没读到数据”。Week 2 和 Assignment 1 会把这一问题用于跨文件追踪。

PYTHON

```python
import json

def load_records(path):
    try:
        with open(path, encoding="utf-8") as handle:
            records = json.load(handle)
    except OSError as error:
        raise RuntimeError("Cannot read question file") from error
    else:
        if not isinstance(records, list):
            raise ValueError("Expected a JSON array")
```

```
        return records
```

这是按课堂原则重写的教学例：文件错误被明确转换并保留原因，JSON 语法错误继续传播。这里没有吞掉未知错误，也没有把失败伪装成空数组。

1.5.4 类型、转换和集合

Ed 列举 `int`、`float`、`bool`、`str` 和 `NoneType`，并演示 `str(40)`、`float("3.14")` 等转换。转换可能失败；把用户输入当作已验证数据会把接口假设扩散到整个程序。

`list` 是可变的有序集合；`tuple` 是不可变序列；`set` 去重且不应依赖其遍历次序表达业务规则；`dict` 表示唯一键到值的映射。字典重复赋同一个键会替换其值，而不是保存两条独立记录。课程后续利用字典明确表示“格式到解码器”的关系。

类型注解不等于运行时校验。给 `multiply(x: int, y: int)` 传字符串，在没有主动校验时仍可能执行字符串重复。类型检查器与运行时测试检查不同性质，两者都不应被误当作完备证明。

1.5.5 身份、值、真值与别名

PYTHON

```python
a = [1, 2]
b = [1, 2]
c = a
assert a == b and a is not b
c.append(3)
assert a == [1, 2, 3]
assert b == [1, 2]

value = 0
assert not value
assert value is not None
```

`a` 与 `c` 是同一对象的别名（Alias）；`b` 是独立对象。假值包括 `None`、`False`、数值零和空集合，但业务含义可能不同。若零是合法分数，`if not score` 不能作为“未提交”的判断；应依据契约检查 `score is None`，再单独验证范围。

字符串池化和小整数复用属于实现优化，不能用 `is` 替代字符串或数值的 `==`。Week 3 会进一步讨论对象复用的动机。

1.5.6 函数对象、闭包与默认参数

一等函数使行为可以注入：`apply_twice(f, x)` 返回 `f(f(x))`。这为 Week 9 的函数式 Strategy 和回调提供语言基础。`lambda` 仅适合简短表达式；把复杂业务塞入匿名函数会削弱可读性。

PYTHON

```python
wrong = [lambda: i for i in range(3)]
assert [f() for f in wrong] == [2, 2, 2]

fixed = [lambda i=i: i for i in range(3)]
assert [f() for f in fixed] == [0, 1, 2]
```

第一组函数共享同一自由变量绑定，循环结束后调用都看到 2。第二组利用默认参数在创建每个函数时绑定当时的值。也可用一个显式的函数工厂创建独立闭包；修复方法应让读者理解绑定时机。

PYTHON

```
def append_bad(items=[]):
    items.append(1)
    return items

first = append_bad()
second = append_bad()
assert first is second
assert first == [1, 1]

def append_good(items=None):
    if items is None:
        items = []
    items.append(1)
    return items

assert append_good() == [1]
assert append_good() == [1]
```

默认参数表达式在执行 `def` 时求值一次。错误不是“列表不能传入函数”，而是多次省略参数时复用了同一默认对象。若调用者显式传入列表，函数是否允许修改它仍需另外规定。

1.5.7 惯用法与资源管理

列表推导式 `[x*x for x in values]` 和字典推导式 `{x: x*x for x in values}` 把映射关系直接表达出来。它们适合短、清楚的变换；多层嵌套或复杂副作用不应仅为节省行数而使用。

`with open(...) as handle` 将资源的进入、退出协议写在结构中，通常可以在正常及异常路径上关闭文件。不要把 `list`、`dict` 等内建名称用于普通变量；`list = [...]` 后再调用 `list("123")` 会尝试调用列表对象，产生类型错误。

Ed 以 `Person` 对象和支持同一 `introduce()` 方法的 `Robot` 引出鸭子类型。重点不是类的名字，而是调用方依赖哪一组能力。后续将用抽象基类（ABC）把这种契约显式化。

1.5.8 课程组织与学习方式

原始讲义安排了 Early Feedback Task（2%）、Week 3-10 周测（合计 8%）、四个各 10% 的作业及 50% 的 Final Exam。该分配是资料中的课程安排记录；实际提交和考场要求以课程当前正式通知为准。

四次作业依次训练代码审查、设计模式重构、测试充分性、Trial Exam。它们共同要求解释依据，而不是只给出运行结果。课件用间隔练习（Spaced practice）组织周测与回顾；本教材各周的情境题也应间隔重做，并解释自己为何排除其他方案。

1.5.9 成绩分析示例：少量测试的证据边界

L1 的 analyze_scores 将 ≥ 90 归 A，80-89 归 B，其余归 F；A/B 计为通过，至少 60% 通过时返回 Majority passed。这里的阈值仅属于课堂小程序，不代表大学正式等级规则。只测 [90] 与 [0] 会遗漏 B 分支、混合输入、60% 边界和空集合。

PYTHON

```python
def analyze_scores(scores):
    total = 0
    passed = 0
    for score in scores:
        if score >= 90:
            grade = "A"
        elif score >= 80:
            grade = "B"
        else:
            grade = "F"
        if grade == "A" or grade == "B":
            passed += 1
        total += 1
    if total != 0 and passed / total >= 0.6:
        return "Majority passed."
    return "Majority failed."

assert analyze_scores([85]) == "Majority passed."
assert analyze_scores([90, 85, 80, 0, 0]) == "Majority passed."
assert analyze_scores([90, 85, 0, 0, 0]) == "Majority failed."
assert analyze_scores([]) == "Majority failed."
```

按该示例，空集进入 Majority failed，且短路避免除零；真实需求是否应改为“无成绩”要另外定义。这个小程序把语言控制流、规格阈值与后续测试充分性连接起来。

1.6 知识关联（Connections）

名字绑定、共享可变状态和异常传播直接决定 Week 2 的封装与审查结论；一等函数、鸭子类型和 ABC 支撑后续模式的多种实现；短路控制流与循环语义决定 Week 7 的覆盖分析；输入转换与边界假设则成为 Week 8 的黑盒测试对象。

由此可见，语言机制不是孤立语法题。一个默认列表可以制造跨测试污染，一个昂贵的属性访问可以隐藏数据库查询，一个被吞掉的异常可以破坏测试的可观察性。

1.7 考试应用（Exam Application）

掌握要求与常见错误：能追踪共享对象、短路求值和异常；能解释“通过测试”与“设计良好”之间的区别。不要把惯用法当作越短越好，不要用身份比较业务值，也不要把课件新闻截图转化为考试结论。以下均为自编练习。

情境题 1：报告串台。函数默认参数是 `errors=[]`，每次返回包含该列表的报告。测试 A 添加一个错误，测试 B 新建报告却读到这个错误。为什么单独运行 B 可能成功？如何修复？

解析。默认列表在函数定义时创建并在省略实参的调用间共享，A 改变了 B 可见的状态。使用 `None` 并在函数内部创建新列表。若错误对象本身也可变，还需规定是否复制元素；仅改默认参数不能自动解决所有别名问题。

情境题 2：生成按钮。循环为三个按钮设置 `lambda: run(i)`，点击任意按钮都执行最后一项。缺陷发生在按钮点击时还是创建时？为什么 `lambda i=i: run(i)` 有效？

解析。创建阶段建立了对共享自由变量的引用；调用阶段才查找该变量，因而读到循环结束值。默认参数在每次创建函数时求值，将该次的 i 保存在不同函数的参数默认值中。

情境题 3：AI 提交。生成代码通过两个正常输入测试，开发者因此不审查它。请从本课程的角度指出尚缺的证据。

解析。尚不清楚错误输入、边界、异常和副作用是否符合契约，也不清楚新增职责、依赖和扩展方式是否合理。应读懂变更、检查职责与接口，再选择与风险对应的测试。不能用测试数量直接推导维护性或正确性。

情境题 4：空值与零。系统约定 `None` 表示未提交，0 表示提交后得零分。`if not score: return "missing"` 有什么问题?

解析。它把两种不同业务状态合并了。应先用 `is None` 检查缺失，再验证类型与分数范围。测试至少覆盖 None、0、一个正数和约定的非法输入。

第 2 章　Week 2：OOP 契约、设计原则、UML 与代码审查

来源：L2 第 1-69 页；Ed「Week 2: Recap of OOP」全部正文；Assignment 1 说明、CODING_STANDARD.md 与实际代码。本章把先修 OOP 语义、设计判断准则和设计交流方法连为一个过程。

2.1　核心思想（Core Idea）

面向对象设计的核心不是“把函数放入类”，而是决定对象对外承诺什么、内部可以怎样变化、不同对象如何协作。继承或方法名只提供结构；真正的可替换性来自行为契约。

本周要能从代码恢复设计、用类图和顺序图表达不同视角，并基于具体证据进行审查。判断设计时，应说明哪一类变化会引起哪一些修改，而不只给代码贴上“违反 SOLID”的标签。

2.2　历史渊源与问题背景（Origin & Motivation）

Ed 用过程式与面向对象组织方式作比较：过程式程序以数据结构和作用于数据的过程为基本单位；OOP 将数据及其操作组合成具有职责的对象，让开发者在更高抽象层次推理对象网络。两种方式都可写出好程序，OOP 也不会自动保证封装和模块化。

UML（Unified Modeling Language，统一建模语言）回应的是团队交流问题。课件指出它吸收了既有建模语言的表达方法，为设计者提供共同符号。自然语言适合说明意图，却容易产生歧义；代码精确但细节过多。UML 允许只表达当前讨论需要的结构或交互。扩展依据见参考资料 R1：OMG UML 规范；本课程只深入使用类图和顺序图。

代码审查则针对“作者的理解不能直接代表团队的理解”。提交补丁或 Pull Request 后，由其他开发者检查质量、潜在缺陷和标准符合性，同时传播领域知识。审查与自动测试互补：很多维护性问题即使测试通过也仍存在。

2.3　重要定义（Definitions）

Abstraction ／抽象：依据使用目的，保留必要性质、隐藏无关实现细节。抽象回答“外部需要知道什么”，不是简单删除代码。

Encapsulation ／封装：把状态与操作组织在有边界的模块或对象内，通过受控接口维护约束。名字前加下划线不等于已经阻止外部修改；返回内部列表仍可能破坏封装。

Inheritance ／继承：子类获得并可扩展或重写父类行为的机制；**Polymorphism ／多态**：客户通过共同契约使用不同实现，由实际对象决定执行的行为。

Composition ／对象组合：一个对象通过持有其他对象来实现行为。这里的“组合优于继承”采用广义对象协作含义；UML 的 Composition 是更严格的整体拥有关系，不能把所有字段引用都画成黑菱形。

Coupling ／耦合：模块间相互依赖的程度及方式；**Cohesion ／内聚**：模块内部职责彼此关联的程度。低耦合与高内聚是方向，不是机械地减少所有调用或所有方法。

Contract ／契约：调用方与实现之间约定的输入条件、结果、状态变化和异常行为。它可用文档、接口和测试表达；仅有类型签名通常不完整。

SOLID ／五项面向对象设计原则：Single Responsibility（单一职责）、Open-Closed（开闭）、Liskov Substitution（里氏替换）、Interface Segregation（接口隔离）、Dependency Inversion（依赖倒置）。各项正式含义与应用见本章完整课件内容的 SOLID 小节。

DRY ／ Don't Repeat Yourself，避免知识重复：系统中的每项知识应有单一、明确、权威的表示。两段文本相似不一定重复知识；两段形式不同却实现同一业务规则，也可能违反 DRY。

Law of Demeter ／迪米特法则，又称最少知识原则：对象应尽量与直接协作者交流，减少对其他对象内部关系的了解。"少写点号"只是提示，不是完整判据。

Code smell ／代码异味：暗示潜在设计问题的迹象，不等于已证明有功能缺陷。**Anti-pattern ／反模式**：反复出现但带来不良后果的解决方式。

Class diagram ／类图：显示类型、属性、操作及静态关系；**Sequence diagram ／顺序图**：显示一个场景中参与者之间按时间发生的消息。前者不能表达完整执行算法，后者不应充当全部源代码的图形抄写。

2.4 实际意义（Practical Significance）

在一个题库系统中，导入、验证、筛选、报告和命令行交互会同时变化。设计原则帮助判断职责边界，UML 帮助向队友解释边界，审查则检查实现是否与边界一致。例如，筛选函数若还悄悄写入 `checked_on`，文档声称"不修改输入"便不能成立。

真实审查意见应包含位置、事实、规则、工程影响和可行改进。"这个类有 15 个方法，所以违反 SRP"依据不足；"税率规则与文件导出格式分别受不同需求驱动，却必须修改同一个类"才指明了独立变化原因。

2.5 完整课件内容（Lecture Reconstruction）

2.5.1 Python OOP 中真正影响设计的机制

实例方法接收 `self`；静态方法（`@staticmethod`）不自动接收实例或类；类方法（`@classmethod`）接收实际调用类 `cls`。静态方法适合不依赖实例状态的相关操作，类方法常用于保留子类类型的替代构造入口。不要因为函数写在类中就假定它使用了某个对象状态。

双下划线方法（Dunder methods）连接语言协议：`__str__` 提供可读字符串，`__len__` 支持 `len(obj)`，`__init__` 初始化对象。精确语义澄清：对象分配通常由 `__new__` 控制，`__init__` 不是返回任意对象的工厂；Week 5 将利用这一差别解释 Singleton。

Python 的 `_name` 是内部使用约定；`__name` 会发生名字改写（Name mangling），主要用于避免继承中的名字冲突，而不是安全访问控制。子类仍须尊重约定。Java 的 private/protected 与 Python 的下划线约定不能机械对应。

PYTHON

```python
from abc import ABC, abstractmethod

class Readable(ABC):
    @abstractmethod
    def describe(self):
        pass

class Book(Readable):
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def describe(self):
        return self.title

    def __str__(self):
        return self.describe()

    def __len__(self):
        return self.pages

b = Book("Design Patterns", 395)
assert str(b) == "Design Patterns"
assert len(b) == 395
```

抽象基类（Abstract Base Class，ABC）可以包含已实现方法，也可以规定抽象方法。仍有未实现抽象方法的类不能直接实例化。与鸭子类型相比，ABC 更明确地表达课内模式的共同接口；但它仍不能自动证明实现满足行为契约。

继承时，`super()` 用于沿方法解析顺序（Method Resolution Order，MRO）继续查找，不只是“固定调用我眼中的那个父类”。Python 支持多重继承，因此 `FlyingCar(Plane, Car)` 的 `move()` 由 MRO 决定，不能同时自然表达“现在开车还是飞行”的业务选择。组合可把 DriveAbility 与 FlyAbility 分别保存；Week 9 再把“当前采用哪种能力”抽成 Strategy。

`@property` 使方法以属性语法访问，可以校验、计算派生值或维持接口兼容。setter 使赋值也通过受控逻辑。应避免昂贵 I/O 和令人惊讶的副作用：`obj.balance` 的外观会使客户预期访问廉价且稳定，Week 5 的延迟查询正好说明风险。

PYTHON

```python
class Stock:
    def __init__(self, count):
        self.count = count

    @property
    def count(self):
        return self._count

    @count.setter
    def count(self, value):
        if isinstance(value, bool) or not isinstance(value, int):
            raise TypeError("count must be an integer")
        if value < 0:
            raise ValueError("count cannot be negative")
        self._count = value

stock = Stock(3)
```

```
stock.count = 5
assert stock.count == 5
```

2.5.2 封装、组合、最少知识与 DRY

课件的 Driver 若通过 `self.car.engine.start()` 控制发动机，便需要知道 Car 的内部字段及协作关系。改为 `self.car.start()` 后，Car 可以内部选择发动机、执行安全检查或改用电驱动，而 Driver 仍只依赖 Car 的公共行为。

不能因此禁止所有链式 API。扩展解释：若每一步都返回同一个有意公开的构建器，链式调用未必泄漏内部结构。判断点是依赖了多少内部知识，而不是点号数量。

对外暴露集合时，可返回副本、tuple、frozenset 或只读视图，但应认识深浅差异。`tuple(self._rows)` 保护的是容器结构；若每个 row 是字典，客户仍能修改字典内容。需要深度隔离时，应复制元素或返回不可变领域值，同时考虑复制的时间与空间成本。

DRY 的典型风险是同一 cooldown 规则在两个函数独立实现。一处改成“至少 N 天”，另一处仍是“超过 N 天”，会产生边界不一致。应把规则放在一个权威位置，让不同入口调用。相反，不相关的两个三行循环，不应仅因形似就被强行合并为复杂的通用框架。

课件还以 Kernighan 关于“越聪明地写代码，越难调试”的警句和 Knuth 关于过早优化的提醒，强调先保持可理解性，再依据实际瓶颈优化。它们是反对不加测量的复杂化，不是反对必要性能设计。

2.5.3 SOLID：把原则还原为可解释的变化

S：Single Responsibility Principle ／单一职责原则。一个类应只有一个变化原因。职责不是“只准有一个方法”；同一职责可以需要很多协作方法。课件的 RevenueReport 混合收益计算、JSON/YAML/Markdown 格式和保存，各自受不同需求驱动。可拆为计算器、渲染器和存储组件，使格式变化不触及收益计算。

O：Open-Closed Principle ／开闭原则。软件实体对扩展开放、对修改关闭。针对预计的变化，通过新增实现扩展行为，减少修改稳定代码。支付函数每加一个渠道就增加分支，是课堂反例。把各渠道实现放到 PaymentProcessor 契约后，可以保持通用客户逻辑稳定。

开闭不是“任何文件永远不能改”。装配新对象、修复错误或改变需求时仍需修改。抽象也不能预知所有变化；若扩展极少，简单分支的成本可能更低。Week 3 的简单工厂虽然减少重复，集中选择逻辑仍可能需要修改。

L：Liskov Substitution Principle ／里氏替换原则。派生类型应能替换基类型而不破坏使用者依赖的正确性。它关心行为，不是现实世界分类是否成立。扩展的契约表达：子类不应加强输入前提、不应削弱结果保证，并应保持基类承诺的不变式；这是理解课程例子的辅助方式。

在可变 Ellipse 中，客户可能认为分别设定两条半轴后，其中一条不会被另一条的 setter 改写。Circle 若强制两轴相等，就破坏该保证。Square/Rectangle 的宽高独立 setter 问题相同。若二者只提供不可变的面积查询，或共同实现 Shape，则数学包含关系未必导致相同问题；结论取决于接口契约。

PYTHON

```
class Rectangle:
    def __init__(self, width, height):
        self.width, self.height = width, height

```

```
    def set_width(self, value):
        self.width = value

    def set_height(self, value):
        self.height = value

    def area(self):
        return self.width * self.height

class Square(Rectangle):
    def set_width(self, value):
        self.width = self.height = value

    def set_height(self, value):
        self.width = self.height = value

def rectangle_client(shape):
    shape.set_width(5)
    shape.set_height(4)
    return shape.area()

assert rectangle_client(Rectangle(1, 1)) == 20
assert rectangle_client(Square(1, 1)) == 16
```

这里第二个断言刻意展示不相容结果，不是宣称 Square 的行为正确满足 Rectangle 契约。修复可以让两者都实现只读 Shape，把改变尺寸的操作留在各自合理的接口中，而不是让 Square 静默改变另一个维度。

I：Interface Segregation Principle ／接口隔离原则。不应迫使客户依赖它不使用的方法。一个 Vehicle 同时要求 drive、fly、sail，会使 Car 被迫提供不支持的 fly/sail。可按能力拆成 Driveable、Flyable、Sailable。盲目提供空实现或运行时抛“不支持”会把类型错误推迟给客户；但也不要把每个方法拆成一个毫无意义的接口。

D：Dependency Inversion Principle ／依赖倒置原则。高层策略与低层细节应依赖抽象，抽象不应依赖细节，细节应实现抽象。课件的 UserService 在构造器中创建 PostgresDatabase，使业务与具体存储绑定。改为接受满足 UserStore 的对象，业务便能用数据库实现或测试替身。

PYTHON

```
class MemoryStore:
    def __init__(self):
        self.users = []

    def save_user(self, user):
        self.users.append(dict(user))

class UserService:
    def __init__(self, store):
        self._store = store

    def create_user(self, user):
        if not user.get("name"):
            raise ValueError("name is required")
        self._store.save_user(user)

store = MemoryStore()
UserService(store).create_user({"name": "Ada"})
assert store.users == [{"name": "Ada"}]
```

这里采用鸭子类型表达存储契约，课程图示可用 ABC 显式表示。依赖注入（Dependency injection）是把协作者提供给对象的技术；依赖倒置是依赖方向的设计原则。注入一个具体数据库对象只减少创建耦合，若业务仍调用其专有细节，就未完全获得抽象边界。

2.5.4 类图的基本语法与阅读顺序

课件的 UML 总览区分结构与行为：类、对象、组件、组合结构、包、部署和 Profile 属结构视角；用例、活动、状态机属行为视角；顺序、通信、计时和交互概览用于交互建模。本课程深入的只有类图与顺序图，不要求由一张总览推导其余图的完整语法。

类框通常分为名称、属性、操作三层。属性可写 `name: Type = default`，操作可写 `method(arg: Type): ReturnType`。`+` 表示 public，`-` 表示 private，`#` 表示 protected，`~` 表示 package。静态成员通常加下划线；抽象类和抽象操作可用斜体，接口可标 `«interface»`。课件着色用于教学提示，不是访问级别的通用 UML 标准。

阅读时先辨认类型及抽象，再沿关系理解职责，最后看多重性与具体成员。不要先被每个方法的细节淹没。下面用重新绘制的矢量图表达本课程需要辨认的关系：

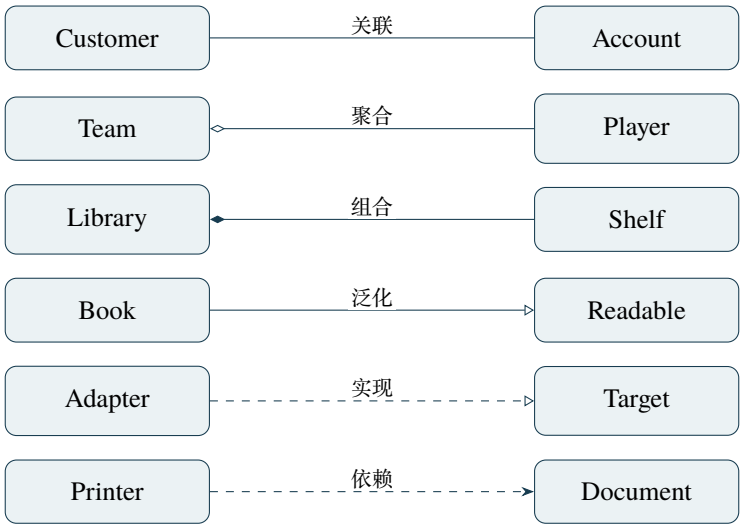


图 2: UML 关系重绘：菱形位于整体端，三角指向一般类型或契约。

Association ／关联：对象间具有一般结构连接，例如 Librarian 知道 Library。它描述关联，不自动规定生命周期。

Aggregation ／聚合：空心菱形位于整体端，表示整体与部分关系，部分可独立存在。课堂以 Team 与 Player 说明。

Composition ／组合关系：实心菱形位于整体端，表达更强的独占整体拥有语义。课堂用 Car 与 Wheel 等模型说明；真实业务是否如此，需要由模型约定决定。Python 引用计数本身不会强制实现这种语义。

Generalization ／泛化或继承：实线加空心三角，箭头指向更一般的父类。**Realization** ／实现：虚线加空心三角，指向被实现的接口。课内讲模式时常将 ABC 和接口作为相似契约使用，画图时仍应会识别两种符号。

Dependency ／依赖：虚线箭头从客户指向供应者，表示其定义变化可能影响客户。参数、局部变量或临时方法调用常体现这种“使用”，区别于长期持有对象的关联。

2.5.5 多重性与关联类

Multiplicity（多重性）写在关联端，描述一个给定对象在某一时刻可以关联多少个对端对象。`1` 是恰好一个，`0..1` 是可选一个，`*` 或 `0..*` 是零到多个，`1..*` 是至少一个，`2..4` 含两个端点。

若 Library 对 Librarian 一端标 `1..*`，表示一个 Library 在当前模型中需要至少一位 Librarian；不是“历史上总共至少雇过一位”，也不是每个 Librarian 必须对应很多 Library。反方向的约束要读另一端标记。

Association class（关联类）表达关系本身拥有数据或行为。Company-Person 的 Employment 可以记录 salary、startDate；这些数据属于某次雇佣关系，不能简单放到 Person 中，因为同一人可能有多段雇佣。Library-Member 的借阅关系同样可由 Loan 记录日期。

课件练习图的判断方法是：Library-Shelf 看整体端是否实心菱形来判断组合；Readable 是抽象契约；Library-Librarian 的 `1..*` 按对端解释；Printer 使用 Readable 的虚线箭头体现依赖。不能仅凭“图书馆拥有书架”这句日常语言忽略图上的符号。

2.5.6 顺序图的时间与对象语义

顺序图把参与者排在横轴，把事件顺序沿纵轴自上向下排列。参与者可以写 `source_file:File`，仅需类名时写 `:File`。竖向虚线是 Lifeline（生命线）；Activation bar（激活条）表示执行或控制焦点的区间，不能把它简单解释为对象存在的全部时间。

消息箭头表达调用；虚线返回箭头可表达结果；创建消息指向较低位置出现的新对象，销毁用生命线上的 X。精确澄清：调用 Python 文件的 `close()` 关闭资源，并不必然销毁该对象。课件以“关闭后结束生命线”简化场景，本书将其读作资源使用场景终结，而不宣称对象已被垃圾回收。

课件第一个邮件场景的顺序是：用户点击 Check Email，客户端发送未发邮件，服务器确认，客户端请求新邮件，服务器返回邮件数据，再请求删除旧邮件。类图即使显示 Client-Server 关联，也不能表达这些先后关系。

文件场景则为：打开 source.txt 读模式，读取第一行，打开 dest.txt 写模式，写入该行，关闭目标，再关闭来源。原图用 bytes written 作为返回值标签；Python 文本流的 `write` 通常返回写入字符数，不能把教材的跨语言消息标签当作所有 API 的精确类型。

2.5.7 组合片段：opt、alt 与 loop

Frame（框）将一组消息组织成条件或循环。`opt [condition]` 是可选交互，相当于无 else 的 if；`alt` 的多个区间配 guard（守卫条件），表达 if/else；`loop [condition]` 表示重复。守卫是选择依据，不是另一个独立对象。

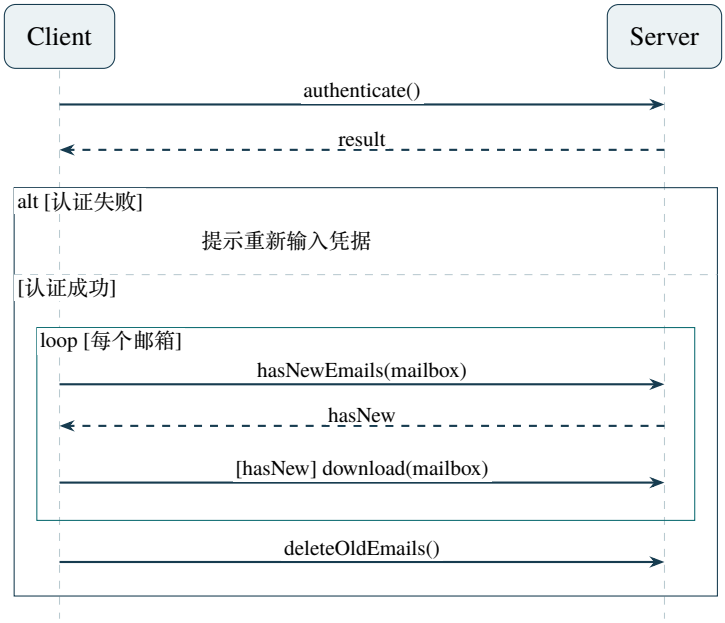


图 3: 邮件同步顺序图重绘：守卫与框界定成功路径，避免认证失败后继续处理。

同步邮件场景先认证：失败时提示重新输入凭据；成功后对各 mailbox 循环，若有新邮件才下载，然后删除旧邮件。应把“认证成功”的框包住相应后续流程，避免画出失败后仍然处理邮箱的歧义。

画图时可省略不影响讨论的局部计算，但不能省略决定业务结果的消息顺序。它适合语言无关的设计交流，也可以由非程序员参与；不适合逐条表现全部实现语句。

2.5.8 代码审查的目标与异味

Code review 是质量保证活动，也承担知识传递。测试偏重执行行为，审查还关注命名、文档、职责、依赖、异常、复杂度和团队规范。

课件列出的三类异味是 God class（上帝类，承担过多不相关职责）、Long parameter list（长参数列表，可能暴露职责或数据建模问题）、Arrowhead anti-pattern（箭头形嵌套，过多嵌套使主路径向右漂移）。可采用提前返回、抽取有业务含义的方法和重新分配职责，但不能为了减少缩进改变条件顺序或异常语义。

课件的 `count_students` 注释说统计通过人数，实际却在 `student.passed()` 时 continue，因此统计未通过者。给定 Alice 39、Bob 49、Charlie replacement 50、Dana 40，普通及补考阈值分别为 40、50，实际结果是 1。若只读注释便会误判。应先恢复真实行为，再判断该改代码还是文档；不能为了让文档正确擅自改变已接受需求。

编码标准带有团队选择性质。课件引用 OpenStack、Google 和 Linux 的规范，强调团队一致性。规范可自动检查一部分问题，但“遵守格式”不能证明职责合理，注释更多也不能补救混乱代码。

2.5.9 全局状态为何造成隐蔽耦合

模块 `database` 保存单个可变 `_connection`。A 模块连接 students.db，B 模块随后连接 lecturers.db；两者调用 `database.execute` 时都使用最后一次写入的连接。课件图中最后连接哪一个库取决于实际导入与调用顺序，核心问题是共享可变单槽状态，而非固定某个库名。

把连接显式作为对象或参数提供，可以让两个客户拥有独立连接，并使测试安排更清楚。全局注册表有时有合理用途，但需要限定修改时机、生命周期和测试隔离，不能把“偶尔有用”解释为任意共享状态都无害。

2.5.10 Assignment 1：把原则落实到可定位证据

本地 CS-01 要求名称与文档准确；CS-02 要求重复使用的值集中为常量且常量集合不可变；CS-03 倡导小而聚焦的函数；CS-04 捕获预期错误，不隐藏异常；CS-05 禁止可变默认参数；CS-06 区分值、身份和真值；CS-07 限制外部修改内部状态。

实际代码中的典型证据包括：SUPPORTED_FORMATS 与 SUPPORTED_STATUSES 是可变 set；`empty_import_report(errors=[])` 共享默认列表；`activity_log` 返回内部列表；`build_candidate_pool` 写入 `question["checked_on"]` 并返回原 question 引用，与“不修改输入”的文档冲突；状态字符串用 `is not` 而非 `!=` 比较。

异常路径应沿 `main` → `_cmd_import` → `session.import_file` → `import_questions` → `load_question_file` 阅读。有效空 JSON 数组会产生零记录报告。找不到文件等错误被宽泛 `except Exception: return []` 隐藏，因而看起来也是有效空数据；JSONDecodeError 是 ValueError 的子类，被前面的 `except ValueError: raise` 重抛，最终由 CLI 的错误处理报告。分支顺序与异常继承关系共同决定结果。教材引用的是实际源代码证据，不把现有 answers.md 当作官方标准答案。

课件还用复杂异常路径、输入记录被修改、truthiness 校验不足等反馈提醒读者：跨文件追踪很常见，审查意见必须落到行为。例如 `if not estimated_minutes` 既不能拒绝所有负数，也不能保证值为整数；应先确定数据契约，再评估检查是否充分。

2.6 知识关联（Connections）

Week 3 的工厂把创建职责从客户中移出；Week 4 的 Composite 让树节点对外满足共同契约，也展示统一接口与 ISP 的冲突；Week 5 的装饰器依赖可替换性；Week 6 的外观应用最少知识；Week 7 的可控制性、可观察性则是依赖设计的直接后果。

UML 应与代码同步。若图上声称依赖抽象，代码却按具体类进行分支，二者描述的设计并不相同。审查时应把图当作设计声明，把实际代码及验证结果当作实现事实。

2.7 考试应用（Exam Application）

需要掌握：五项原则的正式含义、反例与适用边界；UML 符号及多重性方向；真实代码的共享状态和异常路径。常见错误：把方法数量当作职责数量，把数学 is-a 当作行为子类型，把所有 has-a 画成 Composition，把激活条当作对象整个寿命。以下均为自编练习。

情境题 1：胖接口。统一设备接口要求 print、scan、fax，普通打印机通过抛异常实现后两者。团队说“它实现了接口，所以符合替换原则”。请分析。

解析。若客户契约保证三种能力可用，普通打印机无法替代该接口，存在 LSP 问题；迫使只需打印的客户依赖扫描和传真也存在 ISP 问题。按能力划分接口，让多功能机组合实现。若接口本来明确允许不支持并返回能力信息，需据该契约重新判断，而不能只看异常这一语法。

情境题 2：安全日志？ getter 返回 `tuple(self._events)`，每个 event 是可变字典。外部代码执行 `log[0]["action"] = "approved"`。能否说日志已不可修改？

解析。 tuple 只冻结外层结构，字典仍与内部共享。可返回字典副本组成的 tuple，或采用不可变事件值。应根据是否存在更深层嵌套决定隔离深度，而不是无条件使用昂贵的深复制。

情境题 3：顺序图纠错。 图中身份认证失败与成功两个 alt 分支之后，无条件画了下载全部邮箱。需求要求认证成功才下载。哪里有问题？

解析。 图把下载放在两个分支都可能继续的公共后续路径。应将 loop 与下载放进成功分支，失败分支只提示重试或结束。图的抽象性不允许省略这种决定权限的控制条件。

情境题 4：空文件与坏文件。 导入器捕获所有异常并返回 []。测试验证“结果是列表”且通过。请给出一条审查意见与一个改进测试思路。

解析。 指出捕获范围和正常值回退的位置，解释其混淆读失败与空数据、妨碍调用方作出正确响应。测试分别使用合法空数组、文件不存在和非法 JSON，并断言预期结果或明确错误，而不仅检查类型。

第 3 章　Week 3：质量属性、最少原则与 Factory Method

来源：L3 第 1-37 页；Ed「Week 3: Introduction to Design Patterns」全部 19 页；Assignment 2 的 importer 设计。Factory Method 的主要讲解位于 Ed，不能只读本地 PDF 的最后几页。

3.1　核心思想（Core Idea）

设计的价值取决于它如何应对变化。质量属性说明“好”具体体现在哪个维度，设计原则限制不必要复杂性，设计模式则为反复出现的问题提供可交流的解决结构。

本周的中心案例是对象创建：客户不应因每增加一种产品就重复了解并修改所有具体构造过程。工厂方法（Factory Method）把“创建哪一种产品”交给可扩展的创建者子类，同时保留稳定的公共工作流程。

3.2　历史渊源与问题背景（Origin & Motivation）

课件以规则、原则、棋局模式三个层次类比学习设计。知道棋子的走法不代表会布局；知道类与继承的语法，也不代表能在变化中维持清晰结构。成熟开发者把一组相互关联的设计决定识别为整体，形成组块（Chunking），用模式名称压缩交流成本。

课程引用 GoF（Gang of Four，即 Gamma、Helm、Johnson、Vlissides）的《Design Patterns: Elements of Reusable Object-Oriented Software》。模式记录的是跨领域反复出现的问题、解决方式和后果，不是拿来实例化的现成类库。

在直接调用构造器之前，客户可能已决定具体实现；当支付渠道、文件格式或数据库后端变化时，这种决定散落到许多位置。简单工厂先把决定集中起来，GoF Factory Method 再利用子类型扩展创建行为。代价是增加产品与创建者层次，因此模式需要真实的变化压力。

3.3　重要定义（Definitions）

Quality attribute ／质量属性（QA）：用于衡量产品在某一维度表现如何的、可明确化并检验的性质。功能说明“做什么”，质量属性说明“做得怎样”；不能只写“系统性能好”而不指明条件与指标。

Reliability ／可靠性：在规定条件和时间范围内持续正确服务的能力；**Availability ／可用性**：需要服务时系统能够提供服务的程度。频繁短暂停机与极少但漫长停机的权衡，不能仅用一个词概括。

Performance ／性能：延迟、吞吐与资源使用等表现；**Scalability ／可扩展规模能力**：负载及资源增加时表现如何变化，不等于单次执行速度。

Robustness ／健壮性：面对异常输入或条件时仍能适当处理；**Safety ／安全性（避免伤害）**：避免对人、环境或资产造成不可接受伤害；**Security ／信息安全**：抵御未授权访问和恶意行为。中文“安全”须结合英文区分。

Extensibility ／可扩展性：加入新行为的便利程度；**Maintainability** ／可维护性：理解、修改和修复软件的便利程度。两者相关，但可维护性范围更广。

Principle of Least Expressiveness ／最少表达力原则：选择能够自然表达问题的、表达能力最少的计算模型。能用明确映射表达固定关系时，不必引入任意程序控制流。

Principle of Least Power ／最少能力原则：为用途选择足够而不过度强大的语言或机制；**Least Astonishment** ／最少惊讶原则：行为应符合语法和约定造成的合理预期；**Least Effort** ／最少努力原则：用户往往采用最容易完成目标的途径。

Simple factory ／简单工厂：集中选择并创建具体产品的函数或对象；**Static factory method** ／静态工厂方法：具有描述性名称的静态创建入口。它们可以很实用，但不等于 GoF Factory Method。

Factory Method ／工厂方法模式：定义创建产品的操作，让创建者子类决定具体实例化哪一种产品，使产品使用流程与具体产品创建解耦。角色为 Product、ConcreteProduct、Creator、ConcreteCreator。

Interning ／驻留或池化：按不可变值复用规范对象，避免保存重复对象。它是“每个受管理值对应规范表示”的机制，不等于整个类型只能有一个对象的 Singleton。

3.4 实际意义（Practical Significance）

质量属性通常是横切关注点（Cross-cutting concern）：可靠性可能牵涉存储、重试、错误报告和部署，难以在项目末期像增加一个按钮那样局部补上。设计时要说明想保护的变化方向和愿意承担的成本。

课程的 Factory Method 尤其适合框架在稳定流程中允许扩展点：导入服务保持“创建导入器 → 导入 → 验证结果”，具体服务决定 JSON、CSV 或 JSONL。若系统只有两种固定格式且永不变化，一个简单映射可能更容易维护；不必为了“用了模式”而制造空壳层次。

3.5 完整课件内容（Lecture Reconstruction）

3.5.1 质量不是单一分数

L3 列出可靠性、可用性、性能、规模扩展、健壮性、安全、信息安全、可扩展性和可维护性。这些属性受早期设计影响，而且可能相互制约。增加缓存可能降低延迟，但引入失效与一致性问题；更严格的校验可能提高健壮性，却增加代码与错误处理路径。

扩展的可检验表达例子：“在指定硬件、100 个并发请求下，95% 的读取在 200 ms 内返回”比“响应快”更适合评估设计。它不是课程要求的具体数值，只演示质量目标如何成为可验证条件。

课件以技术债务和开发时间图表说明坏代码具有持续成本，不要求把报告统计当成每个团队都适用的比例。可维护性影响未来修改成本，因而属于工程上的功能交付能力，而不只是审美偏好。

3.5.2 Brooks' Law 与协作成本

Brooks' Law（布鲁克斯定律）指出，为已延期的软件项目增加人手往往会使其更晚。原因包括新成员学习与培训、沟通协调成本，以及任务不能任意切分。它不是任何场景都成立的数学定理。

若 n 人需要彼此直接沟通，潜在成对联系数量为 n(n-1)/2。这是本教材用于解释沟通增长的扩展模型，不代表所有团队实际都必须建立这些连接。清晰模块边界、文档和共享模式词汇的价值，在于降低加入与协作的理解成本。

3.5.3 最少表达力：让关系直接呈现

解码器案例中，客户想知道一个格式能匹配几个解码器、一个解码器可处理几个格式。长 if/elif 需要读者模拟执行；字典直接声明每个格式映射到一个选择。

PYTHON

```
class TextDecoder:
    def decode(self, data):
        return data.decode("utf-8")

class LatinDecoder:
    def decode(self, data):
        return data.decode("latin-1")

DECODERS = {"txt": TextDecoder(), "csv": TextDecoder(),
            "legacy": LatinDecoder()}

def decode(format_name, data):
    decoder = DECODERS.get(format_name)
    if decoder is None:
        raise ValueError("Unsupported format")
    return decoder.decode(data)

assert decode("txt", b"hello") == "hello"
```

这是教学重写，用真实可执行的解码器替代原图 DecoderA/B。映射说明每个键选择一个对象，但不同键可以对应相同类型或同一实例；这两件事也不能混淆。示例创建了两个 TextDecoder 实例，若对象无状态且允许共享，也可显式复用同一实例。

最少表达力不是“一律用字典替换判断”。依赖请求上下文、复杂优先级或动态决策时，硬塞进静态映射反而不自然。关键是减少读者必须推理的自由度。

3.5.4 最少惊讶、最少能力与最少努力

属性 getter 执行大规模数据库查询、普通名字的函数暗中修改入参、Circle 的 setter 改变另一个轴，都会破坏合理预期。最少惊讶与封装、LSP 和文档准确性直接相连。

最少能力关注表达机制的代价。课件比较 XML 与 JSON，意在说明选择复杂格式时需考虑解析特性和潜在攻击面；不能概括为“XML 总不安全”或“JSON 自动安全”。具体安全性取决于解析器、配置、验证和输入可信度。

最少努力则影响 API 设计。若安全、正确的操作需要大量额外步骤，而危险操作是一行默认调用，客户很可能选择后者。改进应让正确路径自然可达，而非只在文档里要求谨慎。

3.5.5 模式分类与适用范围

创建型（Creational）管理对象创建及选择：Factory Method、Singleton。结构型（Structural）组织对象和类之间的组合：Composite、Decorator、Proxy、Adapter、Facade。行为型（Behavioural）组织职责、算法和交互：Command、Iterator、Chain of Responsibility、Observer、Strategy、Template Method。

课件列出的其余 GoF 模式不在本单元当时的教学范围；本书不把 Abstract Factory、Builder、State 等扩写成课程正文。一个模式至少需要名称、问题、解决结构与后果（Consequences）。只有画了几个类框却不说明解决什么问题，不足以完成模式分析。

3.5.6 构造器的局限与对象复用

在普通构造调用中，客户指定了具体类，且通常期待新实例。若想返回某种子类型或复用对象，独立的创建入口更清楚。L3 的示意 `__init__` 返回 Child/Adult 不是有效 Python：`__init__` 必须返回 None。

精确澄清：Python 的 `__new__` 确实能控制实例创建，Week 5 会利用它；因此“构造器永远不能返回别的对象”是课件讨论普通初始化接口时的简化，而非 Python 对象模型的绝对定理。

Interning 以不可变字符串说明复用动机：一般字符串相等判断可能需要检查字符，规范对象复用可以节省重复存储并支持身份快速比较。Python 自动复用哪些常量属于实现细节；不能假定“只要字符串短就必定驻留”。

PYTHON

```python
import sys

left = sys.intern("soft" + str(2201))
right = sys.intern("soft2201")
assert left is right
assert left == right
```

只有在明确控制规范化流程并保留合适引用的局部机制中，身份比较才可参与这种优化。普通业务代码仍应使用 ==。驻留有多个不同值的规范对象，Singleton 则限制某个类的唯一实例；原课件用二者作类比，不应把类比当成等价定义。

3.5.7 从重复业务到简单工厂

支付代码最初在 webstore、POS 和 billing 中复制渠道判断、处理和风险检查。先提取共用操作可以减少 DRY 问题，再引入 PaymentProcessor 多态，使每个产品承担自己的处理规则。

简单工厂集中 `if method == ...` 或注册映射，客户只调用一个创建入口。客户从具体构造器解耦，验证与创建规则也集中，但工厂仍知道全部支持的产品。新增渠道通常要修改中央选择点。

PYTHON

```python
class Cash:
    def process(self, amount):
        return f"cash:{amount}"

class Credit:
    def process(self, amount):
        return f"credit:{amount}"
```

```
PRODUCTS = {"cash": Cash, "credit": Credit}

def create_processor(method):
    product_type = PRODUCTS.get(method)
    if product_type is None:
        raise ValueError("Unsupported payment method")
    return product_type()

assert create_processor("cash").process(20) == "cash:20"
```

这里的工厂是集中选择函数，不含创建者子类覆盖的创建步骤，因而不应仅因名字含 create 就称为 GoF Factory Method。中央映射不是自动错误；若变化集中且有限，它可能是恰当的最少表达力方案。

3.5.8 静态工厂是创建入口，不是模式判据

`User.from_dict(data)`、`User.guest()` 这样的命名入口，可解释创建意图、进行验证或返回已有实例。Python 可使用 staticmethod，也常使用 classmethod 保留子类构造能力。它与简单工厂可以重叠：一个静态方法也可以执行中央选择。

但静态工厂并不通过重写机制把具体产品选择交给 Creator 子类型。考试中要辨认扩展机制，而不是看到 staticmethod 或 factory 字样就下结论。

3.5.9 GoF Factory Method 的完整结构

稳定流程先调用创建步骤，再通过 Product 契约使用结果。ConcreteCreator 只覆盖创建决策；客户选择或接收 Creator 后，无需了解产品具体类型。以下教学重写延续支付案例：

PYTHON

```
from abc import ABC, abstractmethod

class PaymentProcessor(ABC):
    @abstractmethod
    def process(self, amount):
        pass

class CashProcessor(PaymentProcessor):
    def process(self, amount):
        if amount < 0:
            raise ValueError("negative amount")
        return f"cash:{amount}"

class CardProcessor(PaymentProcessor):
    def process(self, amount):
        if amount < 0:
            raise ValueError("negative amount")
        return f"card:{amount}"

class Checkout(ABC):
    def pay(self, amount):
        processor = self.make_processor()
        return processor.process(amount)

    @abstractmethod
    def make_processor(self):
        pass
```

```
28
29 class CounterCheckout(Checkout):
30     def make_processor(self):
31         return CashProcessor()
32
33 class WebCheckout(Checkout):
34     def make_processor(self):
35         return CardProcessor()
36
37 assert CounterCheckout().pay(12) == "cash:12"
38 assert WebCheckout().pay(12) == "card:12"
```

角色对应：PaymentProcessor 是 Product，CashProcessor/CardProcessor 是 ConcreteProduct，Checkout 是 Creator，CounterCheckout/WebCheckout 是 ConcreteCreator。`pay` 是稳定使用流程，`make_processor` 是被重写的工厂方法。

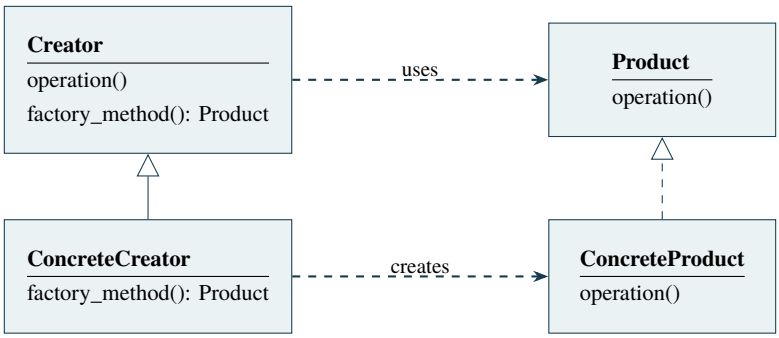


图 **4:** Factory Method：创建者层次与产品层次分开，稳定流程调用创建扩展点。

执行顺序为：客户调用具体 Checkout 的 pay；继承的 pay 调用 self.make_processor；动态分派进入具体创建者；返回产品；pay 调用产品的 process。创建者对象与产品对象处于不同层次，不应把“调用 pay 的客户”误认成被创建的产品。

3.5.10 披萨、数据库与题库的同一结构

披萨案例中，PizzaStore 的 `order_pizza` 保留 prepare → bake → cut → box，而 `create_pizza` 决定具体 Pizza。不同店的创建逻辑可以不同，稳定制作流程不必复制。这也为 Week 10 “固定骨架中留步骤给子类”的 Template Method 建立联系。

Django 案例围绕数据库后端选择游标：框架需要支持不同数据库，而共同操作不应散布所有具体 Cursor 的构造条件。读实际框架时，角色名称未必与教科书一致，要寻找“稳定流程调用可被具体后端覆盖的创建操作”。

Assignment 2 中，QuestionImporter 是 Product，JSON/CSV/JSONL importer 是 ConcreteProduct；QuestionImportService 是 Creator，其 `import_questions` 创建 importer、导入、验证结果；各具体 service 的 `create_importer` 返回对应产品。后缀注册表选择 ConcreteCreator，不替代 Factory Method 本身。系统依然需要在装配边界决定“选哪个服务”，模式减少的是稳定使用代码里的具体创建耦合。

导入格式变化和共同结果契约分别应被测试：JSONL 跳过空白行、要求每个非空行是 JSON 对象；报告的 accepted 数量必须与结果长度一致，loaded 应等于 accepted + rejected。这样扩展才保留客户预期。

3.5.11 收益、代价与过度设计

收益包括局部扩展创建行为、减少重复、稳定客户工作流程及便于替换产品。代价包括额外抽象、更多类、跨文件追踪与配置复杂性。若每个产品对应一个仅含一行代码的创建者，却没有共享流程或独立扩展需求，模式可能不划算。

课件最后提醒“先识别问题，再选模式”：面向接口、识别变化并封装变化、谨慎选择组合或继承，比记住名称更根本。课间的 C/C++ `x --> 0` 实际可解析为 `x-- > 0`，不是一种新箭头操作符；其教学作用同样是不要被视觉外观误导。

3.6 知识关联（Connections）

Factory Method 具体化了 OCP 和面向接口编程：变化集中在创建步骤，使用步骤依赖 Product。它也是 Template Method 的一种特定创建扩展点。与 Strategy 相比，本周主要改变“得到哪一种对象”，后者主要改变“执行哪一种算法”。

Singleton 同样涉及创建，却限制实例数量；Interning 按值复用；Proxy 可以延迟创建。这些机制解决的问题不同，不能把所有“返回对象”的函数混成一个模式。

3.7 考试应用（Exam Application）

掌握要求：能辨认产品与创建者两套层次，追踪工厂方法被稳定流程调用的位置，解释中央选择点为什么不一定消失。**易错点**：把简单工厂当作 GoF Factory Method，把静态方法当作模式的必要条件，认为 OCP 要求所有文件完全不修改。以下均为自编练习。

情境题 1：扩展 JSONL。一个服务把“创建 importer、导入、检查报告”复制到三个函数中。有人提议只把构造调用移到一个字典。这样解决了什么，没解决什么？

解析。字典集中格式到产品的选择，降低选择表达的复杂性，但没有消除三个流程的重复。可以建立稳定 Creator 操作，通过 create_importer 扩展创建步骤，复用导入后的契约检查。若流程无需子类型扩展，也可用更简单的注入或辅助函数，不能仅因题目出现字典就断言必须使用工厂方法。

情境题 2：错误识别。`User.guest()` 是 staticmethod，返回默认用户。它是否必然是 Factory Method 模式？

解析。不是。这是静态工厂创建入口。需要看到 Creator 的可覆盖创建操作，以及它怎样让稳定流程使用抽象 Product，才能据此识别 GoF 模式。

情境题 3：创建者数目。新增 ApplePay 后添加 ApplePayProcessor 和一个 ConcreteCheckout，启动配置还需登记它。有人说“配置改了，违反 OCP，所以模式失败”。请评价。

解析。OCP 针对被保护的变化方向和边界。核心 Checkout 流程及已有产品可以保持稳定，装配配置选择新类型是合理修改。若业务流程内部仍按渠道判断，才说明相关扩展尚未隔离。

情境题 4：质量属性取舍。为减少创建成本，团队复用一个可变产品实例。它可能改善什么，又引入什么？

解析。可能降低分配和初始化开销，但共享状态会引入测试污染、并发和生命周期问题。应先确认产品可安全共享，再衡量实际性能收益。对象复用不是 Factory Method 的自动保证，也不能把性能收益推导为整体设计更好。

第 4 章　Week 4：Composite 的递归统一与 Command 的操作对象

来源：L4 第 1-69 页；Ed「Week 4: Composite Pattern」全部 8 页；Assignment 2 的规则树、创建者与命令实现。课件的代码客户（Client）指调用某个组件的代码，不是产品最终用户。

4.1　核心思想（Core Idea）

本周用两种不同的统一化降低客户复杂度：Composite 让客户以同一接口使用一个元素和一组元素；Command 把一次请求表示成可保存、传递和撤销的对象。前者把树的递归交给节点，后者把操作内容从触发操作的按钮或界面中移出。

必须掌握的能力是：识别共同契约、追踪递归或委托的执行，并判断统一接口带来的便利是否值得额外结构和状态成本。

4.2　历史渊源与问题背景（Origin & Motivation）

文件系统天然包含文件、目录及目录中的子目录。朴素代码通常分别存 files 与 subdirs，再在每个操作中重复判断类型。增加一种节点会同时影响 size、print、search 等多处函数。Composite 的动机是让每类节点自行实现同一操作，组合节点负责递归调用。

文本编辑器则具有 Copy、Paste、Save、Undo 等多种操作。若 Button 根据 label 判断并执行所有业务，界面触发器便承担越来越多责任。Command 从“直接调用一次方法”发展为“先保存一次请求，再由某个时机执行”，使历史、队列和宏操作成为可能。它不是在所有调用外再包一层的普遍要求。

4.3　重要定义（Definitions）

Composite ／组合模式：将对象组织成表示部分与整体层次的树，使客户可以统一处理单个对象与对象组合。

Component ／组件契约：Leaf 与 Composite 都满足的公共操作；**Leaf ／叶子**：没有子节点的基本元素；**Composite ／组合节点**：保存 Component 子节点，并通过对子节点操作实现整体操作。

Recursive composition ／递归组合：组合节点的子元素既可以是叶子，也可以是其他组合节点。把一组同类型对象存入列表不自动构成 Composite，关键是整体与部分共享契约并可递归组合。

Transparency ／透明性：客户无需按叶子或组合区分操作；**Safety ／接口安全性**：避免暴露对象不适用的操作。本章专指 API 使用安全，不等同于 Week 3 的系统伤害安全属性。

Command ／命令模式：将请求封装为对象，使客户可以用不同请求参数化调用者，并支持请求排队、记录或撤销。

Invoker ／调用者或触发者：要求命令执行的对象；**Receiver ／接收者**：了解并完成实际工作的对象；**ConcreteCommand ／具体命令**：保存接收者、请求参数及必要的撤销信息。

Undo ／撤销：依据操作前状态或可逆差量恢复约定的先前状态。它不是再执行一次正向操作，也不能普遍通过把参数取负实现。

Command history ／命令历史：按顺序保存已经成功执行且可撤销的命令。按后进先出（LIFO）撤销，与正向执行次序相反。

4.4 实际意义（Practical Significance）

Composite 适合目录、界面面板、任务组、表达式树和层次化成绩统计；统一客户接口可减少重复类型判断。但新增“节点种类”容易，不代表新增“所有节点都要支持的操作”也容易：修改公共契约可能触及全部实现。

Command 适合菜单、键盘快捷键、事务编辑、任务队列和数据库迁移。不同入口可触发同一种命令，触发器不必理解操作内容。代价是更多对象与状态管理；不可逆外部副作用还需要补偿或业务限制，单靠模式名称不能保证撤销。

4.5 完整课件内容（Lecture Reconstruction）

4.5.1 接口、鸭子类型与客户视角

课件对比 Java 的 Sender interface 与 Python 的 `send(message)` 鸭子类型，再引入 ABC 表达课程图示的共同契约。Client 是调用该契约的代码。例如 Factory Method 中需要创建产品的业务流程是客户，按钮本身则可能是 Command 的 Invoker。

同一个对象在不同关系中可以有不同角色。Assignment 2 的 Session 同时作为命令历史调用者和修改题目状态的 Receiver，不违反模式的本意；角色是分析职责的工具，不要求每个角色一定单独占一个类。

4.5.2 文件系统的 Before：类型检查为何扩散

朴素 Directory 分别保存 files 和 subdirs；外部 `total_size` 用 `isinstance` 区分两者。`print_tree` 和 `search` 又必须重复相同结构。若新增 ZipFile 或 SymbolicLink，所有外部算法都需认识新类型。

问题不在于 `isinstance` 这个语法永远不好，而在于“节点种类选择”在多个客户中重复。设计目标是让客户只依赖 Node.size，不必知道它是否递归。需要注意：文件系统符号链接可能形成图或环，不能未经定义就把真实文件系统当成无环树。

4.5.3 文件系统的 After：让对象承担递归

PYTHON

```
from abc import ABC, abstractmethod

class Node(ABC):
    def __init__(self, name):
        self.name = name

```

```
    @abstractmethod
    def size(self):
        pass

    @abstractmethod
    def lines(self, indent=0):
        pass

class File(Node):
    def __init__(self, name, size_bytes):
        super().__init__(name)
        self._size = size_bytes

    def size(self):
        return self._size

    def lines(self, indent=0):
        return ["  " * indent + f"- {self.name} ({self._size}B)"]

class Directory(Node):
    def __init__(self, name):
        super().__init__(name)
        self._children = []

    def add(self, child):
        self._children.append(child)
        return self

    def size(self):
        return sum(child.size() for child in self._children)

    def lines(self, indent=0):
        result = ["  " * indent + f"[DIR] {self.name}"]
        for child in self._children:
            result.extend(child.lines(indent + 1))
        return result

root = Directory("root").add(File("abc.txt", 10))
src = Directory("src").add(File("def.py", 120))
src.add(File("xyz.py", 80))
root.add(src).add(Directory("assets").add(File("thumbnail.png", 2048)))
assert root.size() == 2258
assert src.size() == 200
assert root.lines()[2] == "  [DIR] src"
```

教学重写以返回字符串列表代替直接 print，使输出便于断言；核心设计与课堂一致。add 仅放在 Directory 中，避免让 File 提供没有意义的加子节点操作。示例假设无环树，未实现检测 self 或祖先加入；实际工程应在构建层规定并维护这一约束。

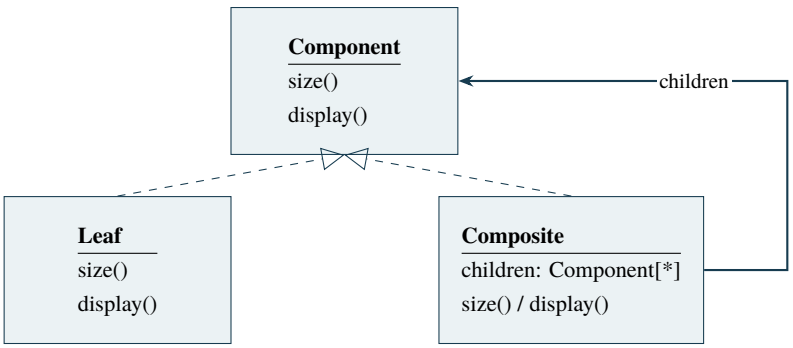


图 5: Composite：叶子和组合实现相同契约，组合还持有递归子组件。

4.5.4 递归执行与新操作

调用 `root.size()` 时，根遍历三个子节点。File 直接返回 10；src 先调用两个 File 得到 120、80，再返回 200；assets 返回 2048；根最后求和。顺序图体现的是每次对子 Component 的多态调用，客户并不展开整棵树。

在无环且每个节点只访问一次的树模型下，遍历时间为 O(N)，递归调用栈为 O(H)，其中 H 是树高。这是由代码导出的复杂度分析。若同一节点被多个父节点共享，按引用遍历可能重复计算；是否允许共享取决于领域语义。

添加打印操作时，Component 声明打印契约，File 打印自身，Directory 打印标题后递归到孩子。添加 search 同理，但必须明确搜索名、内容还是其他属性，以及返回一个结果还是全部结果。不能只写“递归就能完成”而跳过结果合并规则。

课件联系 UNIX 的 Everything is a file（“一切皆文件”）作为统一抽象的类比。它强调共同接口的价值；并不表示操作系统所有对象都具有相同实现或完全一致的操作语义。

4.5.5 任务组与表达式树

TodoItem 的 Task 叶子记录 done，TaskGroup 对孩子执行 `all(child.is_done())`。因此空任务组按 Python `all([])` 的语义为 True；这是否符合产品需求应明确说明。display 递归展示层级。与目录例子的差别在于聚合操作为逻辑 AND，而非数值求和。

表达式树把 Number 作为叶子，Add/Mul 作为组合。每个节点都实现 evaluate 和 infix。课件表达式 `(3 + 4) + (10 * 2)` 的结果为 27。下面以两种操作演示统一递归：

PYTHON

```python
class Number:
    def __init__(self, value):
        self.value = value

    def evaluate(self):
        return self.value

    def infix(self):
        return str(self.value)

class Binary:
    def __init__(self, symbol, function, left, right):
        self.symbol, self.function = symbol, function
        self.left, self.right = left, right

    def evaluate(self):
        return self.function(self.left.evaluate(), self.right.evaluate())

    def infix(self):
        return f"({self.left.infix()} {self.symbol} {self.right.infix()})"

def Add(left, right):
    return Binary("+", lambda a, b: a + b, left, right)

def Mul(left, right):
    return Binary("*", lambda a, b: a * b, left, right)
```

```
27
28 tree = Add(Add(Number(3), Number(4)), Mul(Number(10), Number(2)))
29 assert tree.evaluate() == 27
30 assert tree.infix() == "((3 + 4) + (10 * 2))"
```

这里教学重写以命名构造函数创建 Binary；课件用 Add/Mul 子类，二者都保留树状 Component 协议。新增运算可复用 Binary 的递归结构，但解析用户输入、运算符优先级和除零契约是独立问题，不能声称本例已实现完整计算器。

4.5.6 Composite 与 SOLID 的权衡

若把 add/remove/getChild 放进 Component，客户完全不必区分节点，但 File 或 Button 就要面对不支持的操作。空实现容易掩盖错误，抛异常则把问题推迟到运行时。只把子管理放在 Composite 可改善安全性，却使构建树的代码需要知道具体组合类型。

课件还提出 Composite 同时“管理孩子”和“执行操作”的 SRP 争议。它提醒我们比较设计代价，而不是把原则当作一票否决规则。若管理孩子与组合运算在领域中构成一个内聚职责，未必需要再拆；若维护与算法独立变化，则可考虑分离。

课程目标是理解透明性与安全性的真实取舍，不是得出“Composite 一定违反 SOLID，所以不能用”或“一旦是模式就无需审查”的结论。

4.5.7 Assignment 2 的规则树

EligibilityRule 约定 `exclusion_reason(question)` 返回首个排除原因或 None，并提供 render。ExcludedIdRule、TopicRule、ReuseCooldownRule 等是叶子；AllOfRule 是组合。它按声明顺序检查子规则，一旦某个返回非 None 即停止。因此顺序会影响报告中的首因，即使最终布尔资格相同。

渲染必须递归保留括号，以 AND 连接孩子，空组合渲染为 TRUE。该树不仅执行过滤，也能向使用者解释规则结构。ReuseCooldown 的边界是“距上次使用少于 N 天则排除”，恰好 N 天应允许；last_used 为 None 则允许，非法日期应明确失败。

这里 Composite 与“遇到首个拒绝就停止”的行为同时出现，但它仍以可递归组合的规则树为主要结构，不能只因短路执行就认成责任链。模式识别应结合结构与意图。

4.5.8 Command 的 Before 与角色分工

Button 若以 label 为条件调用 editor.copy_selection、paste_clipboard、save，增加 Print 就必须修改 Button。label 还混合显示文本与行为选择，改显示文案可能意外改变逻辑。

重构后，Button 保存 Command；click 只调用 execute。ConcreteCommand 保存 Editor 并调用对应方法。Client 负责装配“哪个按钮绑定哪个命令”。改变按钮触发的动作只需要改变绑定，多个按钮或快捷键也可复用同一种命令类型。

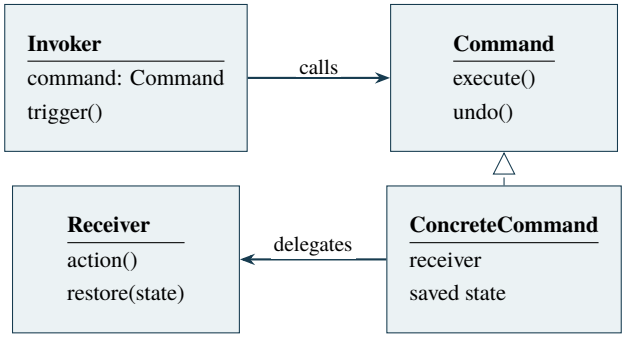


图 **6:** Command：触发者依赖命令契约，命令保留请求和撤销信息并委托接收者。

4.5.9 可撤销命令的可执行教学例

下面把状态保存机制写得比课件“删除最后 N 个字符”的简化示例更明确。它假设本地单线程编辑，并在检测到外部改动时拒绝覆盖。

PYTHON

```
class Editor:
    def __init__(self, text=""):
        self.text = text

class InsertTextCommand:
    def __init__(self, editor, text):
        self._editor, self._text = editor, text
        self._before = None
        self._after = None
        self._state = "new"

    def execute(self):
        if self._state != "new":
            raise RuntimeError("command already used")
        self._before = self._editor.text
        self._after = self._before + self._text
        self._editor.text = self._after
        self._state = "executed"

    def undo(self):
        if self._state != "executed":
            raise RuntimeError("nothing to undo")
        if self._editor.text != self._after:
            raise RuntimeError("editor changed outside history")
        self._editor.text = self._before
        self._state = "undone"

class History:
    def __init__(self):
        self._commands = []

    def execute(self, command):
        command.execute()
        self._commands.append(command)

    def undo_last(self):
        if not self._commands:
            return False
        command = self._commands[-1]
        command.undo()
        self._commands.pop()
```

```
        return True

editor = Editor("Hello")
history = History()
command = InsertTextCommand(editor, " SOFT2201")
assert editor.text == "Hello"
history.execute(command)
assert editor.text == "Hello SOFT2201"
assert history.undo_last()
assert editor.text == "Hello"
assert not history.undo_last()
```

创建 Command 只保存请求，不应立即执行主操作；执行与构造分开，才有排队或延迟的意义。execute 成功后才能加入历史，undo 成功后才应移除历史。本例用“先看栈顶，成功再 pop”保持历史；Assignment 2 用“pop 后失败时放回”，在相应假设下也可实现同一目标。

课件的 remove_last(len(text)) 在没有其他编辑时容易理解，但若外部修改已改变文档，机械删除可能破坏数据。状态快照和差量都有代价；扩展：快照耗空间，差量需要更精确的逆操作和版本约束。不能宣称有 undo 方法就已经支持可靠撤销。

4.5.10 历史、宏、队列与真实案例

请求既然成为对象，就可放入列表、作为参数传递、组合成宏或稍后执行。宏若按 A、B、C 执行，通常按 C、B、A 撤销；但“每步可撤销”仍不自动保证整个宏的失败恢复具有事务性。

VS Code 的编辑栈在课件中体现与 Command 相似的结构：编辑项保存可重做、可撤销的信息，历史服务管理调用。Django 的迁移将变更表示成 Operation 对象，CreateModel、AddField、AddIndex 等为具体命令，Migration.apply/unapply 为调用者，SchemaEditor 承担实际数据库修改；database_forwards 与 database_backwards 对应正向和反向操作。

这类案例强调结构相似，而非要求现实代码严格复制 GoF 类名。部分数据库操作不能无损撤回，必须遵守具体迁移契约；模式不会消除现实不可逆性。

4.5.11 Assignment 2 的命令一致性

题目状态命令保存 previous_status，再调用 Receiver 修改状态并记录审计。undo 恢复准确的旧状态，而非假设 approve 的相反动作总是 retire。Session 的 history 只记录成功命令；undo 失败时保持可重试或可诊断的历史。

本地实现中外部状态冲突会导致 undo 拒绝。教学上还应认识异常原子性边界：保住“历史里仍有命令”不自动意味着 Receiver 没有局部修改。若一次命令中的多个副作用可能失败，需要明确提交顺序或恢复策略；这是在课内机制之上的工程扩展。

4.6 知识关联（Connections）

Composite 把递归放到内部，Iterator 把遍历作为外部可控制的协议；它们可以同时用于一棵树。Command 将“请求是什么”对象化，Strategy 将“采用哪一种算法”对象化；形似但目的不同。

SRP、OCP 和封装解释了两种模式为何有价值；Week 7 的测试需要检验其边界：空树、嵌套、首因顺序、构造无副作用、执行失败不入历史、撤销失败保留历史。测试必须验证语义，不能只验证对象创建成功。

4.7 考试应用（Exam Application）

掌握要求：画出 Composite 的递归 Component 关系，识别 Command 的 Invoker/Receiver，按调用顺序追踪执行与撤销。**易错点**：把任何列表叫 Composite，把构造命令当作执行，把撤销理解为执行反向按钮。以下均为自编练习。

情境题 1：目录操作。开发者为每种新文件类型修改 size、search、print 三个外部函数。如何改进？增加一种全新操作是否同样容易？

解析。建立 Node 公共契约，由叶子实现自身行为、Directory 递归调用孩子。新增类型可实现既有操作而不改客户；但新增公共操作通常需要更新契约及所有实现。应比较项目更常增加类型还是增加操作。

情境题 2：按钮 add。为统一 GUI Component，所有 Button 都必须实现 add(child)，但 Button 不允许孩子。应该怎样讨论这个设计？

解析。它提高操作透明性，却让不适用操作进入叶子契约，存在 ISP 与替换风险。可把子管理限定在 Panel，构建层识别组合节点，使用层只依赖 draw。不要用静默忽略 add 来掩盖客户错误。

情境题 3：失败撤销。Session 先 pop 命令再调用 undo；undo 抛异常，历史为空。缺陷与改法是什么？

解析。未成功撤销的命令从历史丢失。可成功后再 pop，或在异常时放回并重抛。还需另外检查 Receiver 在失败前是否部分修改；历史一致性与业务状态原子性是两个问题。

情境题 4：规则首因。AllOfRule 包含 Topic 与 Cooldown，两者都会拒绝同一题目。交换顺序会不会改变行为？

解析。资格布尔值仍为不合格，但首个 exclusion reason 可能变化，后续规则是否执行也变化。若后续规则有异常或副作用，还会改变可观察结果。顺序属于这里的契约，测试应覆盖。

第 5 章　Week 5：Decorator、Proxy 与 Singleton

来源：L5 第 1-77 页；L6 第 3-12 页对幂等与 Singleton 范围的澄清。三个模式分别讨论可选行为组合、访问控制与实例唯一性，不应仅根据“包装”“缓存”等字样分类。

5.1　核心思想（Core Idea）

通过组合对象，可以在保持客户接口的同时增加行为或改变访问方式。但共同结构不意味着相同目的：Decorator 为某个对象叠加职责，Proxy 代表真实对象控制访问，Singleton 限制一个类的实例创建与访问。

本周应能追踪包装层的调用顺序和返回顺序，解释延迟初始化的资源收益与隐藏成本，并理解 Singleton 对全局状态和测试隔离的影响。

5.2　历史渊源与问题背景（Origin & Motivation）

数据存储从只需读写，演变为可压缩、可加密、可记录日志。若为每种功能组合建立子类，n 个可选行为最多形成 2 的 n 次方种组合。Decorator 将组合从类层次移到对象结构，让每个行为只有一个实现。

远程图片则面临另一问题：构造所有 RemoteImage 时就下载全部内容，大量网络工作可能从未被使用。Proxy 把昂贵对象的创建推迟到首次 display，并复用已经创建的对象。

Singleton 的原始动机是维护某个共享资源或协调点的唯一实例。但“确实只能有一个”与“到处都能方便访问”是不同需求；把两者绑定容易把全局依赖藏进应用。课件明确以批判态度讲解 Singleton，而不是推荐默认使用。

扩展背景：Python 的 @ 语法由 PEP 318 使函数/方法变换靠近定义位置，减少定义后重复赋名的繁琐（R2）。它与 GoF Decorator 有包装思想上的联系，但 Python 装饰器并不必然实现 GoF 的对象角色结构。

5.3　重要定义（Definitions）

Decorator ／装饰器模式：动态地给单个对象附加职责，通过与被装饰对象相同的契约，提供继承组合之外的灵活扩展方法。

Wrapper ／包装对象：持有另一个对象并通过委托提供行为的对象。它只是结构词，可能属于 Decorator、Proxy、Adapter 等不同模式。

Delegation ／委托：一个对象把请求或部分工作交给另一个对象完成。委托不要求继承，也不意味着调用前后没有其他处理。

Proxy ／代理模式：为另一个对象提供替身或代表，以控制对真实对象的访问。Subject 是共同契约，RealSubject 是真实对象，Proxy 负责访问策略。

Lazy initialization ／**延迟初始化**：在真正需要时才创建或初始化资源；与单纯提前创建后缓存不同。**Virtual proxy** ／**虚拟代理**：以延迟创建昂贵对象为主要目的的代理，是本周图片案例的类型。

N+1 query problem ／ **N+1 查询问题**：一次读取 N 个主对象后，又为每个对象单独执行一次关联查询，合计 N+1 次。它取决于加载策略，不能仅凭双重循环就断言发生。

Singleton ／**单例模式**：确保某个类仅有一个实例，并提供对该实例的访问方式。唯一性通常有进程、解释器或容器边界，不自动扩展到多进程和分布式系统。

Idempotence ／**幂等性**：对同一状态执行一次或重复执行某操作，其约定效果不因重复而继续改变。对状态转换 F，可写 F(F(s)) = F(s)。操作每次返回不同日志编号，仍需按具体契约判断何种效果被纳入幂等。

Race condition ／**竞态条件**：结果依赖多个执行活动的时序。两个线程都看到“尚无实例”后分别创建对象，就是 Singleton 的典型竞态。

5.4 实际意义（Practical Significance）

Decorator 常用于流处理、序列化、压缩、校验与日志；Proxy 常用于远程对象、权限检查、缓存与惰性资源。二者都让客户保留共同接口，但会增加运行时层次，使性能与异常来源不那么明显。

Singleton 有时有严格的资源唯一需求，但大多数“共享配置或服务”也可以由应用装配层创建一个对象并注入。后者既可共享同一实例，也可在不同测试中提供独立实例，从而把生命周期和依赖显式化。

5.5 完整课件内容（Lecture Reconstruction）

5.5.1 继承组合爆炸与重构目标

FileDataSource 最初只负责 read/write。增加 CompressedFileDataSource 后，再增加 EncryptedFileDataSource；若需要同时压缩和加密，又出现新子类。日志、访问控制和性能测量继续加入后，类名开始编码功能组合，维护者必须理解各组合中的重复与次序。

目标是：向单个对象增加行为、允许按需求组合、保持 DataSource 接口、尽量不修改 FileDataSource。Decorator 用组合实现这些目标；它并未取消继承，通常仍通过实现 Component 来保证替换。

5.5.2 DataSource 与装饰器层次

共同契约声明 `write(data: bytes)` 与 `read() -> bytes`。ConcreteComponent 完成基础存储；BaseDecorator 同时实现契约并保存一个 DataSource；各 ConcreteDecorator 负责一种职责，再转发请求。

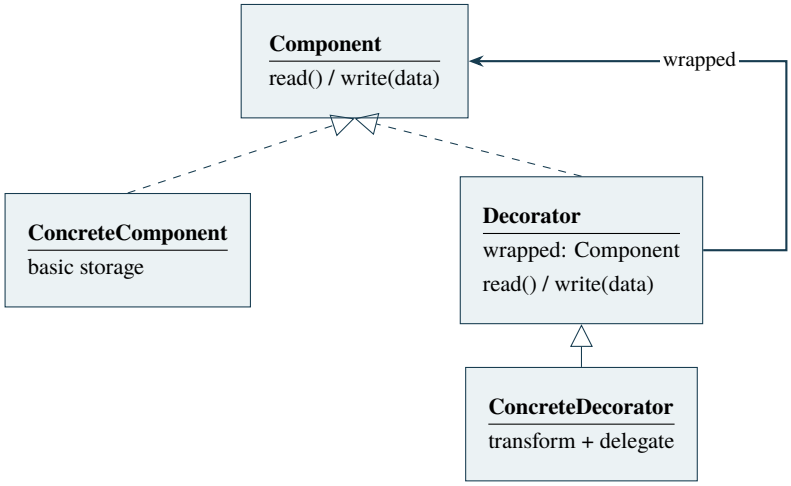


图 **7:** Decorator：每层仍满足共同接口，因此可把另一层装饰器作为被包装对象。

以下教学重写用内存存储替代真实文件，保证读者可以直接运行；压缩使用标准库 zlib。EncryptionDecorator 注入编解码函数，示例反转字节仅作为检查调用顺序的可逆测试替身，不是加密算法。

PYTHON

```
import zlib

class MemorySource:
    def __init__(self):
        self.data = b""

    def write(self, data):
        self.data = data

    def read(self):
        return self.data

class DataSourceDecorator:
    def __init__(self, source):
        self._source = source

    def write(self, data):
        self._source.write(data)

    def read(self):
        return self._source.read()

class CompressionDecorator(DataSourceDecorator):
    def write(self, data):
        self._source.write(zlib.compress(data))

    def read(self):
        return zlib.decompress(self._source.read())

class EncryptionDecorator(DataSourceDecorator):
    def __init__(self, source, encrypt, decrypt):
        super().__init__(source)
        self._encrypt, self._decrypt = encrypt, decrypt

    def write(self, data):
        self._source.write(self._encrypt(data))

    def read(self):
        return self._decrypt(self._source.read())
```

```
40
41 base = MemorySource()
42 reverse = lambda data: data[::-1]
43 source = CompressionDecorator(
44     EncryptionDecorator(base, reverse, reverse)
45 )
46 payload = b"SOFT2201 " * 20
47 source.write(payload)
48 assert base.data == zlib.compress(payload)[::-1]
49 assert source.read() == payload
```

如果改回文件存储，基础对象只需使用 rb/wb 读写 bytes。它无需知道压缩与加密的存在。新增日志装饰器也可围绕相同契约转发；这体现 OCP 与关注点分离。

5.5.3 顺序决定实际数据变换

在 `CompressionDecorator(EncryptionDecorator(file))` 中，写入从最外层进入：compress(data) → encrypt(compressed) → file.write。读取先从底层取得字节，再 decrypt，最后 decompress，逆向恢复原数据。

把外层改为 EncryptionDecorator，写入则先加密再压缩，不能假定效果相同。即使两组流程都能用匹配的逆操作读回原数据，存储大小、可观察副作用与错误位置也可能不同。日志在内层记录的可能是转换后数据，在外层记录的可能是原始输入；这种差别可能影响保密要求与调试意义。

Decorator 保持的是客户契约，不只是方法名。若原 write 承诺完整存储或抛出错误，某个装饰器静默丢掉数据就破坏替换；“每个功能各一个类”也不能自动保证 SRP，因为一个装饰器仍可能混合多个独立职责。

5.5.4 Python @ 与对象装饰的联系和区别

`@decorator` 的基本含义是定义对象后应用变换。若写 `@A`、下一行 `@B` 再定义 f，等价于 `f = A(B(f))`。应用装饰器的时机与调用包装后函数的时机要区分；最外层先接收调用，不意味着最外层最先被应用。

课程列举 property、classmethod、lru_cache，以及 Flask 路由和 Pydantic 验证装饰器。`@Test` 不是 Python 通用内建装饰器，不应照抄课件举例而假定其存在。

PYTHON

```
1  from functools import wraps
2
3  def logged(function):
4      @wraps(function)
5      def wrapper(*args, **kwargs):
6          result = function(*args, **kwargs)
7          return result
8      return wrapper
9
10 @logged
11 def double(x):
12     return 2 * x
13
14 assert double(4) == 8
15 assert double.__name__ == "double"
```

这个最小例子只展示包装与元数据保留；如果需要真正日志，可在委托前后记录。它不是要求把每个 Python 装饰器都画成 Component/BaseDecorator 的 GoF 类图。

5.5.5 Proxy 的问题：构造不应隐藏大规模工作

RemoteImage 在 __init__ 中下载资源时，创建 500 个对象就可能触发 500 次下载。客户也许只显示其中几张，却已经承担全部开销。构造器内做真实网络工作的副作用，还会使对象创建难以测试。

一种方案是把 downloaded 标记放回 RemoteImage，在 display 中判断。它可以工作，但把加载政策与图片功能混在同一个类中。单独的 ImageProxy 负责“何时创建和复用”，RealImage 负责“如何下载和显示”，可将变化原因分开。

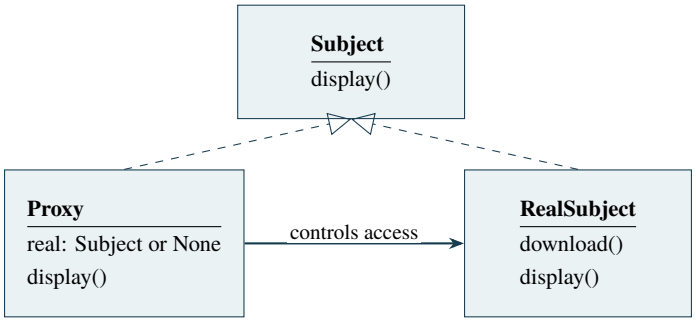


图 **8:** Proxy：客户使用共同 Subject；代理管理访问，课堂图片例在首次显示时创建真实对象。

PYTHON

```python
class RemoteImage:
    downloads = 0

    def __init__(self, url):
        self.url = url
        RemoteImage.downloads += 1

    def display(self):
        return f"display:{self.url}"

class ImageProxy:
    def __init__(self, url):
        self._url = url
        self._image = None

    def display(self):
        if self._image is None:
            self._image = RemoteImage(self._url)
        return self._image.display()

proxy = ImageProxy("photo.png")
assert RemoteImage.downloads == 0
assert proxy.display() == "display:photo.png"
assert proxy.display() == "display:photo.png"
assert RemoteImage.downloads == 1
```

示例用计数器替代真实下载，验证构造无下载、首次显示创建、随后复用。它是单线程教学模型：多个线程同时首次调用仍可能重复创建，失败后的重试语义也尚需另行定义。

5.5.6 Proxy 能决定什么，不能自动保证什么

代理可以决定是否创建真实对象、是否复用缓存对象、是否允许访问。RealSubject 和 Proxy 都满足 Subject，客户不必手动检查是否已加载。

但透明性可能隐藏延迟与失败：一个看似普通的方法或属性访问可能触发网络、数据库、权限或反序列化错误。Django LazyObject 和 SimpleLazyObject 在课件中作为现实案例；实际实现还需覆盖更多语言协议与边界，不能用十行课堂例子代表全部实现。

Decorator 通常由客户装配来增加职责；Proxy 常在内部管理真实对象的创建与访问。两者并无“只能在调用前做事”之类绝对语法界线。辨认时应说明为何需要包装，而不是只看是否保存同类型字段。

5.5.7 N+1 查询与预取的取舍

假设 `get_users()` 进行一条查询，返回 N 个用户；每次首次访问 `user.orders` 又发一条查询，那么遍历所有用户的订单需要 N+1 条查询，500 个用户就可能需要 501 条。若关系已预取、缓存或访问次数不同，实际查询数会不同。

代理使关联加载看起来像廉价属性读取，增加误判性能的风险。课件提出连接查询或批量读取，将 N 个独立查询合并。扩展分析：JOIN 可能重复主表数据，批量查询需要组织映射，提前加载也可能读取最终不用的数据。因此需依据访问模式和测量选策略。

测试可以用查询计数或可记录的假数据层验证调用次数；仅断言展示文本正确，无法发现 N+1 的性能缺陷。

5.5.8 Singleton 的结构：不仅是禁止第二次构造

“构造计数大于一就抛异常”只能阻止第二次创建，却不能让后续客户取得已存在对象。Singleton 还需提供访问入口，保存唯一实例并返回它。

常见结构是类变量 _instance 初始为 None，静态或类方法 get_instance 检查后延迟创建，再返回同一对象。classmethod 的 cls 可指向调用子类；若允许继承，要另外规定各子类是否拥有独立实例。课程只要求理解结构与局限，不要求实现复杂版本。

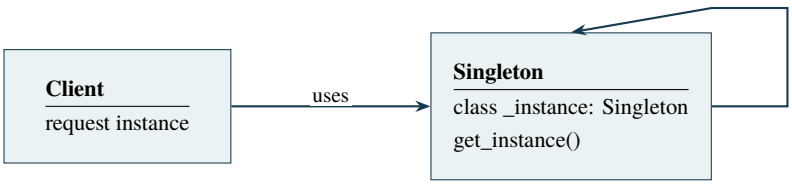


图 **9:** Singleton：类级字段保存共享实例；它不自动保证初始化及内部操作线程安全。

5.5.9 __new__ 与重复 __init__ 的陷阱

PYTHON

```
class NaiveSingleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
```

```
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self, value):
        self.value = value

one = NaiveSingleton(10)
two = NaiveSingleton(20)
assert one is two
assert one.value == 20
```

返回同一对象不等于只初始化一次。每次构造调用仍可能进入 __init__，第二次参数覆盖了第一个客户的状态。课件对应输出两次 Initializing、身份 True、第二次 URL。

一种教学修复是通过 initialized 标记使后续初始化直接返回：

PYTHON

```
class OnceSingleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self, value):
        if getattr(self, "_initialized", False):
            return
        self.value = value
        self._initialized = True

one = OnceSingleton(10)
two = OnceSingleton(20)
assert one is two
assert one.value == 10
```

这会静默忽略后续 value。真实 API 应明确“第一次配置有效”是否可接受；也可拒绝不一致配置。初始化部分失败时，标记放置及实例清理需要另外设计。以上代码不是线程安全的生产实现。

5.5.10 幂等与并发是不同性质

L6 用关灯 turn_off 与切换 toggle 澄清：关灯重复执行结果仍是关；toggle 连续两次会恢复原状，因而不是幂等。Singleton 的初始化守卫是在避免重复初始化改变对象状态，但不代表所有对象方法都自动幂等。

两个线程都看到 _instance 为 None，就可能分别创建对象。课件介绍锁与双重检查思路，并明确双重检查的技术细节不在考试范围。不要把“加了初始化标记”当作线程同步；“只有一个实例”也不能证明其内部可变状态线程安全。

L6 将幂等连接到网络重试与唯一请求标识。重复请求只应产生一次业务效果，与响应能否被重复读取、请求是否乱序等问题分别讨论；Week 6 会继续说明。

5.5.11 Singleton 的工程代价

全局入口制造隐式依赖，测试难以提供替身；共享可变状态使一个测试影响另一个；并行测试需要额外隔离；同一个类同时处理业务与自身实例管理，也可能违反 SRP。课件因此强调它经常成为反模式。

“测试不能并行”应准确理解为未隔离的共享单例状态可能使并行测试不安全；只读或充分隔离的设计不能一概而论。同理，课件图片 Proxy 使用延迟初始化，但并非所有代理都必须懒加载，Singleton 也可以提前创建；这些是课堂实现的共同点，不是两个模式的全部定义。

5.6 知识关联（Connections）

Decorator 依赖 Week 2 的可替换契约，并具体化 Week 3 的组合思想；Proxy 连接性能、最少惊讶和可测试性；Singleton 把全局状态问题带回测试隔离。它们都展示同一个规律：隐藏复杂性让客户简洁，但边界处的成本和失败仍必须有明确契约。

Week 7 可用分支测试检查 Proxy 的“未初始化 / 已初始化”，用调用记录检查 Decorator 的顺序；Week 9 的责任链也使用相似引用结构，但可能在中途停止。Week 10 将基于意图把这些结构重新区分。

5.7 考试应用（Exam Application）

掌握要求：Decorator 写入及读取次序，Proxy 的首次与后续行为，Singleton 类变量与初始化的差别。**常见错误**：只凭包装字段辨认模式，认为懒加载必然省资源，认为唯一实例必然只初始化一次或线程安全。以下均为自编练习。

情境题 1：装饰顺序。压缩包装加密再包装文件，read 时该先解压还是解密？换成加密包装压缩是否等价？

解析。写入先压缩再加密，读取必须先解密再解压。交换结构会改变正向变换顺序，通常不等价；即使都可逆，大小、日志可见内容及错误位置仍可能不同。

情境题 2：查询次数。页面先查 100 个用户，再访问每个用户的 orders。能否直接断言执行 101 次查询？

解析。只有在一次主查询且每个关系首次访问各发一次查询、未缓存或预取的假设下才成立。应检查加载策略或计数证据。改为批量读取能减少往返，但可能增加提前读取与数据合并成本。

情境题 3：全局缓存服务。团队用 Singleton 让所有测试共享缓存，某测试先写入 key，另一个测试预期不存在。如何评价及改进？

解析。这是共享状态和测试顺序耦合。可每个测试创建独立缓存并注入服务；若真实需求确实要求全局对象，则需显式重置或隔离。不能仅让测试按固定顺序运行来隐藏设计问题。

情境题 4：对象身份。 `a = Singleton("A")`、`b = Singleton("B")` 后 a is b 为 True，但 a.url 为 B。Singleton 是否已失败？

解析。实例唯一性可能成立，初始化契约却有缺陷：__new__ 复用不阻止 __init__ 重复覆盖。应规定重复初始化的行为，并用守卫、显式工厂或配置校验实现。

第 6 章　Week 6：幂等、Adapter 与 Facade

来源：L6 第 1-60 页，并衔接 L5 的 Singleton。核心问题从内部类组织移到组件边界：一个接口可能不兼容，也可能过于复杂，两种问题需要不同的职责分配。

6.1　核心思想（Core Idea）

好的边界让客户只需理解自己的目标，而不用到处知道外部系统的表示、单位和调用步骤。Adapter 负责把不兼容的接口翻译为客户需要的契约；Facade 负责把复杂子系统组织成更易使用的高层操作。

本周还要求理解重试中的幂等。接口若只描述方法名称而不描述单位、异常、状态效果与重试行为，仍可能在组件连接处产生严重错误。

6.2　历史渊源与问题背景（Origin & Motivation）

课件以 Mars Climate Orbiter 的英制 / 公制边界问题说明：两个组件分别“看起来正常”，组合仍可能错误。Adapter 的动机不是给类改名，而是把语义转换收敛在一个可以检查和测试的位置。

当客户要按顺序使用库存、汇率与支付等多个服务时，重复的编排会使每个客户都掌握内部知识。Facade 将高层用例集中，使“下单”这样的目标直接成为接口。课件以复杂控制面板的事故讨论说明复杂性会妨碍正确操作；这是一种设计动机类比，不意味着写一个 Facade 就足以实现安全系统。

网络中请求可能重试，客户端未收到响应也不代表服务端没处理。因此 API 设计需要考虑重复效果。L6 用 Stripe 的请求标识机制作为例子，重点是重复请求识别，而非要求背诵某个供应商实现。

6.3　重要定义（Definitions）

Adapter ／适配器模式：把已有类或组件的接口转换成客户期望的另一接口，让原本不兼容的组件可以协作。适配不一定只涉及参数，也可能涉及结果、异常和调用流程。

Target ／目标契约：客户已经理解并依赖的接口；**Adaptee ／被适配者**：提供所需能力但接口不兼容的现有组件；**Adapter ／适配器**：实现 Target 并调用 Adaptee，承担转换职责。

Facade ／外观模式：为子系统的一组接口提供统一的、更高层且易用的入口，使客户更容易完成目标。它通常协调多个对象，但“包住几个类”不是充分判据。

Subsystem ／子系统：围绕某组相关能力协作的一组组件。Facade 可以提供受支持的常见用例入口，不要求隐藏或禁止一切直接访问。

Orchestration ／编排：规定协作对象的调用次序、参数传递及结果组合。它是一项可以内聚的职责，不等于承担所有底层业务。

Idempotent operation ／幂等操作：重复相同操作不会在规定状态效果上继续累加；**Idempotency key** ／幂等键：用于识别同一业务请求重试的标识。唯一字符串本身不是保证，服务端还需记录并正确复用结果或状态。

Interface contract ／接口契约：包括名称、数据类型、单位、有效范围、结果、异常、调用次序和副作用的约定。相同 Python 类型并不保证相同语义。

6.4 实际意义（Practical Significance）

Adapter 常用于第三方库、旧系统、协议转换和逐步迁移；Facade 常用于复杂 SDK、资源初始化和业务编排。两者都能降低客户知识负担，但适配器需要维护转换规则，外观需要维护工作流并避免膨胀。

不要把“客户调用更少方法”直接视为安全或正确。一个外观把库存预留和付款串在一起，仍需处理付款失败和库存恢复；一个适配器把 float 转为 int，也仍需定义金额舍入及范围。

6.5 完整课件内容（Lecture Reconstruction）

6.5.1 幂等：重复相同效果，不是重复所有顺序都无关

对 Light 而言，turn_off 将状态设为 False，多次调用仍为 False；toggle 取反，多次调用会继续改变状态。幂等性针对同一操作及其约定效果。

扩展澄清：幂等不等于交换律。set_value(1) 和 set_value(2) 各自可幂等，但交换执行次序会得到不同最终值。重试去重也不能自动解决乱序请求、并发覆盖和不同请求误用同一键。

Singleton 初始化可以通过守卫避免重复配置，网络 API 可通过请求键识别重复。两者的相似处是“不要把重试当成新的业务动作”，但它们的存储范围与失败模型不同。

6.5.2 从 Assignment 1 的反馈看边界问题

L6 复盘指出同学常在 JSONDecodeError 的继承、异常重抛与跨文件路径上遇到困难，也识别了字符串身份比较、truthiness 与验证、入参修改和文档不一致。统一的教训是：真正的行为跨越接口，不能只读一个函数的表面。

课程讲模式时不严格区分 Java interface 与 Python ABC，意在强调共同契约及多态机制。但这不意味着实际语言中的继承、实现、结构化类型完全等价；写可执行代码时仍需遵守语言规则。

Singleton 的明确范围是：理解类变量保存实例、延迟初始化结构与局限，不必掌握复杂实现或双重检查细节。本教材保留 Week 5 的示例用于理解，不把它升级为额外考试要求。

6.5.3 Adapter 的 Before：让每个客户做转换

客户只会调用 `processor.pay(amount)`，旧服务只支持 `make_payment(cents)`。朴素修复是在每个 checkout 中判断类型，再乘 100 调用不同方法。这样结账流程混合服务识别与单位转换，难以扩展与隔离测试。

直接修改 LegacyPaymentService 也未必可行：它可能属于第三方库，没有源码，或已有其他客户依赖旧契约。目标是保留客户需要的入口，并让现有服务通过一个独立适配对象可用。

6.5.4 Adapter 的结构与单位转换

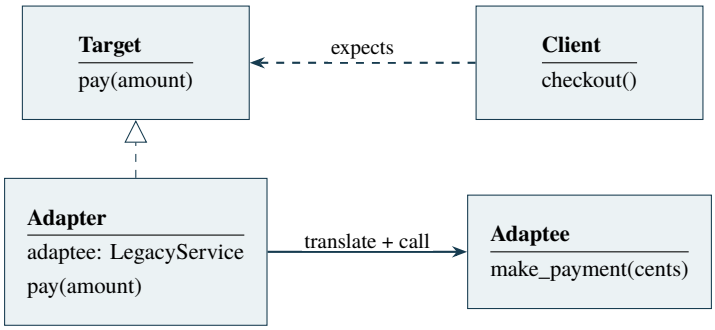


图 **10:** Adapter：客户依赖 Target，适配器转换参数、结果或其他不兼容语义。

PYTHON

```python
class LegacyPaymentService:
    def __init__(self):
        self.last_cents = None

    def make_payment(self, cents):
        self.last_cents = cents
        return {"ok": True, "cents": cents}

class LegacyPaymentAdapter:
    def __init__(self, service):
        self._service = service

    def pay(self, amount):
        cents = round(amount * 100)
        result = self._service.make_payment(cents)
        return result["ok"]

def checkout(processor, total):
    return processor.pay(total)

legacy = LegacyPaymentService()
assert checkout(LegacyPaymentAdapter(legacy), 49.95)
assert legacy.last_cents == 4995
```

本例用鸭子类型表达 Target。客户不知道 make_payment 或 cents；Adapter 知道双方契约并转换输入和结果；Adaptee 保持旧接口。若用 ABC，可让 Adapter 显式实现 PaymentProcessor，结构意图不变。

这是课程 float 金额示例的教学重写。扩展：生产金额接口通常还需要明确十进制精度、允许的小数位数、负值、最大范围与舍入政策；`round` 的默认规则不应被误当作所有业务通用的规定。

6.5.5 不兼容不止方法名

课件列出的方法名差异、美元与分、字典与对象、返回值、异常类型、调用序列、同步与异步接口，都可能由 Adapter 处理。例如目标返回统一成功对象，旧服务返回状态码；适配器可以校验并翻译，而不是把旧状态码泄漏给所有客户。

同步 / 异步转换需要真实运行模型支持；不能仅给同步函数加一个不同名字就使它异步，也不应在任意事件循环中随意阻塞。这是适配边界的工程限制说明，课程不要求实现通用异步桥。

温度案例中，旧传感器返回华氏温度，Target 需要摄氏值。转换为 C = (F - 32) × 5/9，86°F 应对应 30°C。测试应至少包含具有易知结果的值，并检查单位约定；只断言返回 float 无法发现遗漏偏移 32 的缺陷。

6.5.6 Requests 与数据库迁移案例

Requests 的 HTTPAdapter 将 Requests 的请求对象转换为 urllib3 调用，再把响应的 status、headers 和 data 组织为 Requests Response。课件中把 urllib3 称作标准库不够准确：urllib3 是独立的第三方库。这个修正不改变适配结构。

测试适配器时，应检查传给被适配者的参数，以及返回给客户的语义。仅证明两端都被调用过，不足以证明 URL、headers、单位或错误转换正确。

Webflow 的迁移案例把旧 `getPage(pageId)` 与新 `getPage(shardId, pageId)` 统一在客户熟悉的入口后。迁移期间，一些数据仍在旧库，一些已在新库；适配层根据迁移元数据路由，并补充 shardId。这样避免让整个应用掌握分片细节。

此例属于 Adapter-like 代码：现实系统可能同时包含路由、迁移状态和兼容性管理，不必为了分类而忽略这些额外职责。迁移层本身需要验证“查哪个库、如何定位、如何报告失败”。

6.5.7 Adapter、Decorator 与 Proxy 的比较

模式	客户原先的问题	包装后的主要变化	需要保留什么
Adapter	不能使用现有不兼容接口	翻译为 Target	Target 的语义
Decorator	想按对象增加可选职责	增加行为并可层叠	Component 契约
Proxy	需要控制真实对象访问	延迟、授权、缓存等政策	Subject 契约

Adapter 的两侧接口通常不同；Decorator 的包装对象本来已经符合客户接口。Proxy 也保存同类型契约，但重点是访问政策。一个实际对象可同时承担多种作用，答案应明确哪个问题由哪个职责解决，而不是只给一个标签。

6.5.8 Facade 的 Before：客户掌握所有步骤

下单代码若需要依次获取价格、换算货币、预留库存、付款和确认，每个客户都必须了解参与服务和先后次序。某次业务规则改变，多个入口可能只改了一部分，形成顺序或数据不一致。

原课件使用简化的 get_price 与 quantity；下面明确 get_price 是单价，总额必须乘数量。这是为避免读者误学计算语义而作的教学补全，不把外观模式当作金额规则的替代。

6.5.9 Facade 的 After：用高层目标组织协作

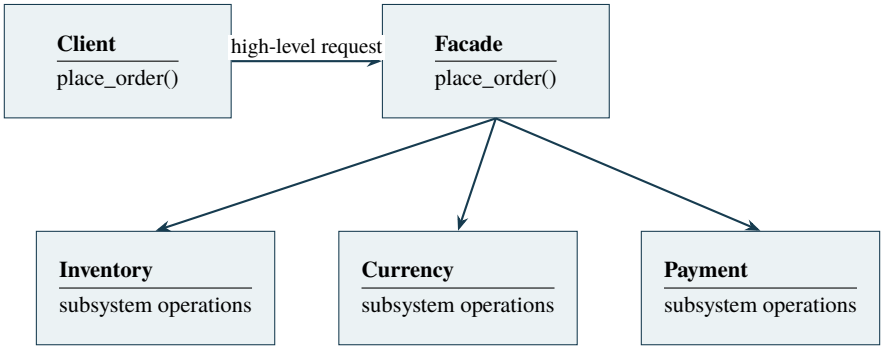


图 **11:** Facade：集中一个高层用例的编排，各子系统继续保留自己的业务职责。

PYTHON

```python
class MemoryInventory:
    def __init__(self, unit_price, stock):
        self.unit_price, self.stock = unit_price, stock
        self.events = []

    def get_price(self, product):
        self.events.append("price")
        return self.unit_price

    def reserve(self, product, quantity):
        if quantity <= 0 or quantity > self.stock:
            raise ValueError("invalid quantity")
        self.events.append("reserve")
        self.stock -= quantity

    def confirm(self, product, quantity):
        self.events.append("confirm")

class FixedCurrency:
    def convert(self, amount, source, target):
        if source == target:
            return amount
        if (source, target) != ("USD", "AUD"):
            raise ValueError("unsupported currency")
        return amount * 1.5  # teaching-only fixed rate

class RecordingPayment:
    def __init__(self):
        self.charges = []

    def charge(self, customer, amount):
        self.charges.append((customer, amount))

class OrderFacade:
    def __init__(self, inventory, currency, payment):
        self._inventory = inventory
        self._currency = currency
        self._payment = payment

    def place_order(self, customer, product, quantity, currency):
        if isinstance(quantity, bool) or not isinstance(quantity, int):
            raise TypeError("quantity must be an integer")
        if quantity <= 0:
            raise ValueError("quantity must be positive")
        unit_price = self._inventory.get_price(product)
        total = self._currency.convert(
```

```
            unit_price * quantity, "USD", currency
        )
        self._inventory.reserve(product, quantity)
        self._payment.charge(customer, total)
        self._inventory.confirm(product, quantity)
        return {"product": product, "quantity": quantity}

inventory = MemoryInventory(20, 10)
payment = RecordingPayment()
facade = OrderFacade(inventory, FixedCurrency(), payment)
assert facade.place_order("U1", "P1", 3, "AUD")["quantity"] == 3
assert payment.charges == [("U1", 90.0)]
assert inventory.events == ["price", "reserve", "confirm"]
```

它只实现课堂关注的顺利执行路径及基本参数检查。若付款在预留后失败，库存恢复仍未实现；这个限制是明确的，不能把顺利路径测试当作事务正确性证据。扩展实现可考虑释放预留、幂等付款、订单状态机等，但不把这些尚未写入示例的机制说成已完成。

客户只需知道 place_order；外观负责流程，库存仍负责库存，货币服务负责转换，支付服务负责收费。把它们放入 Facade 不代表搬走全部底层实现。

6.5.10 Facade 是否是 God Class

外观依赖多个子系统是常态，不足以单独证明上帝类。若它只协调一个内聚的下单用例，其依赖可以合理；若同时计算税、推荐商品、训练反欺诈模型、生成报表和管理员工，则有多个无关变化原因，应拆分职责或提供更聚焦的外观。

Facade 与最少知识原则相连：客户不再穿透多个内部对象。但外观本身增加一个抽象层，必须命名、维护并测试。过度简化还可能隐藏客户确实需要的控制选项；可以保留高级用法，不必把外观变成唯一强制入口。

6.5.11 Paramiko 与 logging 的案例

Paramiko SSHClient 把 Transport、Channel、SFTPClient 等内部协作包装成更易使用的入口。原先客户需要管理连接、通道、执行命令、读取和关闭；通过 SSHClient 可以用较高层的连接与执行接口完成常见目标。

logging.basicConfig 协调 Handler、Formatter 和根日志配置，体现外观式简化。它并不意味着所有日志场景只能用一个函数；复杂程序仍可直接配置各对象。具体初始化重复行为受库契约影响，本课程重点是识别高层入口与内部协作的关系。

6.5.12 选择 Adapter 还是 Facade

Adapter 关注“我有能力，但客户不认识我的接口”；Facade 关注“客户需要知道太多步骤才能达到目标”。Adapter 常围绕一个 Adaptee，Facade 常协调多个对象，但对象数量不是严格定义。

两者可组合：OrderFacade 使用统一 PaymentProcessor，而 LegacyPaymentAdapter 把旧支付服务接到这一契约。Facade 不必知道旧服务单位，客户也不必知道库存与付款顺序，各层保护不同变化。

6.6 知识关联（Connections）

Week 2 的 DIP 和最少知识解释了边界为何应稳定；Week 5 的同接口包装帮助区分本周的异接口翻译；Week 7 的集成测试特别适合发现单位、结果和异常假设不一致；Week 8 的黑盒边界测试则可检查 0、负值、范围和舍入。

幂等与 Command 的重试、撤销不同：重试同一个请求应避免重复效果，撤销则主动改变到之前状态。两者都需要保存足够事实，不能靠方法名保证。

6.7 考试应用（Exam Application）

掌握要求：识别 Target/Adaptee/Adapter，追踪输入与结果翻译；解释外观的编排职责与上帝类区别；区分幂等、去重和顺序。以下均为自编练习。

情境题 1：单位错误。旧温度服务返回 86.0，客户将其标为摄氏度。团队只把方法改名为 read_temperature。问题是否解决?

解析。没有。名称兼容不等于语义兼容，还需将华氏度转换为摄氏度，86 对应 30。应在适配边界集中转换并用已知样例断言数值及单位。

情境题 2：外观还是适配器。某类依次调用库存、付款和邮件，向客户提供 checkout。另一个类把 send_sms(number, body) 转为客户期待的 notify(user, message)。如何分类?

解析。前者主要是 Facade，简化一个多组件用例；后者主要是 Adapter，转换客户契约与现有接口。若同时附加审计日志，也可能含装饰职责，但不能因此忽略原本的适配目的。

情境题 3：付款超时。客户没收到响应便再发一次 charge。为什么不能由此推断第一笔未收费?如何利用幂等思想?

解析。请求可能已被处理而响应丢失。对同一业务请求复用唯一标识，服务端按契约识别重试并避免重复业务效果。换一个新键通常表示新请求，不能解决原请求不确定性。

情境题 4：异常路径。 Facade 已预留库存，付款抛异常；测试只检查库存减少过且通过。这个断言证明了什么，还遗漏什么?

解析。它只证明预留发生，未证明整个用例成功或失败状态合理。应检查异常是否报告、库存是否按规格恢复或保持待处理、后续确认是否被跳过。工作流集中化让这些规则更容易放在一处，但不会自动实现它们。

第 7 章　Week 7：测试基础、RIPR 与白盒覆盖

来源：L7 第 1-68 页；L8 第 7-8 页关于 coverage.py 与表达式的补充；Assignment 3 说明、被测代码及覆盖率汇总脚本。本章把“执行过多少”与“发现了什么问题”严格区分。

7.1　核心思想（Core Idea）

测试是在有限成本下选择输入、执行软件并评价行为，以获得可以支持或反驳质量声明的证据。测试的难点不仅是让代码运行，还包括选择有用的条件、制造错误可见性，以及写出能揭露错误的判定。

白盒测试使用实现结构选择测试；覆盖率显示哪些结构被执行，但不保证那些执行产生了正确结果。达到 100% 覆盖仍可能遗漏输入边界、交互或错误断言。

7.2　历史渊源与问题背景（Origin & Motivation）

课件以 Java 二分搜索的整数溢出开场：一个看起来熟悉且逻辑正确的算法，仍可因语言数值范围而失败。`(low + high) / 2` 的和可能先溢出，再进行除法。软件构造需要同时关注算法、实现语言和环境假设。

穷举测试通常不可行。一个接受三个 32 位整数的函数，单输入组合就有 2 的 96 次方种，更不用说文件、环境和执行时序。因此测试本质上是选择问题。结构覆盖提供系统化选取依据，RIPR 则解释为什么“执行了缺陷”仍可能检测不到它。

课件联系航天与金融软件失败，并引入 V 模型及不同测试层级。历史案例的教学目的，是说明测试证据需要与系统风险和生命周期配合；不要求把一次事故简化为“多写几个单元测试就能避免”。

7.3　重要定义（Definitions）

Software testing ／**软件测试**：执行软件并依据预期或规格评价其行为。只运行而不评价，不构成充分的测试判定。

Dynamic analysis ／**动态分析**：通过执行观察程序；**Static analysis** ／**静态分析**：不运行程序而检查代码或其他表示。代码审查和部分类型检查属于后者。

Fault ／**缺陷**：程序中的错误步骤、过程或实现；**Error** ／**错误状态**：由缺陷造成的内部状态偏差；**Failure** ／**失效**：观察行为偏离规格或预期。缺陷可能存在却未被执行，错误状态也可能被后续计算掩盖而不造成外部失效。

Test case ／**测试用例**：包括输入、执行条件及期望结果的一次具体检查；**Test suite** ／**测试套件**：有组织的一组测试用例。

Test oracle ／**测试判定依据**：用来判断观察结果是否符合预期的方法或规则。课件讨论“如何决定行为正确”，本书用常见术语 oracle 标识这一问题。

RIPR：Reachability（可达）、Infection（感染）、Propagation（传播）、Reveal（揭露）。它描述一个缺陷被某测试观察到所需的四个条件。

Unit / Integration / System testing ／**单元、集成、系统测试**：分别按较小组件、多个组件交互、完整系统划分被测范围。**Acceptance testing** ／**验收测试**：评价是否满足用户或业务接受条件。

Regression testing ／**回归测试**：在修改后重新执行检查，以确认以前需要保留的行为未被破坏。它与测试范围是不同维度。

Controllability ／**可控制性**：向软件提供所需测试输入和条件的便利程度；**Observability** ／**可观察性**：读取行为、状态效果或输出并作出判断的便利程度。

Coverage criterion ／**覆盖准则**：定义需要执行的一组结构要求；**Test adequacy** ／**测试充分性**：相对于特定准则，对测试是否满足要求的评价。充分性必须说“相对于什么”，不能当作绝对正确性。

Subsumption ／**包含或蕴含关系**：若每个满足准则 A 的测试套件都满足准则 B，则 A subsumes B。它比较的是满足条件的逻辑关系，不是某次报告里的百分比大小。

7.4 实际意义（Practical Significance）

测试设计应说明目的、假设、选择策略和辅助制品，如测试替身。一个业务单元若接受数据库对象与时钟，可控制性更好；若出错时只返回空值而不区分失败，可观察性变差。设计与测试不是前后完全分离的活动。

测试首先要有可信的预期。数学函数可以直接计算结果，路线规划或图像生成则较难给出唯一正确答案。扩展实践可使用约束、性质、已知案例或人工评价，但本课程在白盒覆盖题中主要要求具体且明确的断言。

7.5 完整课件内容（Lecture Reconstruction）

7.5.1 二分搜索：输入选择必须尊重语言语义

Java int 最大值为 2,147,483,647。low = 1,500,000,000、high = 1,700,000,000 时，两者的和超过范围并按 Java 整数规则溢出，`(low + high) / 2` 得到 -547,483,648，而期望中点为 1,600,000,000。

修复写作 `low + (high - low) / 2`；在二分搜索中满足 $0 \le \text{low} \le \text{high}$ 的索引前提下，它避免中间加法溢出。不要把该公式说成对任意有符号整数区间都无需条件成立。

JAVA

```java
class MidpointCheck {
    public static void main(String[] args) {
        int low = 1_500_000_000;
        int high = 1_700_000_000;
        int bad = (low + high) / 2;
        int good = low + (high - low) / 2;
        if (bad != -547_483_648 || good != 1_600_000_000) {
            throw new AssertionError("unexpected midpoint");
        }
    }
}
```

无需实际分配超大数组，也可以先用这个算术检查揭示表达式缺陷；完整搜索测试还需验证索引更新、边界与未找到时的返回。Python 整数通常不按固定 32 位范围溢出，所以照搬小整数 Python 测试不能验证同一 Java 风险。

7.5.2 测试与调试，缺陷与失效

测试用于发现或评估不符合预期的行为；调试（Debugging）定位原因并修复。失败的测试说明某个预期未成立，但还需检查是产品缺陷、测试错误、规格误解还是环境问题。

Fault → Error → Failure 是因果链而非同义词。例如某行把乘法写成加法是缺陷；执行后变量值偏离正确状态是 error；若最终打印结果不同则发生 failure。若错误变量随后被重新赋值，可能没有外部失效。

课程会宽松使用 bug 等词，但考试分析若需要区分，应依据上述层次。把“这一行有 bug”直接等同于“所有输入都会失败”是不正确的推理。

7.5.3 RIPR 的四步必要条件

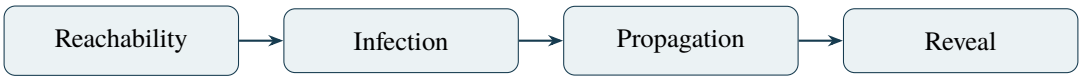


图 **12:** RIPR：执行到缺陷只是第一步，错误必须产生、传播并被判定观察。

Reachability：测试必须执行到缺陷所在位置。仅测试 factorial(-1) 的提前返回路径，就无法检测只在非负路径中的循环缺陷。

Infection：执行缺陷必须造成与正确版本不同的状态。若把 `range(2, n + 1)` 错写为 `range(2, n)`，n = 1 时两者都不进入循环，可能没有状态差异。

Propagation：错误状态必须影响后续可观察行为。若错误变量在使用前被覆盖，错误可能被掩盖。若程序打印了错误但测试只读取返回值，是否能揭露还取决于后一步。

Reveal：判定必须实际检查差异。错误 factorial(3) 返回 2，而测试只断言 `result != 3`，测试仍通过，虽然 bug 已到达、感染并传播到返回值。

PYTHON

```
def buggy_factorial(n):
    if n < 0:
        return None
    result = 1
    for value in range(2, n):  # intentional fault
        result *= value
    return result

assert buggy_factorial(-1) is None
assert buggy_factorial(1) == 1
assert buggy_factorial(3) != 3       # too weak to reveal the fault
assert buggy_factorial(3) == 2       # observed faulty result
assert buggy_factorial(3) != 6       # expected mathematical value is 6
```

最后两条是用于展示观察差异的教学断言，不是产品应采用的正确功能测试。真正的规格测试应断言 `factorial(3) == 6`，并在当前错误实现上失败。

7.5.4 越界循环与断言强度

课件的求和函数从 i=0 开始，以 `i <= len(arr)` 作为循环条件。`sum([1, 10])` 依次读 arr[0]、arr[1]，之后还试图读 arr[2]，抛出 IndexError，不能正常返回 11。应改为严格小于，或直接遍历元素。

这说明错误可通过异常暴露，而不仅通过错误返回值。测试需要规定正常输入不应抛出该异常；错误输入则应检查约定异常。一个完全不调用被测函数的“测试”，或捕获异常后无条件通过的测试，都不能提供所声称的证据。

7.5.5 V 模型与测试范围

课件 V 图将开发层次与相应验证层次联系：用户需求对应用户验收测试，软件需求对应系统测试，架构设计对应集成测试，模块设计对应单元测试，底部是代码实现与执行调试。

扩展概念澄清：Verification 关注制品是否满足已规定的要求，Validation 关注是否满足实际使用需要。一个模块完全符合错误规格，也可能未满足用户目标。因此不能把所有问题都压给最底层单元测试。

单元测试可对一个 Rule 检查结果；集成测试可检查 Adapter 与服务的单位和结果约定；系统测试可从 CLI 或界面执行完整用例；回归测试可以由上述任一层级的既有测试组成。黑盒 / 白盒依据的是测试设计信息来源，与这些范围也正交。

7.5.6 测试的心理模型：寻找反例

课件用登录场景区分“证明一个正确用户名能登录”与“尝试错误密码、空密码、锁定或删除账户”。后者寻找假设被违反的场景，不是为失败而失败，而是检查系统是否依规格拒绝或响应。

四张卡片题的规则是“若一面为偶数，另一面为蓝色”，可见面为 3、8、蓝、红。应翻 8 和红色：8 的背面可能非蓝，红的背面可能偶数，二者都能构成反例。翻 3 或蓝不能否定该单向条件。这一推理训练帮助避免只测试肯定例。

课件以“从未失败的测试是否有用”提出挑战，并介绍测试选择节省计算的实践。严谨理解是评价增量信息、风险覆盖与运行成本；一个长期通过的回归测试仍可能防止未来退化，不能仅因它从未失败就删除。

7.5.7 输入域、错误猜测与可测试性

输入域不只来自函数参数，还包括 UI 输入、文件、环境变量、系统状态和其他数据结构。Error guessing（错误猜测）依据经验列出可疑情形，例如空集合、边界、重复、缺失文件、错误编码。它可利用代码审查结论，但需要与系统化准则配合。

可控制性提高意味着测试能容易安排所需条件，例如用注入时钟制造 cutoff 边界；可观察性提高意味着能检验目标效果，例如命令执行后的状态与历史。UI 测试通常更依赖环境和操作机制，但不能把“UI 难测”当成跳过关键用户行为的理由。

7.5.8 从源代码布局到执行结构

Statement（语句）执行动作，如赋值、return、if、for；Expression（表达式）计算值，如乘法、比较和函数调用表达式。语句中可以包含表达式。视觉上的一行、一个逻辑条件和一个可执行语句不是同一种单位。

`x = 5; y = 1; z = 0` 一行含三条赋值；跨多行书写的一个推导式也不会自动变成多条顶层语句。`else` 在本课程中是 if 结构的一部分，不另算独立动作。`elif` 则可按前一判断为假后再次执行的判断来推理。

程序控制流由语言语义决定。C 不以缩进确定 if 的范围；课件改编的 Secure Transport 例子有两个 goto fail，第二个实际上无条件执行。于是 signature_ok 的判断不可达，函数始终返回初始 err=0，没有输入能让其返回 1。看起来对齐并不代表同属 if。

7.5.9 行覆盖与语句覆盖

Line coverage ／行覆盖计算选定口径下执行过的可执行源码行；注释和空行通常不计。一行只要部分执行便可能被记录为已覆盖，所以不能据此断言该行所有行为已执行。

Statement coverage ／语句覆盖要求所考虑的可执行语句被测试执行。课程手算时按语句结构讨论；实际工具常以源代码行映射近似可执行单元，报告数目可能与手工逻辑计数不同。

L7 print_x 例子中的 print(x)、y=0、if x>0、y=1、return y 按课件计五个非空执行位置；x=0 不执行 y=1，覆盖四个。这个例子用于解释口径，不应把函数定义行是否计入等工具细节泛化为所有报告的固定规则。

语句覆盖比“代码在屏幕上有几行”更贴近执行结构，却仍不要求每个决定都采用所有结果：

PYTHON

```
def flag(x):
    y = 0
    if x > 0:
        y = 1
    return y

assert flag(1) == 1
```

上述测试执行了函数内的每一条语句，但没有测试 x>0 为假的执行结果。它给出“100% 语句覆盖不必然包含 100% 分支覆盖”的反例。

7.5.10 分支覆盖与控制流图

Branch coverage ／分支覆盖检查每个所定义决策结果的流向是否出现。对 if/else，要检查 True 与 False；即使没有显式 else，False 路径仍是分支。循环通常也需考虑进入 / 继续与正常退出，具体工具口径必须说明。

PYTHON

```
def classify_text(text):
    if len(text) >= 5:
        return "long"
    elif len(text) >= 3:
```

```
        return "medium"
    return "short"

assert classify_text("hello") == "long"
assert classify_text("abc") == "medium"
assert classify_text("a") == "short"
```

第一个判断有真、假两条；只有前者为假才到第二个判断，而第二个也有真、假两条，共四个决策结果，不是因为最终有三个标签就算三条分支。

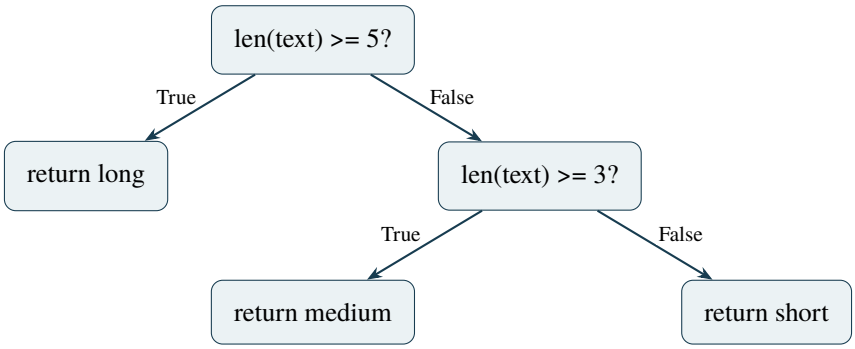


图 **13:** 两个判断产生四个分支结果，虽然最后只有三个返回类别。

对上述三例，length=5 覆盖第一个 True；length=3 覆盖第一个 False 和第二个 True；length=1 覆盖两个 False。分析应逐个判断记录，不要仅列“出现了 long/medium/short”。

7.5.11 条件、表达式与工具的边界

L7 提到 line、statement、condition、branch 等粒度。本课程重点是语句与分支。扩展辨析：复合判断中的原子条件取值与整个判断结果不是一回事；`a and b` 的 b 还可能被短路。只说“if 执行过”不能说明所有条件被评价。

L8 指出，同一行内的三元表达式以及查字典虽然可能表达多种结果，按本课程及相应 coverage.py 行间流向口径不算独立分支。不能因此推导“这些行为不需要测试”。工具没有计数的语义差异仍可由黑盒测试检查。

Exception paths 在本课程覆盖题中不要求计算，异常分支处理受工具影响。Structurally partial branches（结构上不完整分支）如 `while True` 也由工具特别处理，课件明确考试不会包含此类计数。教材说明这些边界是为了防止错误泛化，不把它们变成新增手算要求。

7.5.12 不可达代码与覆盖不能达到的要求

一个函数先排除非整数，再分别对 number<0 和 number>=0 返回，最后的 return unknown 对普通整数不可达。到达最后一条语句要求前两个数值判断都为假，与整数全序条件矛盾。应报告不可达原因，而不是无限添加随机输入。

不能为了 100% 而删去必要防御逻辑或修改被测程序来迎合报告。应检查是死代码、未建模环境、工具排除规则还是测试缺失。代码结构准则提供证据，却不能单独决定业务规则。

7.5.13 覆盖准则的包含关系

在本课程采用的普通控制流模型、相同可执行范围与可达要求下，100% branch coverage 蕴含 100% statement coverage；反过来不成立。证明思路是所有分支结果都被走过时，相应分支中的语句也已执行；反例用本章 flag 的单 if 程序即可。

不要把“测试 A 有 80% 分支覆盖、B 有 90% 语句覆盖”当成对两套测试强弱的完整比较。Subsumption 描述每一个满足准则的套件之间的逻辑关系，不比较任意两个未完全满足准则的百分比。

即使达到完整分支覆盖，也可能没有触发缺陷所需的具体值，或断言过弱而未 Reveal。覆盖主要帮助检查 Reachability，不代替 RIPR 的其余条件。

7.5.14 coverage.py 与 Assignment 3 的正确工作流

启用 `--branch` 后，Cover 列是语句与分支执行机会的合并比例，不是独立分支覆盖率。概念上为（已覆盖语句 + 已覆盖分支）除以（总语句 + 总分支）；实际工具的可执行单元依其源码映射定义。R3 给出该工具的正式说明。

Assignment 3 要求先记录原始覆盖，再补测试，且按题隔离测量。覆盖报告 JSON 中可分别取 covered_lines/num_statements、covered_branches/num_branches；本地 coverage_summary.py 正是分别输出它们。当总数为零时报告 n/a，不应随意解释为证明了全部行为。

三个被测对象是 FizzBuzz、Composite quiz results 和 Adapter assessment reporting。FizzBuzz 需覆盖组合整除、仅第一种、仅第二种、普通数字以及 n<=0；Composite 需覆盖叶子、嵌套与空组合；Adapter 需覆盖有提交、无提交和零总分。只是让每个方法调用一次，很可能仍缺关键结果。

本书不运行或修改原作业测试来替代学生提交，也不把当前目录已补充测试的覆盖数字称为“原始 starter 覆盖”。其教学目标是让读者能自己推导和复核。

7.6 知识关联（Connections）

SRP、DIP 与显式依赖改善可控制性；清楚的结果和错误接口改善可观察性；Command、Composite 和 Adapter 提供有结构的测试目标。白盒覆盖能发现未走路径，Week 8 的黑盒设计则从规格找输入边界与组合，两者共同减少盲区。

观察到错误不等于定位原因，定位并修复后也应保留回归测试。正确的循环是：明确契约 → 设计测试 → 观察失效 → 调试 → 修复 → 验证既有行为与相关风险。

7.7 考试应用（Exam Application）

课件明确覆盖题侧重指出哪些语句 / 分支被覆盖，而非手算百分比。仍应理解比例的意义，以免误读 Assignment 3 报告。以下为五道自编练习。

情境题 1：Reach 但未 Infect。 factorial 的循环终点少了 +1，测试只用 n=1 且通过。这说明哪一步不足？

解析。 输入走到有缺陷的循环范围计算，但正确与错误范围都不产生乘法迭代，未造成结果差异，缺少 Infection。n=3 可造成输出差异；还需用正确值 6 作判定才能 Reveal。

情境题 2：全部语句。 本章的 flag(1) 执行所有语句。缺失哪个分支，最小补充思路是什么？

解析。缺少 x>0 为假的流向。加入 x=0 或负数，并断言返回 0。新增测试既补结构证据，也应检查结果，而非只调用函数。

情境题 **3**：三个输出四条分支。 classify_text 的套件只用长度 5 和 1。哪些结果尚缺?

解析。第一判断真、假都已覆盖；第二判断为假已覆盖，为真未覆盖。长度 3 或 4 可以补充，并应断言 medium。不能因为两个判断都执行过就说分支全部覆盖。

情境题 **4**：**100%** 但错误。对 Adapter 的所有分支都执行过，但代码误把百分比乘以权重后忘记除 100。为什么覆盖仍可能为 100%?

解析。错误算术不一定改变控制流。若测试只检查列表长度或结果非空，RIPR 的 Reveal 失败。需要对已知权重与百分比断言精确得分，并检查总分与报告比例。

情境题 **5**：多维分类。一个测试从 CLI 启动应用，使用真实导入器和内存数据库，在重构后再次执行。它是回归测试，还是系统测试?

解析。可以同时是。系统 / 集成描述被测范围，回归描述修改后保持行为的目的。黑盒 / 白盒还取决于设计输入时使用了哪些信息，不能由“通过 CLI”直接决定。

第 8 章　Week 8：黑盒测试、时间边界与 Iterator

来源：L8 第 1-73 页；Ed「Week 8: Iterators」全部 6 页及其中 itertools、生成器代码。L8 前半部分讲黑盒测试，后半部分返回设计模式，不能把 Iterator 误归为后续周而遗漏。

8.1　核心思想（Core Idea）

即使看不到实现，也能从规格、输入域、边界与业务假设系统地选择测试。黑盒测试关注的不是“代码还有哪行没走到”，而是“哪些不同性质的输入可能使行为不同”。

Iterator 则回应另一种边界问题：客户只想逐个取得元素，不应为此知道集合是列表、字典、树还是延迟数据流。它把遍历机制与底层存储分开，并为测试复杂集合提供统一观察入口。

8.2　历史渊源与问题背景（Origin & Motivation）

日期时间看似普通数字，实际还受到日历、时区、夏令时和人为规则影响。课件从 Assignment 2 的 cooldown 提问出发，说明随机选几个“正常日期”并不能检查需求假设。使用黑盒方法前，必须先澄清系统究竟承诺什么。

等价类划分试图用少量代表值检查预期行为相似的区域；边界值方法进一步关注行为变化附近。它们不是证明同类所有输入都正确，而是为无法穷举的域制定有理由的抽样。

Iterator 的菜单案例由两家餐馆使用不同容器引出。客户最初直接遍历各自存储，随后发现 getter 虽改了访问路径，却没有隐藏表示。真正需要抽象的是“如何逐个取得下一个元素”，而不是把公开字段简单改名为 get_items。

8.3　重要定义（Definitions）

Black-box testing ／黑盒测试：依据功能规格和外部契约设计测试，不依赖程序内部实现结构。可以由开发者或 QA 执行，不限定为只测 UI。

Input domain ／输入域：被测功能可能接收的全部输入及相关条件；应根据规格区分合法域、非法域及参数之间的约束。

Normal ／正常假设：测试设计只考虑合法输入；**Robust ／健壮假设**：同时考虑合法及非法输入。这里不是说“Robust 测试就已证明软件健壮”。

Weak ／弱假设：按参数独立性质选择测试，不系统覆盖交互组合；**Strong ／强假设**：考虑参数组合可能引起不同错误，选择组合测试。Normal/Robust 与 Weak/Strong 是两个独立维度。

Random Choice ／任意选择测试设计：按课程用语，从输入域任意选值，缺少系统覆盖划分。它不特指使用随机数生成器，也不同于有概率模型的统计随机测试。

Equivalence partition ／等价类或等价分区：依据规格，预计在某一行为意义上相似的一组输入。它是有依据的测试假设，不是只按数值大小机械等分。

Boundary value analysis ／边界值分析：关注有效范围或行为类别边界及邻近值的方法，常用边界、刚内侧、刚外侧的代表值。

Iterator ／迭代器模式：提供顺序访问聚合对象元素的方法，同时不暴露其内部表示。ConcreteIterator 保存遍历状态，Aggregate 提供创建迭代器的入口。

Iterable ／可迭代对象：能够提供迭代器的对象；**Iterator ／ Python 迭代器**：支持 __next__，并由 __iter__ 返回自身的对象，耗尽时抛 StopIteration。可迭代对象不必自身保存当前位置。

Generator ／生成器：使用 yield 暂停和继续执行、逐个产生值的对象机制。它通常是迭代器，不等于一次性预先构造所有元素的列表。

8.4 实际意义（Practical Significance）

黑盒方法适合外部 API、需求验收和实现尚未完成时的测试设计；也可用于单元、集成与系统测试。它能够发现代码覆盖忽略的需求缺口，例如 100 分未被任何等级区间接纳、65 岁同时属于两档票价。

Iterator 适合多种菜单、集合及延迟数据源。客户稳定后，内部表示可以改变，遍历也可加入筛选或批量加载。代价是额外状态、耗尽语义以及“遍历时集合发生变化”的契约；统一接口不会自动解决这些问题。

8.5 完整课件内容（Lecture Reconstruction）

8.5.1 时间并不总是单调向前的墙上读数

`datetime.now()` 读取民用时间，可能受时区、夏令时或系统时钟校正影响。用它相减衡量一个任务实际耗时，结果可能令人意外。课件用夏令时回拨举例；具体回拨发生在哪个小时取决于地区，不应把示意中的 02:00→01:00 当作所有时区规则。

测量经过时长通常应使用 `time.monotonic()`。其值的绝对起点没有可移植业务意义，差值才适合计时（扩展依据 R4）。它不适合直接保存“明天悉尼晚上 11 点”的日历事件。

PYTHON

```
import time

start = time.monotonic()
total = sum(range(1000))
elapsed = time.monotonic() - start
assert total == 499500
assert elapsed >= 0
```

不应断言 elapsed 必定大于 0，因为时钟分辨率可能使极短操作的读数相同；也不应把单调时钟读数当作可跨机器比较的 UTC 时间戳。

8.5.2 闰秒、跳日与时区规则

课件问 23:59:59 之后是什么：在相关 UTC 闰秒情形中可出现 23:59:60，否则通常进入 00:00:00。编程语言与时间戳表示不一定直接支持秒值 60；例如 Python datetime 不直接接受它。测试前必须确定所测时间模型，不能把民用时间的所有性质直接套入库类型。

Samoa 跳过一个日历日期的案例说明：地区并不保证经历每个当地日期，UTC offset 也不只因夏令时而变。课件的时区数据库与 Cloudflare 闰秒案例说明时间规则具有维护与平台依赖，不是固定不变的自然常数。

L8 的“加一天是否等于加 24 小时”取决于业务语义。跨夏令时切换时，保持次日同一本地时刻，与让绝对时间增加 86,400 秒可以不同。测试应明确要验证哪一种操作，不能先执行库默认行为再把结果视为需求。

8.5.3 日历事件的存储契约

“悉尼时间晚上 11 点完成测验”通常需要绑定地区时区及其规则；“旅行时每天中午服药”可能表示随当地时间重复，也可能要求固定间隔。两种合理需求不能只用“转为 UTC 就全部解决”概括。

扩展建模建议：区分 Instant（绝对时刻）、Local date/time（当地日期时间）、Time zone（地区规则）和 Duration（经过时长）。这些词用于澄清课件问题，不将其扩展为课程另一个完整时间 API 单元。

对 `add_month(Jan 31, 1)`，结果可能按规格截到二月最后一天、拒绝操作或采用其他规则。若需求未定义，测试者应指出歧义，而非凭个人直觉断言二月 28 日或三月 3 日必然正确。

8.5.4 四种假设组合

组合	选值目标	必须先说清的前提
Weak Normal	合法参数区域的代表值，独立考虑各参数	不系统要求交互组合
Weak Robust	加入非法区域，便于分别定位校验问题	本教材例子一次引入一个非法参数
Strong Normal	合法区域间的组合	参数交互可能导致错误
Strong Robust	合法与非法区域的组合	接受更高组合数量与判定成本

对于 `days_in_month(year, month)`，month=13 属非法输入；year 与 month 又会交互，因为二月天数依赖闰年。只分别测试一个闰年和二月但从不把二者放在一起，并没有检查该交互。

课件用 quantity 1-100、discount 0-50 展示不同选值，Weak Robust 示例也出现同时非法的 (-5,70)。这可说明“开始考虑非法值”，但不应把该单一例子当作严格的独立故障计数规则。本教材采用一次一个非法参数作为可解释的弱健壮示例，强健壮再系统组合。

8.5.5 Random Choice 的价值与局限

对 quadratic_root(a,b,c) 任意选 (2,7,3)，或者对价格选 (37,10)，能快速开始测试，却未给出覆盖判据。测试者不知道是否遗漏判别式零、非法数量或折扣端点。

Strong Normal 可以组合选定的合法代表值，Strong Robust 再组合非法值，但只要代表值选择没有依据，数量增加也不保证覆盖重要性质。黑盒设计的下一步，是为“哪些输入在规格上有不同意义”建立分区。

8.5.6 等价类划分的步骤

先确定输入域和需求；按预期行为把域划分成互不重叠、合起来覆盖所考虑范围的区域；从每个区域选择代表；若参数交互会改变行为，则进一步组合或细分。

课程成绩例子可规定：

输入区间	输出	一个内部代表
score < 0	非法	-1
$0 \leq$ score < 50	FA	25
$50 \leq$ score < 65	PS	55
$65 \leq$ score < 75	CR	70
$75 \leq$ score < 85	DI	80
$85 \leq$ score ≤ 100	HD	90
score > 100	非法	101

原 L8 表格最后一档写为 [85,100)，却又把非法定义为 >100，造成 100 未被覆盖。本表明确包含 100，这是依据课程“0-100 分”定义作的边界澄清；如果某题另行给出不同规格，必须按题目分析并指出冲突。

每个等级区间是一个行为等价类，不要求内部所有值都返回完全相同的附加数据；它们在所测等级分类意义上相似。对于字符串、枚举和复合记录，也可按格式有效性、是否存在及角色等划分，不限于连续数轴。

8.5.7 日期分区不能忽略闰年规则

“31 天月、30 天月、普通年二月、闰年二月”比“任选几个月份”更有理由。课件进一步用 2023、2024、1900、2000 揭示只按是否能被 4 整除并不充分。

公历规则是：能被 4 整除，且不能被 100 整除，除非也能被 400 整除。因此 2024 与 2000 是闰年，1900 与 2023 不是。此规则提供了额外的年分区；它不是从“多数年份表现相同”就可以跳过的细节。

PYTHON

```python
def is_leap(year):
    return year % 4 == 0 and (
        year % 100 != 0 or year % 400 == 0
    )

def days_in_month(year, month):
    if isinstance(year, bool) or not isinstance(year, int):
        raise TypeError("year must be an integer")
    if isinstance(month, bool) or not isinstance(month, int):
        raise TypeError("month must be an integer")
```

```
    if not 1 <= year <= 9999 or not 1 <= month <= 12:
        raise ValueError("date outside teaching domain")
    if month == 2:
        return 29 if is_leap(year) else 28
    return 30 if month in (4, 6, 9, 11) else 31

assert days_in_month(2026, 1) == 31
assert days_in_month(2026, 4) == 30
assert days_in_month(2023, 2) == 28
assert days_in_month(2024, 2) == 29
assert days_in_month(1900, 2) == 28
assert days_in_month(2000, 2) == 29
```

此教学实现显式限定 year 1-9999，避免把未说明的负年份或天文历法混入测试预期。类型与范围的非法输入还应另测；上面六条只验证合法行为分区。

8.5.8 边界值分析：检查变化发生的位置

对整数合法区间 [min,max]，可考虑 min-1、min、min+1、max-1、max、max+1；是否包含外侧值取决于 Normal/Robust。还应有内部常规值作为基线。对多个行为区间，要检查每个转换点，而不只检查全域最小和最大值。

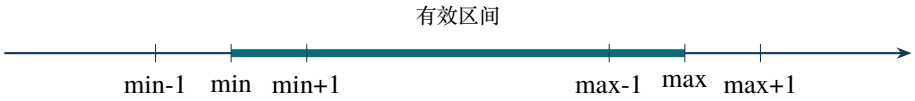


图 **14:** 整数区间的边界及邻近值：是否选择外侧值取决于 Normal/Robust 假设。

ticket_price(age) 若规定 0-4 免费、5-17 儿童、18-64 普通、65 及以上老年，可以围绕 0、4/5、17/18、64/65 选择相邻输入。课件列出 0、1、3、4、5、6、16、17、18、19、63、64、65、66；若需 Robust，还应加入 -1 等非法值并明确错误预期。

L8 下一页故意或不一致地写“18-65 普通”及“≥ 65 老年”。65 同时落入两档，应先澄清优先级或修正规格，而不是让实现分支顺序暗中决定需求。这正是测试需求分析的实际价值。

连续数值没有通用的“刚好加 1”；应按数据精度选择最邻近可表示值或有意义的业务间隔。边界也可能是长度、集合大小、时间截止点或状态迁移，并不限于数值上下界。

8.5.9 强假设下的边界依赖

`is_valid_date(year,month,day)` 的 day 上界依 year 与 month 而变：2023 二月 28，2024 二月 29，四月 30，五月 31。Weak 只独立改变 day 容易遗漏这种依赖；Strong 要在不同年 / 月类别中检查边界。

可选用 (2023,2,28)/(2023,2,29)、(2024,2,29)/(2024,2,30)、(2024,4,30)/(2024,4,31)。输入值需配明确预期 True/False。选择多少例取决于规格、分区及交互假设，没有一个脱离这些前提的“所有日期函数最少 N 个测试”。

8.5.10 Iterator 的 Before：getter 没有隐藏遍历

PancakeHouseMenu 使用 list；DinerMenu 使用按名称索引的 dict。Waiter 最初直接读 _items；改用 get_items_list/get_items_map 后，仍要决定用索引还是 items()，并承担筛选、排序与遍历规则。

问题包括依赖具体表示、封装不足、重复循环，以及存储改变会迫使 Waiter 修改。统一 getter 若仍返回内部可变容器，也不能保护状态。设计目标是让每个菜单提供统一的遍历对象。

8.5.11 GoF 风格迭代协议

Iterator 声明 has_next 与 next；ConcreteIterator 保存当前位置；Aggregate 提供 create_iterator。列表迭代器用索引，字典迭代器可保存 values 的快照；客户统一先检查 has_next，再取 next。

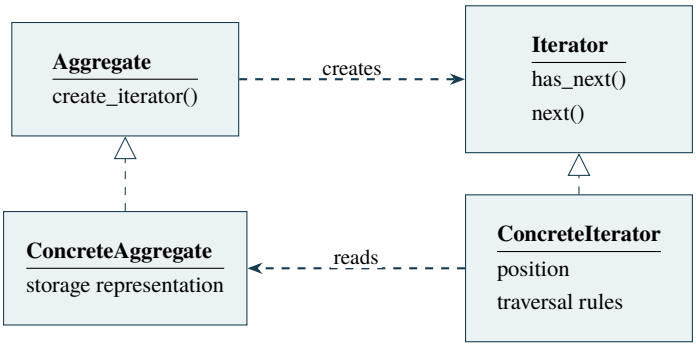


图 **15:** Iterator：遍历位置属于迭代器，客户不必了解聚合内部容器。

PYTHON

```
class MenuIterator:
    def __init__(self, items):
        self._items = tuple(items)
        self._index = 0

    def has_next(self):
        return self._index < len(self._items)

    def next(self):
        if not self.has_next():
            raise StopIteration
        item = self._items[self._index]
        self._index += 1
        return item

class ListMenu:
    def __init__(self, items):
        self._items = list(items)

    def create_iterator(self):
        return MenuIterator(self._items)

class DictMenu:
    def __init__(self, items):
        self._items = dict(items)

    def create_iterator(self):
        return MenuIterator(self._items.values())

def collect_menu(menu):
    iterator = menu.create_iterator()
    result = []
    while iterator.has_next():
        result.append(iterator.next())
    return result
```

```
assert collect_menu(ListMenu(["pancake", "waffle"])) == [
    "pancake", "waffle"
]
assert collect_menu(DictMenu({"main": "soup"})) == ["soup"]
```

本例统一把值复制到 tuple，说明快照型遍历的取舍：调用者不会因后来修改容器而改变本次序列，但创建时要 O(N) 额外空间，并非懒加载。菜单元素本身仍可能是可变对象。

8.5.12 Python 的 __iter__、__next__ 与 EAFP

Python 采用取得下一个值，耗尽时抛 StopIteration 的协议，而不是强制 has_next。for 循环先取得 iter(obj)，反复调用 next，捕获耗尽后结束。EAFP（Easier to Ask for Forgiveness than Permission）倾向直接尝试并处理适当异常；LBYL（Look Before You Leap）先检查条件。二者应按场景使用，不等于支持捕获一切异常。

PYTHON

```
class ListIterator:
    def __init__(self, items):
        self._items = tuple(items)
        self._index = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self._index >= len(self._items):
            raise StopIteration
        item = self._items[self._index]
        self._index += 1
        return item

class Menu:
    def __init__(self, items):
        self._items = tuple(items)

    def __iter__(self):
        return ListIterator(self._items)

menu = Menu(["soup", "salad"])
assert list(menu) == ["soup", "salad"]
assert list(menu) == ["soup", "salad"]
it = iter(menu)
assert next(it) == "soup"
assert list(it) == ["salad"]
assert list(it) == []
```

Menu 每次创建新迭代器，支持独立遍历；已经耗尽的 it 不会因再次调用 iter(it) 自动重置。若集合自己同时实现 __iter__/__next__ 并返回自身，可能只允许共享一个当前位置，嵌套循环或重复遍历会相互影响。课件允许这种合并实现，但读者应理解其限制。

8.5.13 itertools：统一协议使组合成为可能

Ed 补充 `combinations(players,2)` 产生无重复二人组合；`zip_longest` 对齐长度不同的序列并填充值；滑动窗口可由 `tee` 复制迭代视角、逐个错位，再由 zip 对齐。

PYTHON

```
from itertools import combinations, zip_longest, tee, islice

assert list(combinations(["A", "B", "C"], 2)) == [
    ("A", "B"), ("A", "C"), ("B", "C")
]
assert list(zip_longest(["A", "B", "C"], [85, 92],
                        fillvalue="N/A"))[-1] == ("C", "N/A")

def sliding_window(iterable, size):
    if size <= 0:
        raise ValueError("window size must be positive")
    streams = tee(iterable, size)
    shifted = [islice(stream, offset, None)
               for offset, stream in enumerate(streams)]
    return zip(*shifted)

assert list(sliding_window([1, 2, 3, 4, 5], 3)) == [
    (1, 2, 3), (2, 3, 4), (3, 4, 5)
]
```

这是 Ed 算法的等价教学重写，使用 islice 表达跳过前缀，并校验窗口大小。tee 并不保证无限输入完全不占额外内存：多个分支消费速度差距大时，它需要缓冲尚未被较慢分支读取的元素。这属于理解惰性操作代价的扩展。

8.5.14 生成器与无限序列

PYTHON

```
from itertools import islice

def count_up_to(n):
    for value in range(1, n + 1):
        yield value

def infinite_ones():
    while True:
        yield 1

assert list(count_up_to(5)) == [1, 2, 3, 4, 5]
assert list(islice(infinite_ones(), 4)) == [1, 1, 1, 1]
```

yield 在每次取得下一个元素时继续执行，避免先建立巨大列表。它节省的是预先保存全部输出的需求，不保证每个生成器内部都没有大量缓存，也不保证无限迭代会自动终止。

Ed 用构造万亿元素列表与无限 ones 对比，要求客户自己规定停止条件。遍历大型数据时，还要定义资源何时释放；不能因为对象是迭代器就假定读取零成本。

8.6 知识关联（Connections）

黑盒测试补足 Week 7 的结构盲区；日期时间说明最少惊讶与需求契约的重要性；Iterator 同时使用封装、DIP 和 OCP。它也可以遍历 Composite 树，但“整个树执行同一操作”与“客户逐个取得节点”是不同职责。

生成器、Proxy 和 Iterator 都可能延迟工作；这种共同机制不使它们成为同一模式。分析应分别说明访问控制、遍历协议或数据生产的目的。

8.7 考试应用（Exam Application）

掌握要求：四种假设、分区与边界的区别、日期交互、迭代器角色与耗尽。**常见错误**：只选区间内部值、把不存在的需求当预期、以为 getter 隐藏了表示、以为迭代器能自动重用。以下为五道自编练习。

情境题 1：成绩缺口。规格最后档写 [85,100)，非法为 >100。你应该如何为 100 写断言？

解析。先报告规格未覆盖 100，不能凭实现输出自动建立 oracle。若课程或题目已明确 0-100 合法，再据该权威要求把最后档补为闭区间。测试边界的目的包括发现规格歧义。

情境题 2：闰年交互。测试包含 (2024,1) 和 (2023,2)，能否说明已测闰年二月？

解析。不能。参数分别出现不代表组合出现。应加入 (2024,2)，并包含 1900/2000 以区分世纪规则。选择理由来自日期规格，而不是覆盖工具显示哪个 if 未执行。

情境题 3：截止时间。每天“当地时间中午提醒”一律存为首次换算的 UTC 时间，并以 24 小时重复。有什么潜在问题？

解析。跨夏令时或旅行后，固定 UTC 间隔可能不再是期望当地中午。应明确事件绑定时区或跟随用户所在地的规则，再设计转换与重复测试；不是所有提醒都采用同一种正确模型。

情境题 4：共享迭代位置。 Menu.__iter__ 返回 self，self.current 保存索引。两个客户各取一个 iter(menu)，是否拥有独立遍历？

解析。通常没有，两个变量指向同一状态对象。一个客户推进会影响另一个。让可迭代集合每次返回新的迭代器，才能提供独立位置；若设计本来是一次性流，应明确这一契约。

情境题 5：只补覆盖。一个 65 岁票价缺陷通过修改条件即可修复，但全套分支覆盖本来已达 100%。为什么黑盒方法仍有价值？

解析。分支可能用 30 和 80 等内部值覆盖，却没检查 64/65/66。边界值方法从规格选这些值，能揭露 off-by-one 或重叠规则；结构覆盖没有保证边界输入被执行。

第 9 章　Week 9：责任链、Observer 与 Strategy

来源：L9 第 1-82 页。目录和旧周计划曾把责任链列在 Week 8 后半，但它在当前 Lecture 9 中有完整讲授，本章按实际内容归属 Week 9。

9.1　核心思想（Core Idea）

本周把三种变化分别封装：谁处理一个请求、谁对状态变化作出反应、当前使用哪种算法。责任链把接收者选择分散到可组合处理器；Observer 让主体通过共同通知契约连接多个反应者；Strategy 把可替换行为交给独立对象或函数。

必须能说明调用何时停止、通知谁、状态归谁、算法由谁选择。三者都减少集中条件判断，却各有不同的执行与错误传播语义。

9.2　历史渊源与问题背景（Origin & Motivation）

工单系统初期用 if/elif 处理密码、账单和技术问题。每增加一种类型，就修改同一处理函数；责任链把匹配与处理放入各处理器，并把顺序作为装配关系。

下载管理器开始只更新进度条，随后加入速度、通知、日志、指标、病毒扫描和历史面板。若主体直接认识每个反应组件，就不断扩大依赖。轮询可以让组件自己检查状态，但带来重复检查和延迟。Observer 改为主体通知已登记的观察者。

FlyingCar 则以 mode 决定驾驶或飞行，同时接收道路、天气和空域参数。许多参数在某一模式中无意义。Strategy 把算法及其专有状态一起移出，让上下文只负责使用当前策略。三种模式均来自具体维护压力，不是为模式分类而新增类。

9.3　重要定义（Definitions）

Chain of Responsibility ／责任链模式：让多个处理者有机会接收请求，避免发送方直接绑定某个具体处理者；请求沿处理关系传递，直到按约定处理或终止。

Handler ／处理者：定义处理请求与转交后继的共同操作；**Successor ／后继**：当前处理者不能或不应终结请求时的下一个处理者。

Observer ／观察者模式：定义对象间一对多的依赖，使主体状态变化时，已登记的依赖者得到通知并相应更新。

Subject / Observable ／主体或被观察者：维护状态与订阅关系，触发通知；**Observer ／观察者**：实现共同 update 契约；**Subscribe / Unsubscribe ／订阅与退订**：建立或解除具体实例的通知关系。

Polling ／轮询：消费者按周期或循环主动读取状态；**Push model ／推送模型**：通知中直接携带所需数据；**Pull model ／拉取模型**：通知携带主体或可访问上下文，观察者再读取需要的状态。Observer 的 Pull 仍由事件触发，不等于不断轮询。

Strong / Weak reference ／强引用与弱引用：强引用维持对象的可达关系；弱引用允许所指对象在不再受其他强引用需要时被回收。弱引用不是自动取消所有资源职责的通用保证。

Strategy ／策略模式：定义一族算法，分别封装并使其可互换，让算法独立于使用它的客户变化。

Context ／上下文：保存并委托当前 Strategy 的对象；**ConcreteStrategy** ／具体策略：实现特定算法及其专有参数。策略可由类对象表达，也可由满足调用契约的函数表达。

9.4 实际意义（Practical Significance）

责任链适合请求路由、验证、日志或中间件等具有可组合处理机会的场景；Observer 适合界面、进度、事件与缓存失效等一对多反应；Strategy 适合运输、定价、排序、统计等可互换算法。

模式减少中心对象的具体知识，却把复杂性转移到装配、生命周期或选择策略上。若顺序、异常、重入和资源释放没有明确契约，类图再漂亮也会产生难以追踪的问题。

9.5 完整课件内容（Lecture Reconstruction）

9.5.1 责任链的 Before：集中请求分类

工单函数依次检查 is_password_reset、is_billing_issue、is_technical_issue。每种分支包含匹配与工作，增加类型或调整优先级都修改同一函数。客户可能还需知道应该直接调用哪个处理类。

重构目标是让客户只向链入口发请求，由处理器自行决定。Handler 可提供模板式 handle：能处理则 process，否则交给 next。不同处理器只定义自己的匹配与工作。

9.5.2 责任链的结构、终止与未处理结果

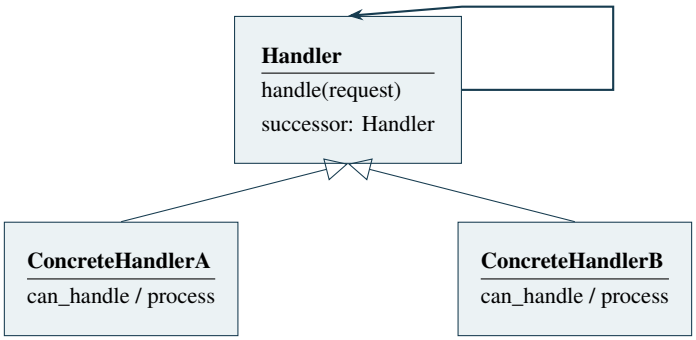


图 16: 责任链：共同 handle 契约与后继关系，具体停止政策由实现和需求决定。

PYTHON

```
class Handler:
    def __init__(self, kind, action, successor=None):
        self._kind = kind
        self._action = action
        self._successor = successor

    def handle(self, request):
        if request["kind"] == self._kind:
```

```
            return self._action(request)
        if self._successor is not None:
            return self._successor.handle(request)
        return "unhandled"

chain = Handler("password", lambda req: "reset",
        Handler("billing", lambda req: "bill",
        Handler("technical", lambda req: "support")))

assert chain.handle({"kind": "billing"}) == "bill"
assert chain.handle({"kind": "other"}) == "unhandled"
```

这是缩小的教学实现，以构造参数表达具体处理逻辑；课件采用 PasswordHandler/BillingHandler/TechnicalHandler 子类，核心传递机制相同。客户装配顺序后只调用入口。

账单请求先由 PasswordHandler 检查、不匹配，转给 BillingHandler；后者处理并返回，TechnicalHandler 不执行。若无人匹配，必须有明确约定，例如返回未处理结果或抛出专门异常。不能把 None 同时当作“处理成功但无结果”和“无人处理”而不说明。

9.5.3 次序、不同实现与过度使用

多个处理器可能都能处理某请求，此时首个匹配决定结果；调整顺序可能改变语义。验证链也可能规定每个处理器执行后继续，形成不同变体。课件明确“处理后停止”不是所有责任链实现的绝对要求。

外部循环遍历 Handler 列表也可以组织处理机会，但是否等价取决于是否共享终止和结果传递契约。仅有 `for handler in handlers: handler.handle(req)` 且无终止机制，通常会让全部处理器执行，不能声称自动等价于首个匹配的链。

如果所有阶段固定且每次必须执行，直接管道（Pipeline）可能更清楚；如果仅有几个永久固定的条件，额外间接层未必值得。扩展还需防止循环后继使请求无限递归，但这不是课堂示例已经实现的检测。

9.5.4 责任链与 Decorator

两者都可保存同接口对象并转发。Decorator 的目的通常是叠加职责，调用最终到达核心组件，返回时逐层后处理；责任链主要是给各处理器机会，可能提前终止。

同一个请求可能经过相似的箭头结构，但“是否一定转发”“谁完成主要工作”“顺序是否代表优先级”不同。不能把“有 next 字段”当作完整模式定义。

9.5.5 下载主体的依赖膨胀与轮询替代

最初 DownloadTask 保存 ProgressBar 并在 set_progress 中调用 update。加入 SpeedDisplay、DownloadLogger、Notifier 后，主体还要直接依赖它们。每增加一种观察效果都修改主体，违背预期的扩展方向，也混合状态管理与具体呈现责任。

轮询让观察组件循环读取 download.progress。课件指出其重复检查、CPU 消耗、更新延迟、旧状态保存和周期管理问题。带休眠的定时轮询不必都是忙等待，但仍存在周期与延迟取舍；不是所有系统都能或必须支持事件推送。

Observer 让主体维护共同观察者契约；新反应只需实现契约并登记，不必修改下载主体的具体通知逻辑。

9.5.6 Observer 的结构与可运行示例

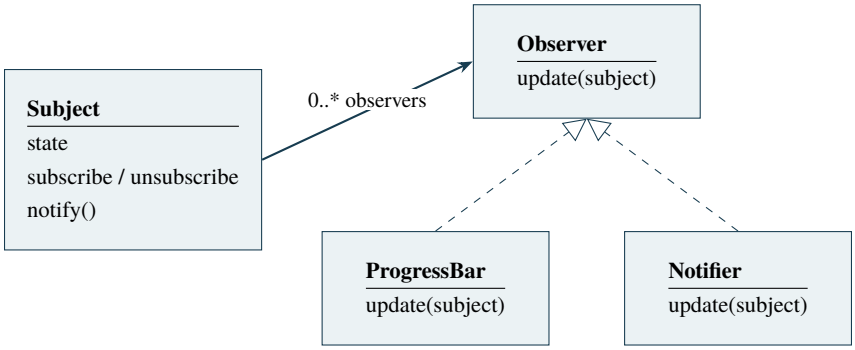


图 **17:** Observer：主体依赖共同通知接口，订阅关系连接的是具体对象实例。

PYTHON

```
class DownloadTask:
    def __init__(self, filename):
        self.filename = filename
        self.progress = 0
        self._observers = []

    def subscribe(self, observer):
        self._observers.append(observer)

    def unsubscribe(self, observer):
        self._observers.remove(observer)

    def set_progress(self, progress):
        if not 0 <= progress <= 100:
            raise ValueError("progress outside 0..100")
        self.progress = progress
        for observer in tuple(self._observers):
            observer.update(self)

class ProgressRecorder:
    def __init__(self):
        self.values = []

    def update(self, task):
        self.values.append((task.filename, task.progress))

class CompletionNotifier:
    def __init__(self):
        self.completed = []

    def update(self, task):
        if task.progress == 100:
            self.completed.append(task.filename)

task = DownloadTask("lecture.mp4")
bar = ProgressRecorder()
notifier = CompletionNotifier()
task.subscribe(bar)
task.subscribe(notifier)
```

```
task.set_progress(30)
assert bar.values == [("lecture.mp4", 30)]
assert notifier.completed == []
task.unsubscribe(bar)
task.set_progress(100)
assert len(bar.values) == 1
assert notifier.completed == ["lecture.mp4"]
```

与课件一样，主体传入自身，观察者从 task 读取需要的状态，即 Pull model。notifier 在 30 时也收到通知，只是不产生完成效果。务必区分“被通知”与“输出或执行了某种动作”。

本例扩展使用 tuple 快照，使本轮通知期间的订阅列表修改不直接破坏遍历；其语义是本轮开始时已登记的对象仍在快照中。重复订阅会重复通知，unsubscribe 删除一个匹配项；生产接口可以选择去重，但必须明确。

9.5.7 订阅关系属于对象实例

video 与 log_file 是两个独立 DownloadTask。给 video 登记 ProgressBar，不会让它自动观察 log_file。ConcreteObserver 类相同也不表示所有实例共享订阅。

Client 负责创建对象、订阅和退订；配置完成后，具体通知时机由 Subject 的状态变化流程决定。客户不应在每次 set_progress 后手动更新所有视图，否则又把耦合放回装配代码。

状态更新通常应先成立，再通知观察者；否则 Pull 模型可能读到旧状态。通知是否只在值变化时触发、重复 100 是否应产生重复完成通知，都是需要额外规定的契约。本例每次 set_progress 都通知，没有实现“只通知一次完成”的政策。

9.5.8 Push 与 Pull 的选择

Pull 的 `update(subject)` 让观察者自己读取所需字段，增加灵活性，却使其了解主体接口，并可能读取到不一致的变化中状态。Push 的 `update(progress=30)` 传输必要数据，减少读取，却可能因新增需求而扩大事件参数。

扩展：不可变事件快照可以使一次通知对应明确事实；它需要定义事件结构及版本，不是 Observer 模式自动提供。选择时应考虑观察者到底依赖完整对象还是某次变化的数据。

Observer 的 Pull 不等于 Polling：前者是主体先通知后读取，后者是观察方不断询问。用相同“拉取”中文词而不解释触发方式，容易把两种成本模型混淆。

9.5.9 强引用、退订与垃圾回收

Subject 的观察者列表持有强引用。即使局部变量 bar 设为 None，观察者仍可能从 download._observers 可达，因而不会因该局部变量消失就被回收。若整个主体也不再可达，情况不同；“内存泄漏”通常指长期存活主体保留了已经不需要的观察者。

退订移除这条关系；若没有其他强引用，观察者才可能被回收。Python 的回收时机及资源释放不应仅靠把变量设 None 来推断。

课件用 Django Signal 的弱引用说明一种生命周期策略：接收者可以不因订阅本身被强行保留。但观察者若还有文件、线程或其他资源，弱引用不会代替这些资源的显式管理；同时没有其他强引用的临时接收者也可能过早消失。

9.5.10 通知失败、顺序与重入

课堂实现按订阅顺序遍历，但观察者应尽量独立，不依赖另一个观察者先更新。否则系统形成难以看见的额外依赖，更改订阅顺序就会改变结果。

若某个观察者抛异常，有两种常见政策：Fail-fast 立即传播并中止后续通知；或隔离失败、记录错误并继续。模式本身不规定选择。上面的教学代码采用前者；不能据此宣称后续观察者一定会收到。

此外，异常发生时 Subject 的 progress 已更新，不会自动回滚。扩展：观察者内再次调用 set_progress 可能产生重入或无限通知，需要锁、排队或明确限制。课件没有提供通用重入框架，本书不把它当作现成保证。

9.5.11 真实事件与响应式系统

Watchdog 以文件事件处理器接收文件系统变化；Django Signal 保存接收者并依次调用；Tkinter 的 StringVar 可通过 trace_add 在值变化时触发回调。它们的命名可能与 Subject/Observer 不一致，但共同点是注册反应与事件驱动通知。

React useState 示例体现状态更新后调度更新、必要时重新渲染的响应式思想。它与 Observer-like 传播存在关联，但不应把完整 React 更新模型等同于课堂的同步循环。课程借此扩展视野，不要求用十行 Observer 实现框架行为。

9.5.12 Strategy 的 Before：mode 与无关参数

FlyingCar 的构造器同时接收 mode、road_condition、weather、airspace_clear。驾驶时天气和空域参数可能无用；飞行时道路参数无用。start_journey 包含分支并混合两种算法，新增航行又要扩大中心逻辑。

把 FlyingCar 直接细分为 DrivingCar/FlyingOnlyCar 可以固定子类行为，但同一对象运行时切换不方便。Strategy 通过保存一个 TravelStrategy，把选择和执行机制分开。

9.5.13 Strategy 的重构与运行时替换

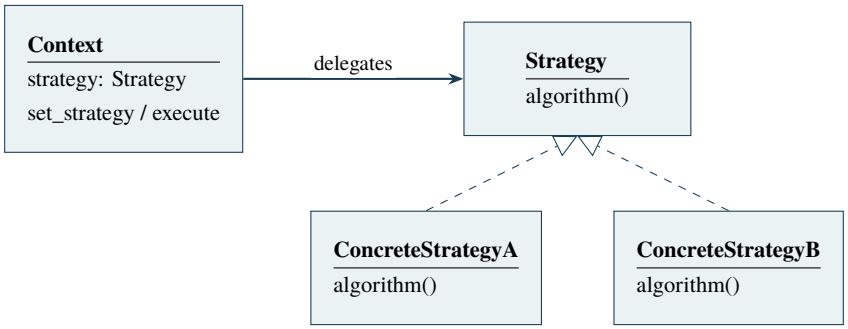


图 **18:** Strategy：上下文委托算法，客户负责提供或切换所需策略。

PYTHON

```
class DriveStrategy:
    def __init__(self, road_condition):
        self._road_condition = road_condition
```

```
    def travel(self, start, end):
        style = "carefully" if self._road_condition == "bad" else "smoothly"
        return f"drive {style}: {start}->{end}"

class FlyStrategy:
    def __init__(self, weather, airspace_clear):
        self._weather = weather
        self._airspace_clear = airspace_clear

    def travel(self, start, end):
        if not self._airspace_clear:
            return "cannot fly: airspace closed"
        if self._weather == "stormy":
            return f"fly with turbulence: {start}->{end}"
        return f"fly smoothly: {start}->{end}"

class FlyingCar:
    def __init__(self, strategy):
        self._strategy = strategy

    def set_strategy(self, strategy):
        self._strategy = strategy

    def start_journey(self, start, end):
        return self._strategy.travel(start, end)

car = FlyingCar(DriveStrategy("bad"))
assert car.start_journey("Sydney", "Melbourne").startswith("drive carefully")
car.set_strategy(FlyStrategy("clear", True))
assert car.start_journey("Melbourne", "Brisbane").startswith("fly smoothly")
assert FlyStrategy("clear", False).travel("A", "B") == (
    "cannot fly: airspace closed"
)
```

Driving 与 Flying 的专有参数移到各策略对象；FlyingCar 只依赖 travel 协议。新增 SailStrategy 无需修改 start_journey，但客户或配置仍要选择何时使用它。模式没有自动决定最优策略。

本例把天气字符串视为已接受的教学输入；若生产规格只允许若干天气值，仍需验证非法值。返回“不能飞”属于此例的结果契约，也可以在别的设计中用异常或专门结果类型表达，不能混用而不更新客户。

9.5.14 三种实现多行为的方法

条件判断集中而直接，适合固定且少量情况；继承让子类提供行为，适合稳定的子类型差异；Strategy 让上下文持有可替换协作者，适合独立算法及运行时选择。

Strategy 常通过组合体现“算法可换”，具体策略本身也可以通过继承实现共同接口。L10 说继承和组合作为实现技术都可能出现；这不等于只要有继承就实现了 Strategy。必须识别算法如何被封装和替换。

9.5.15 函数与 lambda 作为轻量策略

当算法无状态、只需一个操作时，Python 一等函数可以避免很多空壳类。课件用平均数计算切换 mean 与 median；调用契约必须规定空输入行为。

PYTHON

```python
def mean(values):
    if not values:
        raise ValueError("empty values")
    return sum(values) / len(values)

def median(values):
    if not values:
        raise ValueError("empty values")
    ordered = sorted(values)
    middle = len(ordered) // 2
    if len(ordered) % 2:
        return ordered[middle]
    return (ordered[middle - 1] + ordered[middle]) / 2

class AverageCalculator:
    def __init__(self, strategy):
        self._strategy = strategy

    def set_strategy(self, strategy):
        self._strategy = strategy

    def compute(self, values):
        return self._strategy(values)

calculator = AverageCalculator(mean)
assert calculator.compute([1, 2, 3, 100]) == 26.5
calculator.set_strategy(median)
assert calculator.compute([1, 2, 3, 100]) == 2.5
```

median 先排序，奇数取中间，偶数取两者均值；它改变的是统计意义，不是对所有数据都与 mean 等价。因此选择策略属于业务决定。mean O(N)，此排序版 median O(N log N)，也是可比较的工程代价。

9.6 知识关联（Connections）

责任链的 Handler.handle 常具有 Template Method 式骨架；Observer 的注册回调体现后续控制反转思想；Strategy 使用组合并把变化封装，与 Week 2 的 FlyingCar 多重继承问题形成因果闭环。

测试时，责任链需要检查前序拒绝、后继接收及无人处理；Observer 需要检查订阅实例、退订、条件性反应和失败政策；Strategy 需要检查上下文委托、策略切换及各算法边界。类图只说明可能的协作，不能代替这些行为证据。

9.7 考试应用（Exam Application）

掌握要求：解释三种模式的问题、职责与调用流，明确顺序和失败政策。**常见错误**：把 Observer 的 Pull 当作轮询、把“收到通知”当作“一定输出”、以为策略自动选择自己。以下为五道自编练习。

情境题 1：链顺序。两个处理器都匹配投诉工单，一个是通用 Support，一个是 BillingComplaint。首个匹配处理后停止。把通用者放前面有什么后果？

解析。专门者可能永远收不到该类请求。应按业务优先级装配，或让通用者排除专门情况。测试需使用重叠匹配输入，而不只为每个处理器找一个独占输入。

情境题 2：通知与输出。 ProgressBar、Logger、CompletionNotifier 都订阅下载，进度更新为 30。Notifier 没有输出，是否说明没有通知?

解析。不一定。它可能已收到 update，但只有 100 才产生完成效果。应区分被调用与其内部条件；测试可记录 update 调用，另测条件行为。

情境题 3：回收疑问。局部变量 observer 置 None 后对象仍存活。Subject 是长寿命单例且保存观察者列表。最可能的关系是什么?

解析。 Subject 列表仍持有强引用，对象仍可达。应在不再需要时退订，或根据契约设计弱引用机制。只删除一个局部变量不移除订阅关系。

情境题 4：失败政策。第一个观察者抛异常，第二个未收到通知，但 progress 已变。能否断言 Observer 模式实现错误?

解析。不能仅凭这个结果。Fail-fast 允许后续不执行且状态已更新；需要检查需求是否要求隔离失败、继续通知或回滚。模式不替团队决定该政策。

情境题 5：Strategy 或 Command。一个对象保存“支付这张账单的金额和接收者”，稍后可排队执行；另一个对象只决定计算手续费的算法。应怎样区分?

解析。前者意图是请求对象化，属于 Command；后者提供可互换计算，属于 Strategy。它们都可能只有 execute 类方法，但方法名不决定模式。Command 的参数通常描述一次请求，Strategy 的状态通常配置算法。

第 10 章　Week 10：Template Method、控制反转与模式全景

来源：L10 第 1-79 页。本章先完整重构报表案例，再按课件回顾顺序比较全部 13 种模式。它把“记住类图”提升为“根据问题选择变化机制”。

10.1　核心思想（Core Idea）

如果多个算法共享稳定步骤顺序，只有某些步骤不同，就应把骨架集中在一个位置，把可变步骤留给子类。模板方法（Template Method）正是用继承与动态分派表达这种控制关系。

本周最终能力是解释模式之间“像在哪里、差在哪里”：对象创建、包装、请求传播、算法替换和遍历可以共享技术，却服务不同目的。好的答案先给出变化需求，再说明结构和代价，而不是从类名猜模式。

10.2　历史渊源与问题背景（Origin & Motivation）

销售报表读取 CSV、汇总销售、格式化并导出 PDF；库存报表读取 XLSX、组织库存行、格式化并导出 CSV。直接复制完整流程会重复“读取 → 处理 → 格式化 → 导出”这一知识。只加一个声明 generate_report 的抽象接口，虽然统一调用名称，却没有消除流程重复。

Template Method 将共同骨架移到基类，子类只填写差异。在框架中，这种思想使框架控制生命周期、用户提供扩展点。课件用 Hollywood Principle 的“我们会调用你”类比控制反转；历史比喻服务于理解调用方向，不要求记忆影视行业细节。

Assignment 2 反馈显示 Factory Method 容易与简单工厂混淆，并发现 cooldown 校验等知识重复。L10 因而采用与首次授课不同的回顾顺序，帮助按机制重新组织模式，而不是机械重复分类表。

10.3　重要定义（Definitions）

Template Method ／模板方法模式：在一个操作中定义算法骨架，把某些步骤延迟到子类实现，使子类可以改变特定步骤而不必改写算法的整体结构。

Template method ／模板方法：负责调用各步骤并确定顺序的方法；**Primitive operation ／基本步骤操作**：供模板调用、可由子类提供实现的操作。

Hook ／钩子：基类提供默认或空实现的可选扩展步骤，子类可选择覆盖。抽象步骤要求具体子类实现，Hook 不强制覆盖；二者的契约不同。

Inversion of Control（IoC）／控制反转：整体控制流程由框架或高层组件决定，应用或低层扩展点由其在适当时机调用。这里反转的是相对于“应用主动安排全部调用”的控制关系。

Hollywood Principle ／好莱坞原则：Don’t call us, we’ll call you，即由控制骨架调用扩展代码。本课程用它解释 Template Method 和框架回调。

Framework ／框架：提供应用结构或生命周期，通常调用用户提供的扩展代码；**Library ／库**：提供由使用者主动调用的能力。大型软件可以在不同层次同时具有两种角色。

Variation point ／变化点：设计中特意允许扩展或替换的决定，例如产品类型、算法、步骤、访问政策。它不是“所有可能变化”的总称，而是具体选择保护的边界。

10.4 实际意义（Practical Significance）

Template Method 适合导入处理、报表、测试生命周期和具有稳定顺序的业务算法。它集中公共规则，使新增实现不必复制同一骨架；修改共有步骤顺序也集中在一处。

代价是子类依赖基类生命周期和内部扩展契约。若步骤顺序经常变化、需要运行时组合多个独立算法或客户需要任意流程，继承骨架可能过于僵硬，Strategy 或显式组合会更适合。

10.5 完整课件内容（Lecture Reconstruction）

10.5.1 从两份报表提取真正的共同知识

销售报表：读取 sales_and_expenses.csv，求 sales 列之和，格式化总销售，导出 PDF。库存报表：读取 inventory.xlsx，把项目与数量组织为多行，格式化，再导出 CSV。

两者的共同点不是每行文本都相同，而是步骤之间的依赖顺序：必须先取得数据才能处理，必须先处理才能格式化，最后才导出。这种结构知识本身适合单一表示。

若只声明 `ReportGenerator.generate_report` 为抽象方法，而在每个子类复制四步，接口多态成立，DRY 问题却仍存在。Template Method 的增量是复用流程实现，不只统一方法名称。

10.5.2 模板、步骤与共同默认实现

基类的 generate_report 调用 fetch_data、process_data、format_data、export_report。读取、处理和导出可以是抽象步骤，统一的格式化可在基类提供默认实现。子类无需覆盖全部方法，更不应为了实现某一步而复制模板。

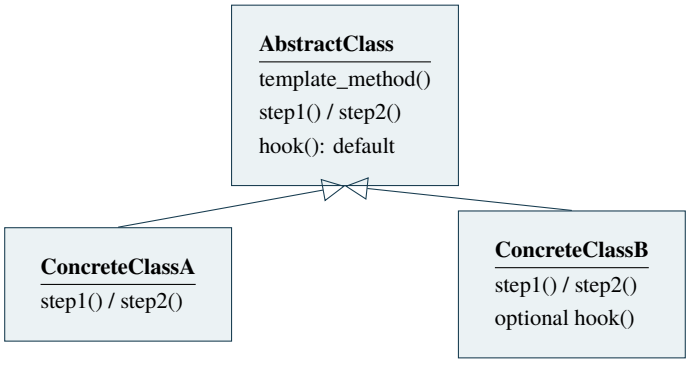


图 **19:** Template Method：公共方法保留步骤顺序，子类实现差异，Hook 可选覆盖。

PYTHON

```python
from abc import ABC, abstractmethod

class ReportGenerator(ABC):
    def generate_report(self):
        data = self.fetch_data()
        processed = self.process_data(data)
        formatted = self.format_data(processed)
        formatted = self.before_export_hook(formatted)
        self.export_report(formatted)
        self.after_export_hook()
        return formatted

    @abstractmethod
    def fetch_data(self):
        pass

    @abstractmethod
    def process_data(self, data):
        pass

    def format_data(self, processed):
        return f"Formatted Report:\n{processed}"

    def before_export_hook(self, formatted):
        return formatted

    @abstractmethod
    def export_report(self, formatted):
        pass

    def after_export_hook(self):
        pass

class SalesReportGenerator(ReportGenerator):
    def __init__(self, rows):
        self._rows = tuple(rows)
        self.exported = None

    def fetch_data(self):
        return self._rows

    def process_data(self, data):
        total = sum(row["sales"] for row in data)
        return f"Total Sales: USD {total}"

    def export_report(self, formatted):
        self.exported = ("PDF", formatted)

class InventoryReportGenerator(ReportGenerator):
    def __init__(self, items):
        self._items = dict(items)
        self.exported = None

    def fetch_data(self):
        return self._items

    def process_data(self, data):
        return "\n".join(f"{item}: {qty}" for item, qty in data.items())

    def before_export_hook(self, formatted):
        return formatted + "\nChecked"
```

```
62
63      def export_report(self, formatted):
64          self.exported = ("CSV", formatted)
65
66  sales = SalesReportGenerator([{"sales": 10}, {"sales": 20}])
67  assert sales.generate_report().endswith("Total Sales: USD 30")
68  assert sales.exported[0] == "PDF"
69  inventory = InventoryReportGenerator({"coffee": 3})
70  assert inventory.generate_report().endswith("coffee: 3\nChecked")
71  assert inventory.exported[0] == "CSV"
```

教学重写用注入内存数据和记录导出类型代替外部 CSV/XLSX/PDF 库，完整保留控制骨架。示例没有真正生成报表文件，避免把占位 generate_pdf/read_xlsx 当成已实现函数；读者可单独为边界加入真实 I/O。

10.5.3 执行追踪：同一方法体调用不同实现

Client 创建 SalesReportGenerator 并调用 generate_report。该方法体来自基类，但 self 是具体销售报表对象；fetch_data 和 process_data 因动态分派进入销售实现，format_data 使用基类默认，export_report 再进入销售实现。

对于库存对象，调用顺序相同，读取和处理的内容不同，before_export_hook 也被覆盖。统一顺序在一个位置维护，子类细节可替换。顺序图应表现“基类骨架的 self 调用进入具体步骤”，而不是画成客户依次亲自调用每一步。

10.5.4 Hook 的可选性与返回值契约

Hook 不是一个可以随意插入而无需规定语义的空方法。若 before_export_hook 可以变换 formatted，模板必须接住返回值；默认实现应返回原值，覆盖实现也必须返回契约要求的结果。

若只记录日志而不变换，可采用无返回值的通知式 Hook。课件的说明存在“用于修改数据”与部分示意代码未返回的简化，本例明确采用变换式，避免把 None 传给导出。after_export_hook 则是通知式，默认不执行操作。

钩子增加灵活性也增加组合复杂性。过多 Hook 让子类需要理解大量隐含顺序，可能比清楚的组合对象更难维护。应只暴露真实需求所需的扩展点。

10.5.5 控制反转与框架／库

在 Template Method 中，高层 ReportGenerator 决定顺序，低层报表子类在指定步骤被调用。这体现 IoC。unittest 框架会调用用户编写的 setUp 与 test 方法；用户调用 numpy 函数则体现库式使用。

IoC、DI 与 DIP 不等价：IoC 讨论谁控制流程，DI 讨论谁提供协作者，DIP 讨论策略与实现的依赖方向。它们常共同出现，但仅向构造器传一个参数不自动产生框架生命周期，仅被框架调用也不保证依赖方向合理。

Python 不能仅靠本例语法完全阻止子类覆盖 generate_report。文档、代码审查和测试应保护骨架契约；某些语言可用 final 限制覆盖。不要声称“基类写好了，子类就绝不可能改变顺序”。

10.5.6 Template Method 与 Factory Method

课件将 Factory Method 解释为关注创建步骤的专门化：Creator 的稳定 operation 调用 factory_method，ConcreteCreator 决定 Product。一个更长的 Template Method 中也可包含创建对象的工厂步骤。

二者共享继承与动态分派，但关注点不同。Template Method 保留算法骨架，Factory Method 把创建产品的类型决定延迟给子类。不能反推每个工厂方法都必须出现四步算法，也不能把每个含抽象方法的类都叫完整模板模式。

支付重构体现这一点：中央 PaymentFactory 的条件选择变为 Checkout.make_processor 的重写；PaymentProcessor 及各产品层次负责执行，Checkout 层次负责选择创建。它是 Week 3 中最需要反复追踪的双层结构。

10.5.7 创建控制：Factory Method 与 Singleton

Factory Method 允许具体创建者控制产品选择；Singleton 则限制实例数量并提供共享访问。前者的变化点是创建产品类型，后者的约束是对象身份与生命周期。二者可以组合，但没有天然互相替代关系。

L10 回顾的 Singleton 以类变量存自身类型对象，初始 None 后缓存。Proxy 的例子则在每个代理实例中存 RealSubject；这是“类级共享”与“对象级引用”的结构区别。课堂两例都懒初始化，但不是所有模式实现都必须如此。

10.5.8 包装家族：Proxy、Decorator 与责任链

Proxy 和 RealSubject 实现共同 Subject，代理决定访问；Decorator 和 ConcreteComponent 实现共同 Component，包装可反复层叠，调用与返回构成前后处理；责任链的 Handler 同样保存同接口后继，却可能匹配后停止。

Decorator 通常希望增加行为同时保持基础功能正确，而责任链希望为请求找到合适处理者；Proxy 希望控制访问。OCP 的收益都要绑定具体变化：可加入新政策、装饰行为或处理者，而客户仍使用稳定入口。

不要把“Decorator 每层都必须调用下一层”写成无例外语法定律；它是本课程案例的常规语义。若一个包装器主动拒绝请求，就需分析它是否兼具访问政策以及是否符合原契约。

10.5.9 传播与操作：Observer、Command 和 Strategy

Observer 将一个主体事件传播给多个已登记依赖者，主体不预先认识所有具体类型；它建立可变化的一对多关系。把主体称为 publisher、观察者称为 subscriber 有助于理解，但不能据此宣称它已经具备分布式消息系统的可靠投递语义。

Command 封装一次请求及执行 / 撤销所需信息，使触发、排队和历史管理独立；Strategy 封装可替换算法，供 Context 委托。两者都可用对象或函数表达动作，意图和生命周期不同。

例如“给这份文档插入 A”是一次 Command；“如何选择排序算法”是 Strategy；“文档改变后更新多个视图”是 Observer。一个编辑器可同时使用三者，它们保护不同方向的变化。

10.5.10 接口边界：Strategy、Adapter 与 Facade

Strategy 的客户原本就接受某算法契约，目标是替换算法；Adapter 的客户无法直接使用既有接口，目标是翻译；Facade 的客户面对过多步骤，目标是提供高层用例入口。

它们都可能保存对象并转发，但返回结果与数据转换的职责不同。向实际系统选择模式时，应先问：算法要换？接口不兼容？还是调用过于复杂？同一个“service wrapper”名称无法回答这些问题。

10.5.11 统一结构与遍历：Facade、Composite 与 Iterator

Facade 让客户通过一个高层入口协调多个子系统；Composite 让一组节点本身也满足单个节点契约；Iterator 让客户顺序取得元素而不依赖存储。

Composite 常在内部递归处理整个树，Iterator 把逐步遍历控制权给客户。Facade 所协调的服务通常不是递归部分整体结构，也不需要都实现同一 Component。把“都是一个方法调用很多对象”当成相同模式，会遗漏这些关键差异。

10.5.12 用变化点重新组织全部模式

模式	主要变化点或约束	关键责任位置
Factory Method	创建哪种产品	ConcreteCreator
Singleton	实例唯一与共享访问	类级实例管理
Composite	部分整体的递归组织	Composite 与共同 Component
Decorator	哪些附加职责及组合	装饰对象链
Proxy	何时及是否访问真实对象	Proxy
Adapter	两侧接口与表示差异	Adapter
Facade	高层用例的协作步骤	Facade
Command	请求何时执行与如何保留	Command 与 Invoker
Iterator	如何取得下一个元素	ConcreteIterator
Chain of Responsibility	谁有机会处理请求	Handler 链与装配
Observer	哪些对象响应状态变化	Subject 订阅关系
Strategy	当前使用哪种算法	Strategy 与 Context
Template Method	固定骨架中的步骤细节	基类骨架与子类步骤

这张表辅助选择，不构成“关键词 → 唯一答案”的机械规则。现实类可能同时参与多个模式，最有价值的说明是给出角色对应和调用依据。

10.5.13 Assignment 2 的 DRY 回顾与代码阅读能力

课件反馈把重复 cooldown 校验、CLI / moderation 逻辑和 importer 流程作为讨论材料。重复知识可能写得不同，寻找它要理解规则而非只搜索相同行。

工厂结构难辨认时，可从稳定业务入口向内追踪：谁调用 create_importer？它的返回类型是什么？具体产品如何进入共同流程？Composite 则从首个 Component 操作向子节点看，Command 则分别找触发方、对象中保存的请求信息和实际接收者。

课程强调阅读陌生代码、依据设计目标提出评价。这种能力可迁移到不同团队标准；即使某组织对分层或编码风格有不同偏好，也需要明确事实、契约、变化和后果。

10.5.14 期末说明与范围边界

L9/L10 说明考试结构与 Assignment 4 相呼应，SOFT2201 有 20 道 MCQ、两节设计原则 / 模式简答及一节测试；L8 还说明测试可能针对实现设计模式的程序。由此可以组织综合练习，但不能推出某个模式必考或某章必占多少分。

L10 末尾预告下一周的接口、类型、泛型、调试与进一步审查。本地无对应后续讲义，因此本书不把这些预告当作已讲授内容。复习者准备完整 Final 时，应继续核对后续课程正式资料；这是一项资料范围说明，不是本书对未知考点的预测。

10.6 知识关联（Connections）

Template Method 把 DRY 与多态连接起来，IoC 把这一机制推广到框架生命周期；Factory Method 是创建扩展点的近邻，Strategy 则提供组合式的算法变化。Week 10 的模式比较把整个课程转成“结构怎样保护变化”的分析方法。

测试应同时保护公共骨架和具体步骤：用记录步骤的子类断言调用顺序，用不同具体实现验证结果，用 Hook 的默认及覆盖情形验证可选性，再检查错误是否使后续步骤不当执行。

10.7 考试应用（Exam Application）

掌握要求：分清模板与步骤、抽象步骤与 Hook、IoC/DI/DIP，以及相似模式的意图。以下均为自编练习。

情境题 1：接口统一却重复。两个报表类都实现 generate_report，但各复制相同的校验、格式化和导出顺序。是否已经解决 Template Method 关注的问题？

解析。仅有接口多态，还未集中骨架知识。可提取共同模板，把具体读取、处理或输出步骤留给子类；若流程本就不同，则应避免强行统一。

情境题 2：Hook 丢失结果。 before_export_hook 返回加水印的新字符串，但模板调用后仍把原字符串导出。缺陷在哪里？

解析。若契约允许变换，模板必须使用 Hook 返回值；默认 Hook 也应返回原值。应测试默认情况与覆盖情况，不能只检查 Hook 被调用。

情境题 3：运行时选择。同一会话中需要随时切换报表排序算法，但读取、输出流程稳定。更适合怎样组合？

解析。可用 Template Method 维护流程，同时把排序作为注入 Strategy；二者可以协作。只用子类切换整个生成器也能做，但可能把独立变化绑在一起并造成子类组合增加。

情境题 4：框架调用。测试框架自动调用你写的 setUp。它体现 IoC、DI 还是 DIP？

解析。直接体现 IoC，因为生命周期由框架安排、扩展代码被调用。是否使用 DI 或符合 DIP，需要另看协作者提供方式及依赖结构，不能从这一事实直接推断。

第 11 章　核心概念中英文对照

本表用于定位与区分概念，不能代替各周的完整定义和案例。周次表示主要讲解位置；同一概念可能贯穿多周。

11.1　语言、对象与设计原则

English	中文	简短定义或辨析	周次
Software engineering	软件工程	系统化开发、运行和维护软件	1
Software design	软件设计	决定系统职责、结构、接口和协作	1
Refactoring	重构	保持约定外部行为，改善内部结构	1-4
Maintainability	可维护性	理解、修改、修复软件的便利程度	1、3
Calibrated trust	校准信任	根据理解与证据接受 AI 输出	1
Dynamic typing	动态类型	名字运行时绑定，类型属于对象	1
Type hint	类型注解	为阅读与静态分析提供类型信息	1
Mutability	可变性	对象状态可以原地修改的性质	1-3
Object identity	对象身份	是否为同一个对象，使用 is	1
Value equality	值相等	按值协议比较，通常使用 ==	1
Truthiness	真值性	对象在条件中的真假解释	1
Alias	别名	多个名字引用同一对象	1
Late binding	晚绑定	闭包自由变量通常在调用时查找	1
Mutable default argument	可变默认参数	定义时创建并可能跨调用共享的默认对象	1、2
Short-circuit evaluation	短路求值	前部足以决定结果时不求值后部	1、7
First-class function	一等函数	可赋值、传参和返回的函数	1、9
Duck typing	鸭子类型	依据所支持的行为使用对象	1、2
Abstraction	抽象	保留使用所需性质并隐藏无关细节	2
Encapsulation	封装	通过边界和受控操作维护内部约束	2
Inheritance	继承	由已有类型派生并复用或改变行为	2
Polymorphism	多态	通过共同契约调用不同实现	2
Composition	对象组合	持有协作者实现功能；不都等于 UML 黑菱形	2
Abstract Base Class	抽象基类	可声明抽象操作、约束具体子类的类	2
Name mangling	名字改写	Python 对双下划线名字的类相关改写	2
Dunder method	双下划线方法	实现语言对象协议的方法	2
Property	属性描述接口	以属性语法调用受控读取或设置逻辑	2
Contract	契约	输入、结果、异常和状态效果的约定	2、6
Coupling	耦合	模块间依赖关系的程度和形式	2

接下页

English	中文	简短定义或辨析	周次
Cohesion	内聚	一个模块内部职责的关联程度	2
Single Responsibility	单一职责原则	一个类应只有一个变化原因	2
Open-Closed	开闭原则	对预期扩展开放，减少修改稳定代码	2、3
Liskov Substitution	里氏替换原则	子类型替换基类型而不破坏正确性	2
Interface Segregation	接口隔离原则	客户不应被迫依赖不需要的操作	2、4
Dependency Inversion	依赖倒置原则	高层策略与实现细节共同依赖抽象	2
Dependency injection	依赖注入	从外部提供协作者的技术	2、10
DRY	避免知识重复	每项知识有单一权威表示	2、10
Law of Demeter	迪米特法则	减少对间接协作者内部结构的了解	2、6
Code review	代码审查	检查变更质量、缺陷、标准并传播知识	2
Code smell	代码异味	潜在设计问题的提示，未必是 bug	2
God class	上帝类	混合过多不相关职责的类	2、6
Arrowhead anti-pattern	箭头形嵌套反模式	多层嵌套使主路径难以追踪	2
Quality attribute	质量属性	衡量产品在特定质量维度的性质	3
Reliability	可靠性	规定条件下持续正确服务的能力	3
Availability	可用性	需要服务时可提供服务的程度	3
Performance	性能	延迟、吞吐与资源使用表现	3
Scalability	规模扩展能力	随负载与资源增长的表现	3
Robustness	健壮性	对异常输入和条件的适当处理能力	3
Safety	避免伤害的安全性	避免不可接受系统伤害	3
Security	信息安全	抵御未授权和恶意行为	3
Extensibility	可扩展性	增加新行为的便利程度	3
Brooks' Law	布鲁克斯定律	延期项目增加人手往往进一步延误	3
Least Expressiveness	最少表达力原则	用足够且自然的最弱计算表达	3
Least Power	最少能力原则	选择用途所需而不过度强大的机制	3
Least Astonishment	最少惊讶原则	行为符合语法与约定带来的合理预期	3
Least Effort	最少努力原则	用户倾向采用最容易完成目标的路径	3
Interning	驻留 / 池化	按不可变值复用规范对象	3

11.2 UML 与设计模式

English	中文	简短定义或辨析	周次
UML	统一建模语言	用标准符号表达系统模型	2
Class diagram	类图	类型、成员及静态关系	2
Sequence diagram	顺序图	一个场景中的时序交互	2
Association	关联	一般结构连接	2

接下页

English	中文	简短定义或辨析	周次
Aggregation	聚合	部分可独立存在的整体部分关系	2
Composition (UML)	组合关系	更强的整体拥有关系，整体端实心菱形	2
Generalization	泛化 / 继承关系	实线空心三角指向更一般类型	2
Realization	实现关系	虚线空心三角指向被实现契约	2
Dependency	依赖关系	客户使用供应者、可能受其变化影响	2
Multiplicity	多重性	某时刻一个对象可关联的对端数量	2
Association class	关联类	为关系本身保存数据和行为	2
Lifeline	生命线	顺序图中参与者随时间的轨迹	2
Activation bar	激活条	执行或控制焦点区间	2
Guard	守卫条件	控制片段是否适用的条件	2
opt / alt / loop	可选 / 选择 / 循环框	顺序图中的条件与重复片段	2
Design pattern	设计模式	反复出现的问题的可复用解决结构	3
Creational pattern	创建型模式	控制对象创建与选择	3
Structural pattern	结构型模式	组织对象与类型结构	3
Behavioural pattern	行为型模式	分配算法、请求与交互职责	3
Simple factory	简单工厂	集中选择并创建具体对象	3
Static factory method	静态工厂方法	有描述性名称的静态创建入口	3
Factory Method	工厂方法模式	由创建者子类决定具体产品	3
Product / Creator	产品 / 创建者	工厂中的使用契约与创建扩展层次	3
Composite	组合模式	统一处理单个节点与递归组合	4
Component / Leaf	组件契约 / 叶子	公共操作与无子节点元素	4
Command	命令模式	把一次请求封装为可管理对象	4
Invoker / Receiver	调用者 / 接收者	触发命令者与完成实际工作者	4
Undo	撤销	按契约恢复先前状态	4
Decorator	装饰器模式	按对象层叠附加职责	5
Proxy	代理模式	代表真实对象控制访问	5
Delegation	委托	将请求或工作交给协作者	4-10
Lazy initialization	延迟初始化	真正需要时再创建资源	5
N+1 query problem	N+1 查询问题	主查询后逐对象读取关系引起额外查询	5
Singleton	单例模式	限制实例数量并提供共享访问	5
Idempotence	幂等性	重复同一操作不继续累加约定效果	5、6
Adapter	适配器模式	把已有接口翻译为客户所需契约	6
Target / Adaptee	目标契约 / 被适配者	客户期望接口与现有不兼容组件	6
Facade	外观模式	提供易用的高层子系统入口	6
Orchestration	编排	安排协作者调用顺序与数据流	6
Iterator	迭代器模式	顺序访问元素而不暴露内部表示	8

接下页

English	中文	简短定义或辨析	周次
Iterable	可迭代对象	可提供迭代器的对象	8
StopIteration	迭代耗尽信号	Python 迭代器无下一项时的异常	8
Generator	生成器	用 yield 逐步产生值并保存执行状态	8
EAFP / LBYL	尝试后处理 / 预先检查	两类执行与异常处理习惯	8
Chain of Responsibility	责任链模式	多处理者依序获得处理请求机会	9
Handler / Successor	处理者 / 后继	当前处理角色及转交目标	9
Observer	观察者模式	状态变化通知已登记的一对多依赖者	9
Subject	主体 / 被观察者	管理状态与订阅关系	9
Push / Pull	推送 / 拉取模型	携带数据通知 / 通知后读取主体	9
Polling	轮询	消费者周期或循环查询状态	9
Weak reference	弱引用	不单独维持所指对象存活的引用	9
Strategy	策略模式	封装并替换一族算法	9
Context	上下文	保存并使用当前策略的对象	9
Template Method	模板方法模式	固定算法骨架，由子类实现可变步骤	10
Hook	钩子	有默认实现的可选扩展步骤	10
Inversion of Control	控制反转	框架或骨架决定何时调用扩展代码	10
Hollywood Principle	好莱坞原则	“我们会调用你”的控制方向	10
Framework / Library	框架 / 库	主要调用用户扩展 / 主要被用户调用	10

11.3 测试与验证

English	中文	简短定义或辨析	周次
Testing / Debugging	测试 / 调试	评价行为与揭露问题 / 定位原因并修复	7
Dynamic / Static analysis	动态 / 静态分析	通过执行观察 / 不执行而检查	7
Fault / Error / Failure	缺陷 / 错误状态 / 失效	实现问题 / 内部偏差 / 可观察违约	7
Test case / Test suite	用例 / 套件	一次具体检查 / 有组织的检查集合	7
Test oracle	测试判定依据	判断结果正确与否的规则	7
RIPR	缺陷显露条件链	可达、感染、传播、揭露	7
Unit testing	单元测试	对较小组件的隔离检查	7
Integration testing	集成测试	检查多个组件的交互和接口假设	7
System testing	系统测试	检查完整系统的行为	7
Acceptance testing	验收测试	检查是否满足业务接受条件	7、8
Regression testing	回归测试	修改后确认应保留行为未退化	7
Controllability	可控制性	安排所需输入与条件的便利程度	7
Observability	可观察性	观察结果及状态效果的便利程度	7

接下页

English	中文	简短定义或辨析	周次
White-box testing	白盒测试	利用实现结构设计测试	7
Test adequacy	测试充分性	相对于给定准则的满足程度	7
Line coverage	行覆盖	按所定口径执行过的源码行比例	7
Statement coverage	语句覆盖	所考虑可执行语句的执行情况	7
Branch coverage	分支覆盖	所考虑决策结果流向的执行情况	7
Unreachable code	不可达代码	在给定条件下无法执行到的代码	7
Subsumption	覆盖准则包含关系	满足 A 必满足 B 的逻辑关系	7
Error guessing	错误猜测	依据经验列举易错输入和场景	7
Black-box testing	黑盒测试	根据功能规格设计测试	8
Input domain	输入域	可能输入及其条件的空间	7、8
Normal / Robust	正常 / 健壮假设	只考虑合法 / 兼顾非法输入	8
Weak / Strong	弱 / 强假设	独立选参数 / 考虑交互组合	8
Random Choice	任意选择	无系统分区依据的选值方法	8
Equivalence partitioning	等价类划分	按预期行为分区并选代表	8
Boundary value analysis	边界值分析	检查行为转变点及邻近值	8
Monotonic clock	单调时钟	适合以差值衡量经过时长的时钟	8
Daylight saving time	夏令时	地区民用时间规则的一种调整	8

第 12 章　考前综合复习与跨章节训练

本章为自编训练，依据已核对的 Trial Exam 四节结构组织，不是原卷、官方答案或考题预测。应先独立回答，再阅读解析。MCQ 练习每题仅选一个最佳答案；简答允许多种合理设计，但必须解释契约、职责与代价。

12.1　从十周内容建立答题方法

设计原则题应按照“具体位置 → 实际行为或依赖 → 原则定义 → 变化为何困难 → 聚焦改进及取舍”展开。不要只有“违反 OCP”，也不要把所有问题都归为“代码太长”。

模式题应先圈出客户及其问题，再识别共同接口、具体实现、对象引用和控制流。说明哪一段需要新增、哪一段可以保持稳定。对相似结构至少指出一个关键的意图差别。

测试题先给规格与输入假设，再列可执行路径、需要覆盖的结果和断言。对模式实现，应同时检查协议、具体算法、空结构、生命周期和错误传播。不能只列输入而不列预期，也不能只报覆盖百分比。

12.2　Section A：20 道情境 MCQ

A1. 函数 make_report(errors=[]) 把 errors 存入返回字典。两个省略实参的调用后，修改第一个报告的 errors 影响第二个。最直接的解释是？

A. 字典总会共享所有键值。B. 默认对象在函数定义时创建并被两次调用复用。C. Python 参数按名称复制。D. 两个报告必然是同一字典。

A2. 一个属性 user.profile 每次读取都同步下载 50 MB 数据。客户把它放入多次显示循环。首先需要解释哪项设计风险？

A. 属性必须永远返回字符串。B. 只有 private 字段才允许下载。C. 属性外观可能隐藏昂贵工作，破坏合理预期。D. 使用了属性便必然违反 DRY。

A3. Rectangle 的契约允许独立改变宽与高；Square 覆盖 setter 时同时改变两者。客户设置宽 5、高 4 后依赖面积为 20。最有依据的结论是？

A. 数学上正方形是矩形，所以替换永远安全。B. 只要方法名相同就安全。C. 问题仅在运行速度。D. 该可变契约下的 Square 不能保持客户所依赖的行为。

A4.　服务接受一个具体数据库对象作为构造参数，但随后调用该数据库独有的 internal_pool.reconnect。哪一评价最准确？

A. 已采用注入，但尚不能据此认定依赖于稳定抽象。B. 只要用了参数就完整符合 DIP。C. 依赖注入不允许传对象。D. 这个事实已经证明符合 LSP。

A5. ImportService 的稳定 import_questions 调用 self.create_importer，具体 service 子类覆盖创建并返回统一 QuestionImporter。这是哪一种关键结构？

A. 只有限制实例数量。B. 创建者子类决定具体产品的 Factory Method。C. 客户直接调用所有产品构造器。D. 仅改变显示名称的 Adapter。

A6. 一个静态方法 User.from_dict 返回 User，没有创建者子类扩展层次。最恰当的分类是?

A. 必然是 GoF Factory Method。B. 必然是 Singleton。C. 静态工厂创建入口，尚不足以认定 GoF Factory Method。D. 因为返回对象，所以不是工厂。

A7. Composite 树已有 size、display 两项操作。增加 ZipNode 可局部实现这两项；现在要增加所有节点必须支持的 validate。哪项最准确?

A. Composite 保证所有新操作都无需修改既有类型。B. 新增类型与新增公共操作具有不同修改成本。C. validate 只能写在客户中。D. 所有叶子必须拥有孩子。

A8. 命令执行抛异常，Invoker 却已经把它加入 undo 历史。最直接的问题是?

A. Command 不允许异常。B. Receiver 不可保存状态。C. Invoker 必须自己实现每个业务动作。D. 历史可能宣称存在一项并未成功执行的可撤销操作。

A9. source=Compression(Encryption(file))，两层 write 都先变换再转发。真实写入文件前的顺序是?

A. 先压缩，再加密。B. 先加密，再压缩。C. 二者同时发生。D. 只由 file 决定。

A10. 单线程 ImageProxy 首次 display 创建 RealImage，以后复用。仅创建 10 个代理而从未 display，按此示例会创建多少 RealImage?

A. 10。B. 1。C. 0。D. 无法因为“单线程”而讨论。

A11. Singleton.__new__ 返回同一对象，但 __init__ 每次重设 url。第二次构造传入不同 URL 后，最可能发生什么?

A. 既有引用永远不受影响。B. 身份相同，但可观察配置被第二次初始化改变。C. Python 自动跳过第二次 __init__。D. 一定得到两个实例。

A12. LegacySensor 返回华氏度，客户需要摄氏度。一个包装类实现客户的 read_temperature 并执行 (F-32)×5/9。其主要意图是?

A. 只限制实例数量。B. 记录可撤销命令。C. 固定算法骨架。D. 通过 Adapter 翻译不兼容接口及单位。

A13. OrderFacade 依赖库存、汇率、支付三个组件，唯一职责是编排下单。只据依赖数量，能否认定 God Class?

A. 不能，应考察是否有多个不相关变化原因。B. 能，超过两个依赖就是 God Class。C. 能，Facade 与 God Class 同义。D. 不能，因为模式永远无需审查。

A14. 错误函数结果为 2，规格要求 6；测试只检查 result!=3 并通过。RIPR 中最直接不足的是?

A. Reachability。B. 一定没有 Infection。C. Reveal。D. Python 不能比较数字。

A15. if len(text)>=5 返回 long；elif len(text)>=3 返回 medium；否则 short。测试长度为 5 与 1，还缺哪条决策结果?

A. 第一判断为真。B. 第二判断为真。C. 第一判断为假。D. 第二判断为假。

A16. 在课程普通控制流模型中，下列关系哪项成立?

A. 100% 语句覆盖必然证明正确。B. 任意 90% 语句套件都比任意 80% 分支套件强。C. 所有程序都能达到 100%。D. 满足完整分支覆盖会满足相应语句覆盖，反向不必然成立。

A17. 票价区间写“18-65 普通”和“≥ 65 老年”，没有优先级。测试者最先应做什么？

A. 指出 65 的规格重叠并澄清预期。B. 根据当前代码先出现哪个分支就认定需求。C. 不测 65。D. 把两个票价相加。

A18. 集合 __iter__ 返回 self，self 还保存 current 索引。两次 iter(collection) 得到的遍历很可能怎样？

A. 自动复制整个集合且位置独立。B. 一定是线程安全。C. 共享同一遍历状态，可能相互推进。D. 完全不能用于 for。

A19. Subject 改变状态后调用 update(subject)，观察者再读 subject.progress。这应理解为？

A. 一定是持续轮询。B. 事件触发的 Pull 型 Observer 通知。C. 没有进行通知。D. 只允许一个观察者。

A20. Template Method 规定 before_export_hook 返回变换后的字符串，默认返回原值。模板应该怎样使用它？

A. 忽略返回值。B. 只由客户决定是否调用。C. 必须改为抽象方法。D. 接收其返回值并传给后续导出。

12.2.1 Section A 参考答案与解析

A1：**B**。共享的是默认列表；两个返回字典仍可以独立。改用 None 并在调用中创建新列表。

A2：**C**。同一语法暗示廉价访问却隐藏昂贵工作；需要清楚接口、延迟策略及测量。不是所有属性计算都不合理。

A3：**D**。替换正确性由可变接口的行为保证决定。可采用共同只读 Shape 等替代设计。

A4：**A**。DI 是提供方式，DIP 是依赖方向和抽象边界；专有内部调用仍造成具体耦合。

A5：**B**。必须同时看到稳定流程、可覆盖创建步骤和 Product 契约，不能只看 create 字样。

A6：**C**。描述性静态创建入口有价值，但没有体现创建者子类型扩展。

A7：**B**。Composite 保护“增加节点种类”的方向；增加公共操作可能触及全部实现。

A8：**D**。应成功后才入栈，并另行考虑操作失败是否可能留下局部副作用。

A9：**A**。从外向内执行正向变换；read 从底层取得数据后按逆向恢复。

A10：**C**。此契约下昂贵创建发生在首次 display，而不是 Proxy 构造。不要把此结论泛化到所有代理。

A11：**B**。实例复用与初始化次数不同；应明确并实现重复初始化政策。

A12：**D**。它改变接口与表示，使现有组件适用于客户契约，而非给本来兼容的对象添加可选职责。

A13：**A**。一个内聚编排职责可以合理协调多组件；是否上帝类取决于责任与变化原因。

A14：**C**。错误已在结果中出现，但弱断言没有揭露与预期 6 的差异。

A15：**B**。长度 3 或 4 可补第二判断真；分别追踪每个决策，不能只数最终标签。

A16：**D**。它是覆盖准则的逻辑关系，不是正确性证明或任意百分比的比较。

A17：**A**。先建立可信 oracle，不能让偶然的实现次序替代业务规格。

A18：**C**。返回自身常意味着共享一个位置。该对象仍可用于 for，但不自动支持独立重入遍历。

A19：**B**。触发者是主体变化；Pull 指观察者如何取得数据，不是反复主动轮询。

A20：**D**。变换式 Hook 的返回属于数据流；忽略它会丢失子类扩展效果。

12.3 Section B：跨章节设计审查

12.3.1 B1 题库平台的职责与契约

某 QuizHub 同时读取 JSON、筛选题目、生成报表、发送邮件，并保存全局连接。新增“最近 N 天使用过的题目不能选”后，CLI 和网页端各自实现一次相同规则。其 getter 返回内部题目字典列表，导入异常统一返回空列表。

请完成四项分析：说明至少三种独立风险；给出聚焦的职责划分；指出哪些问题仅靠“加入一个 Facade”无法解决；说明重构前后的行为验证。

参考分析。第一，导入、选择规则、呈现和通知受不同需求驱动，集中在 Hub 造成多种变化原因。第二，重复 cooldown 知识可能在 N 天边界或缺失日期处理上分歧，属于 DRY 风险。第三，返回内部集合让客户绕过验证修改状态；复制外层列表仍不能保护内部字典。第四，吞异常使读取失败与正常空结果无法区分。第五，全局连接产生隐藏的生命周期和共享状态耦合。

合理划分可以是 ImportService 负责导入流程，EligibilityRule 负责选择契约，ReportGenerator 负责报表流程，Notifier 负责通知；数据存储通过抽象提供。Facade 可协调一次高层用例，但不应把底层计算、所有格式和异常吞掉后都塞进同一个类。

重构验证先记录约定行为，包括正常导入、合法空数据、格式错误、缺失文件、规则首因和排序。若旧程序存在已确认缺陷，应明确哪些属于修复，不假称所有外部行为都原样保持。回归测试检查应保留的行为，新增测试检查新规则与错误修正。

12.3.2 B2 可变形状与“现实世界继承”

绘图工具让 Ellipse 独立设置两条半轴，Circle 必须保持两轴相等。产品经理希望二者共同接受 draw 和 area，开发者因此让 Circle 继承全部 Ellipse setter。

任务。说明真正的冲突，提出两个可行方向，并指出它们的权衡。

参考分析。冲突不是“圆到底是不是椭圆”，而是独立改变一条半轴是否保证另一条不变。Circle 若同步修改两轴，会改变客户预期；若拒绝某些父类允许的设置，也可能加强前提。

方向一是用共同的只读 Shape 契约提供 draw/area，各自拥有合理的尺寸修改 API。方向二是采用不可变值，通过不同构造或转换创建新形状，避免继承不相容的 setter。前者保留明确能力但客户可能需要区分编辑操作；后者更易维护不变式但会产生新对象，并要求调用方接受不可变模型。二者都需要测试客户真正依赖的行为。

12.3.3 B3 代码审查意见的质量

一个 reviewer 只写“这个 Adapter 太长，违反 SOLID”。另一个写“results 中将百分比直接当 earned 返回，使 Target 的 earned/possible 契约改变；建议集中转换并用 50% × 权重 20 = 10 的用例验证”。

参考分析。第二条给出位置、语义证据、受损契约和验证方法，更可执行。长度可以提示异味，却不是充分结论。审查不要求每个问题都贴一个原则名；准确指出单位或状态错误通常比堆砌术语更有价值。

12.4 Section C：模式选择、结构与比较

12.4.1 C1 一个持续演进的测评系统

需求如下：读取 JSON/CSV/JSONL，复用相同导入后校验；规则可以嵌套组合并显示表达式；审核操作可撤销；成绩可能来自原生分数或外部百分比导出；完成测评后多个独立面板需更新；报表读取和输出格式不同但流程稳定。

任务。为每项变化选择一种课程模式，解释角色对应和调用顺序；指出至少一种不必引入模式的情况。

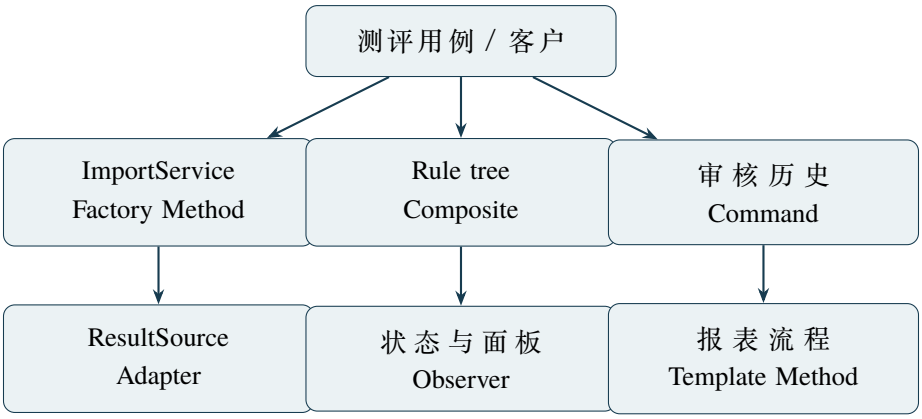


图 **20:** 综合系统的职责地图（非 UML）：各模式保护不同变化方向，不要求合并为一个大类。

参考设计。导入使用 Factory Method：ImportService 保留公共工作流，具体 service 创建 importer 产品。若只是固定格式到函数的映射且没有创建者扩展需求，简单工厂也可能足够，不能因“格式多”就强制增加层次。

规则使用 Composite：Rule 是 Component，具体校验为叶子，AllOf/AnyOf 等为递归组合。若添加 AnyOf，必须单独定义错误合并与渲染语义；不是直接复制 AND 字样就完成。

审核使用 Command：命令保存题目标识、Receiver 与旧状态，Session 触发并维护成功历史。成绩接入使用 Adapter：ResultSource 为 Target，外部导出为 Adaptee，适配器把百分比与权重换算为 earned/possible。

面板更新使用 Observer：测评状态主体管理订阅，只调用共同 update；面板可独立增加和退订。报表流程使用 Template Method：基类固定读取、处理、格式化和导出步骤，具体报表实现变化步骤与必要 Hook。

不是所有方法都要有模式。两个固定数字转换、一条无变化的简单计算，直接写清楚通常更好。判断应比较变化频率、重复知识和间接层成本。

12.4.2 C2 同一包装结构的四种需求

一个包装器 W 持有对象 X 并向它转发。分别给出以下需求中最主要的模式意图：延迟首次下载；按需附加压缩；把旧方法及单位变成客户所需形式；依序寻找首个可以处理工单的对象。

参考分析。分别为 Proxy、Decorator、Adapter、Chain of Responsibility。结构相似的充分说明必须同时包含客户问题与转发语义：Proxy 决定是否访问，Decorator 增加职责，Adapter 翻译契约，责任链分配处理机会并可能终止。

如果包装器同时检查权限并记录日志，可以同时包含 Proxy 与装饰职责；答题应分开解释，不要因存在多种作用就放弃精确分析。

12.4.3 C3 Strategy 与 Template Method 的替换成本

系统有固定“读数据 → 排序 → 输出”流程。客户希望同一对象在运行时切换升序 / 降序，未来还会有按评分优先等规则。另一部分系统希望不同报表在固定流程中采用不同读取方式。

参考分析。排序算法适合 Strategy，对象保存 callable 或策略对象，客户可以替换；读取方式若与报表类型及稳定步骤关联，可作为 Template Method 的抽象步骤。也可以在模板中注入 Strategy。选择不能仅以“有算法”决定，而要看变化粒度、生命周期和是否运行时替换。

测试应分别验证公共顺序、策略切换后的结果、各具体算法边界，以及某步骤失败时后续动作是否执行。模式组合增加可替换性，同时增加装配与契约测试成本。

12.5 Section D：软件测试综合分析

12.5.1 D1 被测规格与程序

以下程序为自编题，借用课程的 Composite 与 Adapter 主题。规定每个叶子满足 $0 \le \text{earned} \le \text{possible}$ 且 possible>0；Section 是无环树，可为空。外部 percent 值为 0-100，权重为正数；缺失提交得 0 分。没有题目时 percentage 为 None。构造器假设调用方满足这些前置条件，本题不要求补写验证。

PYTHON

```
class QuestionResult:
    def __init__(self, earned, possible):
        self.earned, self.possible = earned, possible

    def totals(self):
        return self.earned, self.possible

class Section:
    def __init__(self, children):
        self._children = tuple(children)

    def totals(self):
        earned = 0
        possible = 0
        for child in self._children:
            child_earned, child_possible = child.totals()
            earned += child_earned
            possible += child_possible
        return earned, possible
```

```
class PercentageExport:
    def __init__(self, weights, submissions):
        self.weights, self.submissions = weights, submissions

    def download(self):
        return self.weights, self.submissions

class PercentageAdapter:
    def __init__(self, export):
        self._export = export

    def results(self):
        weights, submissions = self._export.download()
        output = []
        for question_id, weight in weights.items():
            if question_id in submissions:
                earned = submissions[question_id] * weight / 100
            else:
                earned = 0
            output.append(QuestionResult(earned, weight))
        return tuple(output)

def summarise(node):
    earned, possible = node.totals()
    if possible == 0:
        percentage = None
    else:
        percentage = 100 * earned / possible
    return {"earned": earned, "possible": possible,
            "percentage": percentage}

def build_report(source):
    return summarise(Section(source.results()))

source = PercentageAdapter(PercentageExport(
    {"Q1": 20, "Q2": 80}, {"Q1": 50, "Q2": 100}
))
assert build_report(source) == {
    "earned": 90.0, "possible": 100, "percentage": 90.0
}
```

问题一：模式与角色。识别 Component 协议、Leaf、Composite、Target、Adapter 和 Adaptee。为何不能按“保存一个对象”认定 PercentageAdapter 是 Decorator?

参考答案。 totals 是 Component 的鸭子类型契约，QuestionResult 是 Leaf，Section 是 Composite；results 是客户使用的 ResultSource 式 Target 协议，PercentageAdapter 实现它，PercentageExport 是 Adaptee。后者暴露 weights/submissions 的 download，而非同一 results 契约；包装的目的在于接口和单位翻译，不是给已兼容对象附加职责。

12.5.2 D2 从一条成功测试推导覆盖缺口

问题二。只执行上述最后的 source/build_report 测试。哪些重要行为还缺证据?

参考答案。 Adapter 的“未提交”分支没有执行；report 的 possible==0 为真没有执行；没有直接检查空 Section、嵌套 Section、单叶子结果和返回顺序。非空循环运行时通常会经过迭代和正常退出，但这不等于测过从一开始就为空的情形。应区分工具分支覆盖与空输入的黑盒语义。

该样例 total=100，也可能掩盖一些错误。若实现错误地把 earned 当作 percentage，可能恰好得到相同值。应再选总权重不为 100 的案例。

问题三。设计一组小而有理由的测试，覆盖上述行为并采用有揭露能力的断言。

参考测试。

PYTHON

```
# Continue with the classes and functions in D1.
empty = Section([])
assert empty.totals() == (0, 0)
assert summarise(empty) == {
    "earned": 0, "possible": 0, "percentage": None
}

nested = Section([
    QuestionResult(2, 5),
    Section([QuestionResult(3, 5), Section([])])
])
assert nested.totals() == (5, 10)
assert summarise(nested)["percentage"] == 50

mixed = PercentageAdapter(PercentageExport(
    {"Q1": 20, "Q2": 30}, {"Q1": 50}
))
records = mixed.results()
assert [(r.earned, r.possible) for r in records] == [
    (10.0, 20), (0, 30)
]
assert build_report(mixed) == {
    "earned": 10.0, "possible": 50, "percentage": 20.0
}

zero = PercentageAdapter(PercentageExport({"Q1": 20}, {"Q1": 0}))
assert build_report(zero)["percentage"] == 0
full = PercentageAdapter(PercentageExport({"Q1": 20}, {"Q1": 100}))
assert build_report(full)["percentage"] == 100
no_questions = PercentageAdapter(PercentageExport({}, {}))
assert build_report(no_questions)["percentage"] is None
```

这些断言检验金额式单位换算、分母、缺失与零的区别、空结构和递归合并。对于一般浮点小数，可依误差契约用容差比较；这些选值能得到精确的简单结果，便于推导。

12.5.3 D3 黑盒分区与 RIPR 反思

问题四。在既定合法输入域内，percent 的重要分区或边界是什么？若要求 Robust，还需补充哪些前提？

参考答案。检查 0、100、内部值及提交缺失；缺失与显式 0 的得分相同，却可能走不同结构路径。权重及题目数量也影响总分，至少包含空集合、单题与多题，总权重选非 100 以暴露混淆。Robust 需先规定负值、>100、非数字、非正权重的处理；当前构造器只声明前置条件，不能未经授权假定它会抛某种异常。

问题五。若 Adapter 漏了 /100，而测试只断言 len(records)==2，RIPR 如何解释其通过？

参考答案。乘法位置被执行（Reachability），earned 变成错误值（Infection），错误值存入结果并可能传播到报表（Propagation）；长度断言不观察得分差异（缺少 Reveal）。100% 分支覆盖也不改变这个事实。

问题六。团队认为只测 Section.totals 就足以检查 Adapter 和整个 CLI。为什么推理不成立？

参考答案。 Section 的单元结果只说明聚合契约在给定孩子下成立。Adapter 还可能错误转换输入，CLI 还可能选错数据、吞异常或格式化错误。应分层设置单元与集成 / 系统证据，并用回归测试保护重构后的用户行为。

12.6 综合复习的最后检查

能够不用看答案解释每个模式的问题、角色、执行路径和至少一个代价；能够给出 LSP、ISP、DIP 的不同反例；能够从规格选等价类和边界，从实现列未覆盖结果，再用 RIPR 评价断言强度。

若一道设计题有多个合理方案，明确前提并比较修改影响；若规格不足，指出需要澄清的行为；若资料没有提供结论，就保留范围边界。考试准备的目标是可解释、可验证的判断能力。

第 13 章　资料索引、覆盖核对与扩展依据

13.1　本地资料及归属依据

L1-L10 分别对应知识地图中列出的十份 PDF。页码一律指原 PDF 的物理页序，不受幻灯片内部偶有缺号或重复编号影响。所有页面已读取文字，并对整套课件渲染检查；代码截图及图示页结合原图与 OCR 核对。OCR 仅作辅助，歧义处以原图和语言语义为准。

行政通知、课堂休息页、问候、反馈调查、过期截止时间和重复构建动画没有逐字收入正文；其涉及的实际学习目标、考试结构或实质性反馈已归入相关章节。课程案例经过教学重写的代码，均在正文说明；本书图示均为依据已读内容重新绘制的矢量图。

来源	实质性教学内容及原页范围	本书对应
L1，44 页	3-23 工程活动、事故、AI 责任与设计成本；25-35 课程技术目标；36-43 学习组织	Week 1；Ed Python 内容一并重构
L2，69 页	7-8 注释与代码；12-29 类图；30-55 顺序图；57-68 审查、异味、全局状态与 property	Week 2 的完整课件内容
L3，37 页	10-16 质量与协作；17-26 最少原则与封装；27-35 模式、分类与创建；37 语言表面语法	Week 3；Factory Method 补读 Ed
L4，69 页	7-11 接口与客户；14-48 Composite；49-68 Command、撤销与实际案例	Week 4
L5，77 页	9-32 Decorator；33-55 Proxy 与 N+1；56-77 Singleton 及并发、初始化与代价	Week 5
L6，60 页	3-12 幂等、代码审查反馈与 Singleton 边界；14-39 Adapter；40-59 Facade	Week 6
L7，68 页	2-5 数值缺陷；6-39 测试基础、RIPR、层级与假设；40-68 执行结构、覆盖与包含关系	Week 7
L8，73 页	7-8 工具口径；9 考试结构；10-19 时间；20-47 黑盒；48-72 Iterator 与补充跳转	Week 8，部分工具内容并入 Week 7
L9，82 页	4-5 Trial 结构说明；9-23 责任链；25-63 Observer；65-82 Strategy	Week 9
L10，79 页	3-6 作业与 DRY；9-31 Template Method 与 IoC；32-78 全模式比较；79 后续预告	Week 10 与综合复习；预告明确不计为已覆盖后续教学

Assignment 1。5 页说明、CODING_STANDARD.md、README.md、core/session/cli、测试及本地 answers.md 已用于理解题库案例。该 answers 文件是现有学习资料，不被当作官方答案。技术主题包括不可变常量、可变默认值、开闭、封装、文档同步和异常路径。

Assignment 2。4 页说明、before_refactoring 的说明与旧设计、当前 rules/importers/commands/session 及相应测试用于核对模式角色与行为。教材不修改任何作业源文件，不把现有实现视为无需审查的通用生产方案。

Assignment 3。3 页说明、FizzBuzz、quiz_results、result_adapters 与覆盖率脚本用于核对测试目标和工具口径。当前测试及 coverage.json 可能已包含学习者修改，不将其数值宣称为 starter 原始覆盖。

目录中的虚拟环境、包安装元数据、缓存和与课程无关的独立脚本不被误认作讲义，也没有被修改或删除。没有发现无法读取的本地课程 PDF。没有本地 Assignment 4 原卷及 Week 11-13 讲义；不据此猜补缺失内容。

13.2 补读的 Ed 课程正文

以下为已访问的课程页面，可点击打开；访问权限由课程平台管理。

E1：Week 1: Brushing up on Python，13 页，包含控制流、异常、类型、集合、相等性、函数、陷阱、惯用法、对象及鸭子类型。https://edstem.org/au/courses/36363/lessons/109322

E2：Week 2: Recap of OOP，17 页，包含对象语义、dunder、继承、多重继承、访问约定、ABC、staticmethod、property、组合、Demeter、DRY、SOLID 与 Circle-Ellipse。https://edstem.org/au/courses/36363/lessons/109324

E3：Week 3: Introduction to Design Patterns，19 页，包含质量、最少原则、模式、驻留、简单及静态工厂、Factory Method、披萨及 Django。https://edstem.org/au/courses/36363/lessons/109328

E4：Week 4: Composite Pattern，8 页，包含文件系统、任务组、表达式树、角色、图示与 SOLID 权衡。https://edstem.org/au/courses/36363/lessons/109332

E8：Week 8: Iterators，6 页，包含菜单遍历、GoF 迭代接口、Python 惯用法、itertools 与生成器。https://edstem.org/au/courses/36363/lessons/109350

E-A4：Assignment 4: Trial Exam 的 Introduction 与 FAQs，核对分节结构和资料边界；原卷附件没有纳入本地资料或作为本书答案来源。https://edstem.org/au/courses/36363/lessons/120904

13.3 扩展资料的精确用途

本书的教学主体来自课程资料。以下一手来源只支持标为扩展的规范、语言细节和工具语义，不据此扩大课程考查范围。链接于 2026 年 10 月 22 日核对。

R1：OMG，Unified Modeling Language 2.5.1。用于 UML 标准来源及激活、关系语义的澄清。https://www.omg.org/spec/UML/2.5.1/

R2：Python PEP 318，Decorators for Functions and Methods。用于 @ 语法的设计动机、位置与应用顺序；不据此将所有 Python 装饰器等同于 GoF Decorator。https://peps.python.org/pep-0318/

R3：Coverage.py，Branch coverage measurement。用于行间分支、合并百分比与结构上不完整分支的工具说明。https://coverage.readthedocs.io/en/latest/branch.html

R4：Python，time 模块文档。用于 monotonic 的单调计时语义与差值用途。https://docs.python.org/3/library/time.html

R5：Python，Built-in Types。用于相等性、真值、短路和迭代协议的语义核对。https://docs.python.org/3/library/stdtypes.html

13.4 教材自身的验证边界

书中 40 段 Python 代码完成语法检查，其中 38 段独立可运行示例已逐段执行；综合 D2 的续接测试也在 D1 上下文中执行通过。示例中刻意保留的错误均用于说明反例，并在正文注明。文件读取例只作语法及逻辑检查，未宣称验证了所有真实文件和外部服务。

当前环境没有 Java Runtime/JDK，Java 中点示例没有实际编译运行；其语法经人工检查，两个结果已按 32 位有符号算术模拟核对。教学替身的通过不能推导真实数据库、网络、支付或框架集成已验证。

13.5 阅读完后应能够完成的任务

能用本书给出的明确契约推导代码，而不只凭注释；能从维护变化说明设计原则；能依据角色与意图识别全部 13 个模式；能画出基本类关系与场景顺序；能从规格选分区和边界，从代码指出覆盖缺口，并通过具体断言揭露错误。

本书提供系统学习 Week 1-10 及进行 Trial 结构练习所需的理论与分析方法。它不以未知后续课件、当前成绩、未运行的真实外部系统或自编题的表现来承诺实际考试结果。